



KUBERNETES

101

Özet

Bu doküman, Kubernetes'in mimarisi, bileşenleri, kurulum yöntemleri, konfigürasyon adımları ve pod yaşam döngüsünden deployment, namespace ve rollout süreçlerine kadar temel kavramları kapsayan teknik bir başlangıç rehberidir.

Sema Sena Kürüm

İçindekiler Tablosu

1. Kubernetes Mimarisi	3
1.1 Container orchestration	3
1.2 Container Tarihçesi	3
1.3 Kubernetes Nedir?	4
1.3.1 Kubernetes komponentleri -1	4
1.3.2 Kubernetes komponentleri -2	4
2. Kubernetes Kurulum	9
2.2 Kubectl Kurulumu	9
2.8 Kubectl ile Kubernetes Cluster Kurulumu (Worker&Master)	17
2.9 containerd kurulumu (Master & Worker)	19
2.10 kubectl Kurulumu (Master & Worker)	20
2.10 kubernetes cluster kurulumu	21
3. Kubectl config	26
3.1 kubectl config dosyası	27
4. kubectl kullanımı	29
5. Pod	34
5.1 Pod Oluşturma	34
6. YAML	37
6.1 Pod Yaşam Döngüsü	40
6.2 Pod Yaşam Döngüsü Uygulama	42
7. Multi Container Pod	46
7.1 Çoklu Container Pod	46
8. Init Container	50
9. Label ve Selector	55
10. Annotation	61
11. Namespace	61
12. Deployment	64
13. ReplicaSet	68
14. Rollout ve Rollback	70
15. Kubernetes Ağ Altyapısı	76
16. Service	79

17. Liveness probes.....	90
18. Readiness probes	92
19. Resource limits	93
20. Environment variable.....	95
KAYNAKLAR.....	98

1. Kubernetes Mimarisi

1.1 Container orchestration

Bir e-ticaret uygulamasını frontend, backend, cache ve veritabanı olmak üzere dört ayrı modüle ayırarak her birini ayrı konteynerlerde çalıştırdığımızı düşünelim. Başlangıçta Docker ile tek bir sunucu üzerinde her şeyi manuel olarak yapılandırmak yeterli gibi görünüyordu. Ancak sistem büyüdükçe yeni makineler, load balancer, ağ ayarları derken iş yükü ve karmaşa ciddi şekilde arttı.

Konteyner sayısı artınca, hangi servis nerede çalışıyor, IP adresi nedir, kim kiminle konuşacak gibi bilgileri takip etmek bile ayrı bir efor gerektirmeye başladı. Süreçleri manuel yönetmek zamanla sürdürülebilir olmaktan çıktı.

Tam bu noktada devreye **container orchestration** sistemleri giriyor. Uygulamanın bileşenleri, replikasyon ihtiyaçları ve bağlantı kuralları tanımlandıktan sonra, bu sistemler tüm kaynakları otomatik olarak yönetiyor. Gerekli yerlerde yeniden başlatma, yük dağıtımı, hata toleransı gibi işleri bizim yerimize üstleniyor.

Bugün bu alanda en çok bilinen araçlardan biri **Docker Swarm**. Ancak sektörde en yaygın kullanılan ve standart haline gelen sistem, açık kaynak olarak geliştirilen **Kubernetes**.

1.2 Container Tarihçesi

Container kavramı hayatımıza 2013 yılında Docker ile girdi. Ancak bu teknolojinin temeli çok daha eskiye dayanıyor. Linux dünyasında **cgroups** ve **namespace** gibi çekirdek özellikler sayesinde container yapıları uzun süredir biliniyor ve kullanılıyordu. Özellikle Google, bu yapıları kendi üretim ortamlarında yıllardır aktif olarak kullanan firmalardan biri.

Google, altyapısını daha verimli yönetebilmek için ilk olarak **Borg** adında kapalı kaynak bir konteyner orkestrasyon sistemi geliştirdi. Borg, büyük ölçekte konteynerleri yönetebilmek adına yıllar içinde olgunlaştı. Ancak ihtiyaçlar değiştikçe Borg'un yanında daha esnek ve geliştirici odaklı bir sistem olarak **Omega** geliştirildi. Omega, merkezi bir veri deposu etrafında çalışan dağıtık bileşenlerden oluşuyordu ve yazılım geliştirme süreçlerini daha iyi destekliyordu.

Her iki sistem de Google'ın yıllar boyunca edindiği operasyonel tecrübelerin bir ürünüydü. Bu bilgi birikimiyle, Google mühendisleri **Craig McLuckie**, **Brendan Burns** ve **Joe Beda**, yeni nesil bir konteyner orkestrasyon sistemi için 2014 yılında bir prototip oluşturdu. Proje kısa sürede açık kaynak olarak yayınlandı. Bu sayede, topluluk katkılarıyla çok hızlı bir şekilde gelişmesi hedeflendi.

İsim olarak, Docker'ın denizcilik temasına uygun şekilde, Yunanca'da "dümenci" anlamına gelen **Kubernetes** seçildi. Yazımı kolaylaştırmak için de kısaca **k8s** ifadesi kullanılmaya başlandı.

2015'te Kubernetes, 1.0 sürümüne ulaştı ve Google tarafından **CNCF**'ye (Cloud Native Computing Foundation) devredildi. CNCF, Linux Vakfı'nın bir alt kuruluşu olarak bulut tabanlı açık kaynak projeleri destekleyen ve yöneten kâr amacı gütmeyen bir organizasyondur. Bugün Kubernetes başta olmak üzere birçok önemli projenin resmi yöneticiliğini yapmaktadır.

1.3 Kubernetes Nedir?

Kubernetes, container tabanlı uygulamaların yönetimini kolaylaştırmak amacıyla geliştirilmiş, taşınabilir ve genişletilebilir açık kaynak bir platformdur. Temel olarak, container iş yüklerini ve servisleri otomatikleştirme üzerine odaklanır.

Sistem, tamamen modüler bir mimariyle tasarlanmıştır. Tüm bileşenler görevlerine göre ayrılmıştır ve her biri kendi sorumluluğunu üstlenir. Bu yapı sayesinde hem yönetim kolaylaşır hem de sistem esnekliğini korur.

Kubernetes, adım adım ne yapılacağını belirtmek yerine, nasıl bir yapı istendiğini tanımlamaya olanak tanır. Yani kullanıcı, doğrudan işlemleri tarif etmek yerine hedef durumu tanımlar. Sistem bu tanımları temel alarak gerekli işlemleri kendi yürütür.

Deklaratif bir modelle çalıştığı için sürekli olarak mevcut durumu kontrol eder. Tanımlanan yapıdan bir sapma olursa, Kubernetes durumu analiz eder ve istenilen yapılandırmaya otomatik olarak geri döner.

1.3.1 Kubernetes komponentleri -1

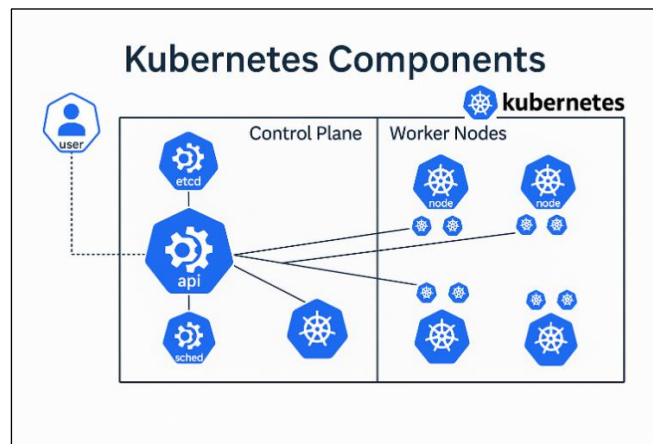
Geleneksel yazılım mimarilerinde, uygulamalar genellikle tüm işlevselliği tek bir yapı altında barındırır. Veritabanı dahil olmak üzere tüm bileşenler aynı uygulama içerisinde yer alır, hatta çoğu zaman gömülü veritabanı kullanılır. Bu yaklaşım başlangıçta işleri kolaylaştırırsa da, zamanla bakım, ölçekleme ve geliştirme süreçlerinde ciddi zorluklar yaratır.

Bunun yerine günümüzde daha yaygın hale gelen yaklaşım, uygulamanın işlevsel parçalarını **bağımsız servisler** olarak tasarlamaktır. Yani her iş mantığı kendi içinde izole edilmiş servislerde çalışır. Bu servislerin bir araya gelmesiyle ortaya çıkan yapı artık sadece bir uygulama değil, bir platform halini alır.

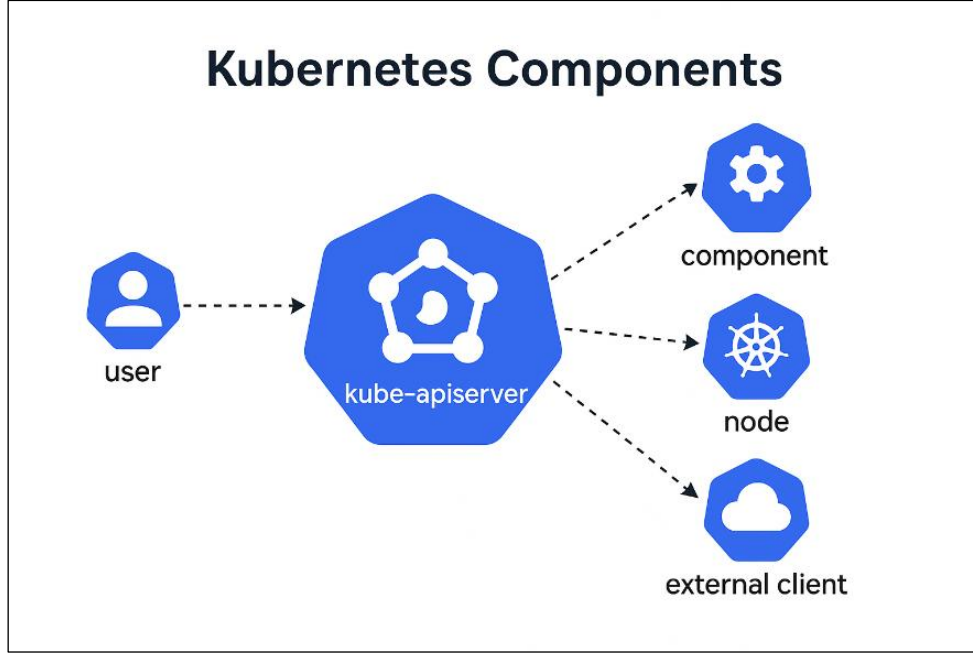
Bu model, esneklik, güncelleme kolaylığı ve yatayda genişleyebilme açısından ciddi avantajlar sunar. Yani aslında burada **mikroservis mimarisinden** söz ediyoruz.

Kubernetes de tam olarak bu yaklaşıma uygun şekilde tasarlanmış bir platformdur. Farklı işlevleri olan container'ları, esnek ve otomatik şekilde yönetebilmek için geliştirilmiştir.

1.3.2 Kubernetes komponentleri -2



a. kube-apiserver



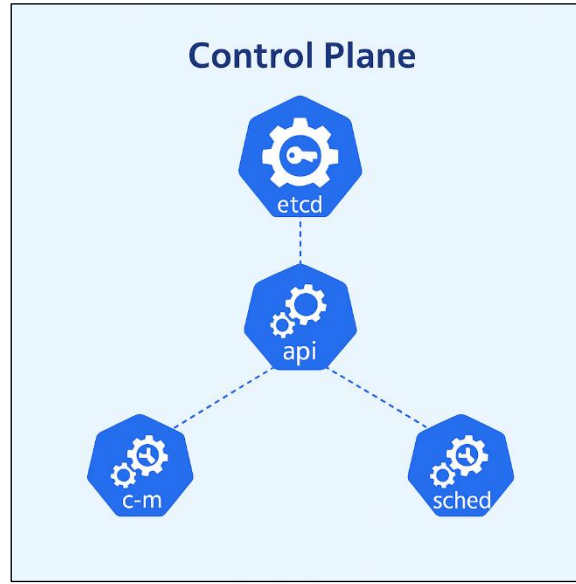
Kubernetes'in merkezi kontrol noktası olan **kube-apiserver**, hem iç bileşenlerin hem de dış servislerin Kubernetes ile iletişim kurmasını sağlayan ana giriş kapısıdır. Kısacası, kümenin "beyni" gibi çalışır.

Tüm bileşenler – ister Kubernetes içindeki node'lar olsun, ister harici istemciler – kube-apiserver üzerinden haberleşir. Cluster içerisindeki diğer komponentler doğrudan birbirleriyle değil, sadece apiserver aracılığıyla iletişim kurar.

Kullanıcılar kubectl gibi komut satırı araçları ya da REST API çağrılarıyla bu sunucuya istek gönderir. Apiserver bu istekleri karşılar, authentication ve authorization süreçlerini yürütür, ardından istekleri uygun şekilde yönlendirir.

Özetle Kubernetes'e erişmek, kaynak oluşturmak ya da bir aksiyon tetiklemek istiyorsanız, tüm yol kube-apiserver'dan geçer.

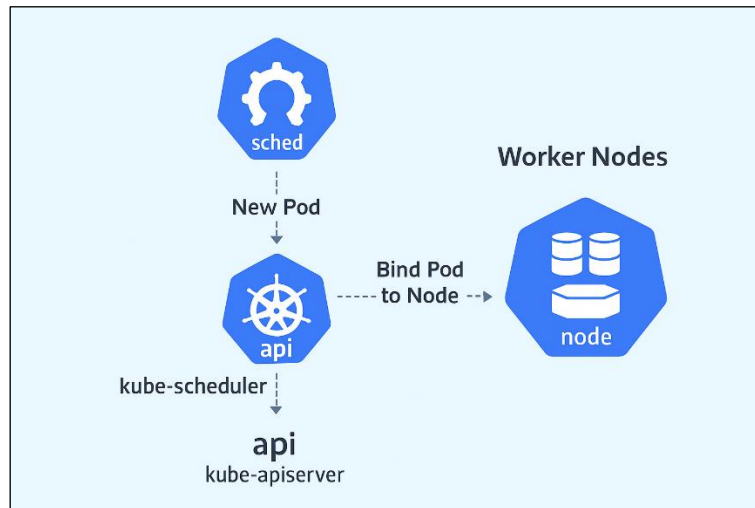
b. etcd



Kubernetes'in ikinci en önemli komponenti etcd tüm cluster verisi, metadata bilgileri ve kubernetes'de oluşturulan tüm objelerin bilgilerinin tutulduğu anahtar-değer "key-value" veri deposudur. Kısacası, kubernetes'in çalışması için ihtiyaç duyulan tüm veriler etcd üzerinde tutulur.

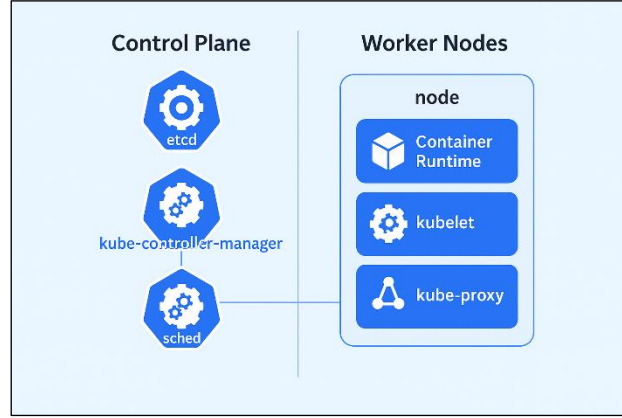
api-server hariç diğer kubernetes komponentleri etcd ile direkt haberleşemezler. etcd ile iletişim kurlmaları gerektiğinde bunu kube-apiserver aracılığıyla yaparlar.

c. kube-scheduler



kube-scheduler yaratılmak istenilen pod'ların gereksinimlerini kontrol ederek o podun en uygun çalışacağı worker node'un hangisi olduğunu tespit eder ve podun burada oluşturulmasını sağlar. Pod'u bir konteyner olarak düşünebiliriz. Scheduler pod'un kaynak isteğini, özel gereksinimlerini (örneğin cpu kaynaklarına ihtiyaç duyması gibi çeşitli parametreleri) göz önünde bulundurur ve bir seçme algoritması sayesinde pod için en uygun node'un hangisi olduğuna karar verir.

d. kube-controller-manager



Tek bir binary olarak bulunsa da içerisinde birden fazla controller barındırır. Kubernetes mevcut durumu istenilen durumla karşılaştırma temelinde uygulama yönetimi sağlar. controller-manager altındaki controller'lar kubernetes'in mevcut durumuyla ondan istenilen durum arasında fark olup olmadığını gözlerler. kube-apiserver aracılığıyla etcd'de saklanan cluster durumunu izler ve eğer mevcut durum ile istenilen durum arasında fark varsa bu farkı oluşturan kaynakları gerektiği gibi oluşturur, günceller veya silerek bu durumu eşitler.

Örneğin, siz kubernetes'e uygulamanızın 3 pod olarak çalışmasını istenilen durum olarak bildirdiniz ve kubernetes'de bunu gerçekleştirdi fakat bir şekilde 1 tanesi bir şekilde dış bir müdahale ile silindi. controller-manager bu durumu algılayarak hemen yeni bir pod daha yaratılmasını sağlar ve istenilen durum ile mevcut durumu eşitler.

Bu saydığımız dört component kubernetes'in yönetim kısmını oluşturur. Control plane olarak bilinen yönetim kısmı master node olarak adlandırdığımız sistemler üzerinde çalışır. Tüm bu component'ler tek bir linux yüklü sisteme kurularak oradan erişilebileceği gibi yüksek erişilebilirlik sağlaması adına birden fazla sisteme de kurulabilir. etcd tamamen bu yapıdan ayrı olarak konuşturılarak daha da yüksek erişilebilirlik hedeflenebilir ama genellikle etcd'de apiserver'ın koştugu yerde durur.

Örneğin 3 sunucu üzerinde 3 apiserver kurularak diğer component'ler bu sistemlere dağıtılabilir. Bu, üzerinde control plane component'lerin koştugu sistemlere cluster altında master node denir. Master node'lar sadece bu sistemleri barındırmak için bulunur. Özellikle production sistemlerde herhangi bir iş yükümüzü bu sistemler üzerinde çalıştırmayız. Bunlarda sadece cluster'ın yönetim altyapısını koştururuz.

İş yüklerimiz ise worker node'lar üzerinde çalışır. Worker node dediğimiz iş yüklerimizi yani pod'larımızı yani container'larımızı üzerinde çalıştırdığımız esas yerlerdir.

Bunlar üzerlerinde containerd ya da docker gibi bir container runtime barındıran ve cluster'a dahil edilmiş sistemlerdir.

Her worker node'da üç temel component bulunur:

1. Container Runtime:

Container'ların çalışmasını sağlayan en temel bileşendir. Varsayılan olarak uzun süre Docker kullanılmış olsa da, Kubernetes bazı teknik ve mimari nedenlerle Docker desteğini sonlandırmış ve onun yerine **containerd** gibi daha hafif ve doğrudan entegrasyon sağlayan bir container runtime'a geçiş yapmıştır.

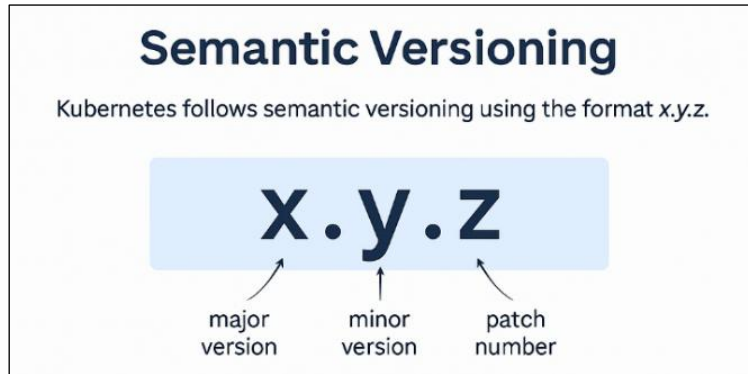
2.Kubelet:

Kubelet'de her worker node'da bulunur. **kubelet** apiserver aracılığıyla etcd'i kontrol eder ve scheduler tarafından bulunduğu node üzerinde çalışması gereken bir pod belirtildiyse kubelet bu pod'u o sistemde yaratır. containerd'e haber gönderir ve belirlenen özelliklerde bir container'ın o sistemde çalışmasını sağlar.

3. Kube-Proxy:

Her worker node'da çalışan bir diğer ve son component'de **kube-proxy**'dir. Çok çok basit şekliyle, oluşturulan pod'ların TCP, UDP ve SCTP trafik akışını yönetir. Ağ kurallarını belirler ve yönetir. Kısacası, network proxy görevini görür.

e. Semantic versioning



Kubernetes semantik versiyonlanmayı takip eder ve x.y.z şeklinde versiyonlanır.

X → majör versiyonu

Y → minörü

Z → patch numarasını belirtir.

Kubernetes topluluğu, yılda yaklaşık üç küçük (minor) sürüm yayımlar. Bu, her 15 haftada bir yeni bir sürüm anlamına gelir .

Her küçük sürüm, yayımlandıktan sonra yaklaşık 14 ay boyunca desteklenir:

- İlk 12 ay boyunca aktif destek sağlanır; bu dönemde düzenli olarak yama sürümleri yayımlanır.
- Sonraki 2 ay, bakım modu olarak adlandırılır ve yalnızca CVE'ler, bağımlılık güncellemeleri ve temel bileşen sorunları için yama sağlanır .

Detaylı bilgi için [buradan](#) okuyabilirsiniz.

2. Kubernetes Kurulum

Ortak Kurulum Adımları (Master ve Worker Node'lar İçin Geçerlidir)

- **Homebrew Kurulumu**
- **Homebrew ile Ubuntu Server'a Kubectl Kurulumu**
- **Docker Kurulumu**
- **Containerd Kurulumu**
- **Kubeadm Kurulumu**

2.1 Kubernetes yönetim araçları

Kubernetes cluster'lar üzerinden işlem yapmak istediğimiz zaman yani kubernetes'e komut vermek istediğimiz zaman bunu apiserver üzerinden yaparız. Bunu gerçekleştirebilmemizde 3 yöntemi vardır:

- 1) Rest API Server** → Bu yöntem genellikle uygulamalarımızın ya da script'lerimizin içinden programatik şekilde kubernetes ile iletişim kurmak istediğimiz durumlarda kullanılır.
- 2) GUI** → İstersek bir grafiksel arayüz ile de kubernetes'i yönetebilir onunla iletişim kurabiliriz.
- 3) kubectl** → kubectl bizlere shell üzerinden çalıştırıp kubernetes apiserver ile iletişim kurarak onu komutlar gönderebileceğimiz kubernetes projesinin resmi ctl aracıdır.

2.2 Kubectl Kurulumu

<https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/> adresinde komutlar içerir.

<https://kubernetes.io/docs/tasks/tools/> sekmesine gelindiği zaman Linux, MacOS ve Windows için ayrı ayrı kurulum adımları mevcuttur.

Fakat bunlardan önce paket manager'ları yüklemek gerekir sisteme.

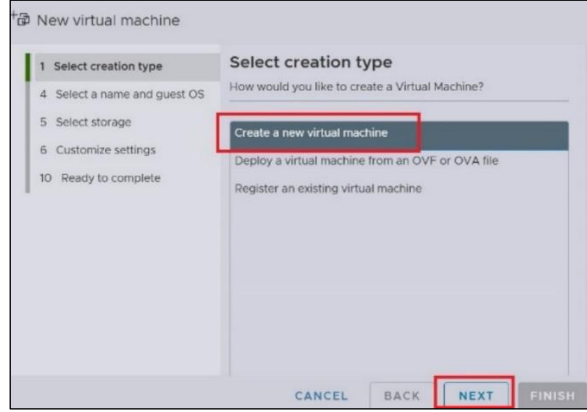
Windows için 3party araç → <https://chocolatey.org/install#individual>

MacOS için 3part araç → <https://brew.sh/>

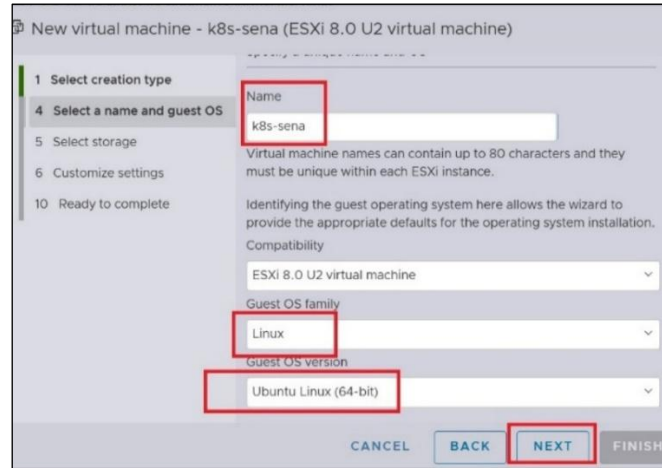
2.3 ESXI Üzerinden Ubuntu 24.04.2 Kurulumu

<https://ubuntu.com/download/server/thank-you?version=24.04.2&architecture=amd64&its=true> sitesinden Ubuntu Server .iso dosyası indirilir.

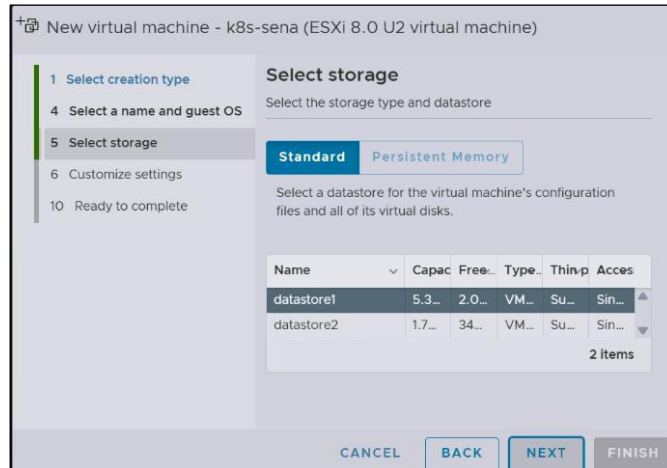
Create +New virtual machine butonuna tıklanır.



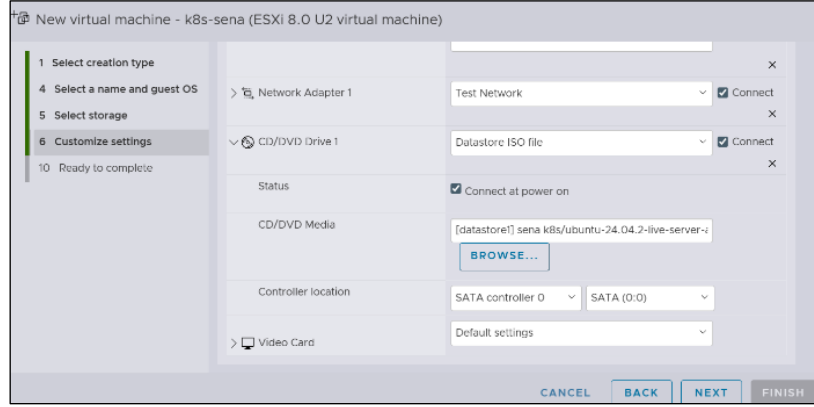
Sonrasında makineye isim verilir. İşletim sistemi ve OS version'u seçilir.



Disk seçilir ve Next deyip devam edilir.



Sonrasında .iso dosyamızı upload ettik ve seçtik.

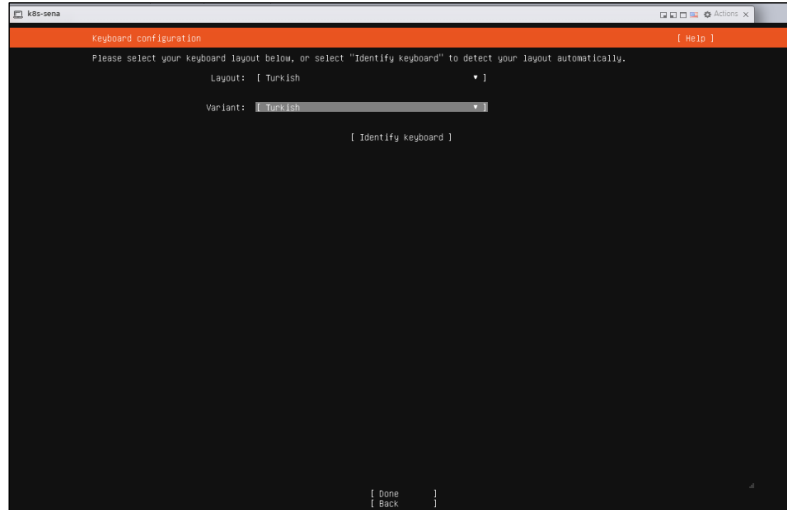


Sonrasında finish deyip kurulumu bitiriyoruz.

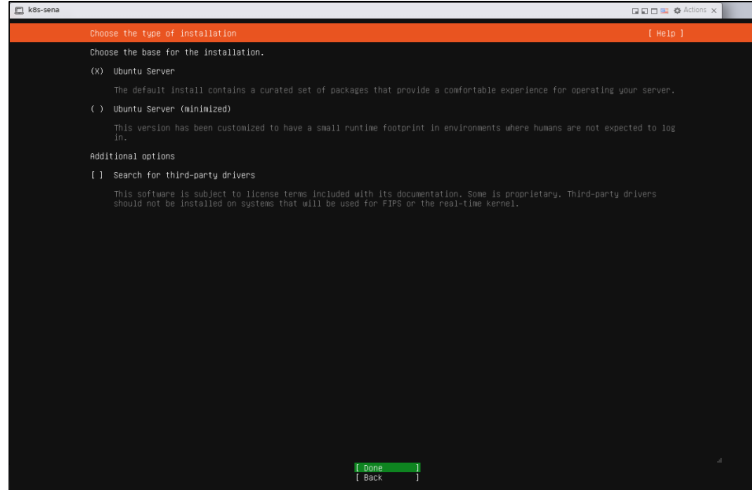


Daha sonra makinemi başlattım.

Dil ve keyboard seçimlerini yaptım.

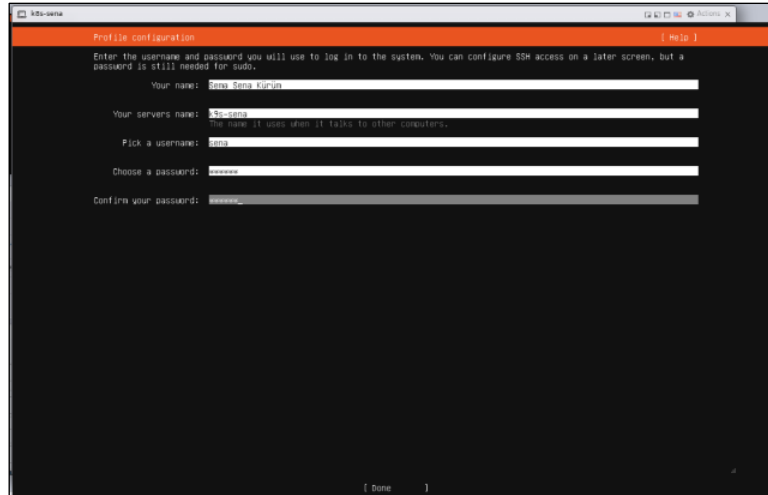


Ubuntu server seçili şekilde devam ediyorum.

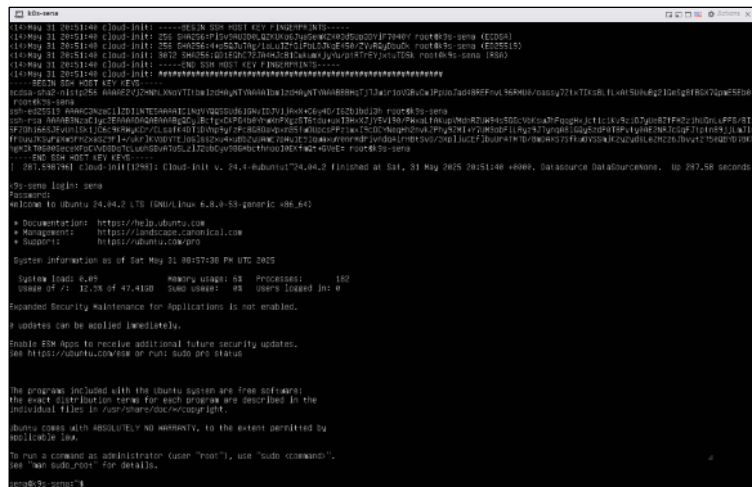


Diğer adımları da geçtikten sonra makineme isim veriyorum.

(Not: Yazım hatası sebebiyle "k9s" olarak yazılmıştır, doğrusu "k8s" olmalıdır.)



Daha sonraki adımları da tamamlayıp finish dedim ve kurulum başarı ile tamamlandı.



2.4 Homebrew Kurulumu (Master & Worker)

Terminal ekranına gelip `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"` yapıştırılır. Root şifresini soracaktır, root şifresini girdikten sonra kurulum başlar.

echo 'eval "\$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"' >> /home/sena/.bashrc
eval "\$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"

Bu sayede

- Homebrew dizinini path'e ekler.
- .bashrc dosyasına kalıcı olarak yazar, böylece her oturumda otomatik tanınır.

```
k8s-sena
/home/linuxbrew/.linuxbrew/sbin:/home/linuxbrew/.linuxbrew/share:/home/linuxbrew/.linuxbrew/var:/home/linuxbrew/.linuxbrew/zsh:/home/linuxbrew/.linuxbrew/share/zsh/site-functions:/home/linuxbrew/.linuxbrew/var/homebrew:/home/linuxbrew/.linuxbrew/Cellar:/home/linuxbrew/.linuxbrew/Caskroom:/home/linuxbrew/.linuxbrew/Frameworks
=> /usr/bin/sudo /bin/mkdir -p /home/linuxbrew/.linuxbrew/Homebrew
=> /usr/bin/sudo /bin/chown -R sena:sena /home/linuxbrew/.linuxbrew/Homebrew
=> Downloading and installing Homebrew...
remote: Enumerating objects: 302508, done.
remote: Counting objects: 100% (158/158), done.
remote: Compressing objects: 100% (112/112), done.
remote: Total 302508 (delta 61), reused 90 (delta 46), pack-reused 302350 (from 3)
remote: Enumerating objects: 55, done.
remote: Counting objects: 100% (33/33), done.
remote: Total 55 (delta 33), reused 33 (delta 33), pack-reused 22 (from 1)
=> Updating Homebrew...
=> Downloading https://ghcr.io/v2/homebrew/portable-ruby/portable-ruby/blobs/sha256:b5c9268815fbab7b7971307b2871f4206e0811
#####
=> Pouring portable-ruby-3.4.4.x86_64_linux.bottle.tar.gz
Warning: /home/linuxbrew/.linuxbrew/bin is not in your PATH.
Instructions on how to configure your shell for Homebrew
can be found in the 'Next steps' section below.
=> Installation successful!

=> Homebrew has enabled anonymous aggregate formulae and cask analytics.
Read the analytics documentation (and how to opt-out) here:
  https://docs.brew.sh/Analytics
No analytics data has been sent yet (nor will any be during this install run).

=> Homebrew is run entirely by unpaid volunteers. Please consider donating:
  https://github.com/Homebrew/brew#donations

=> Next steps:
Run these commands in your terminal to add Homebrew to your PATH:
  echo >> /home/sena/.bashrc
  echo 'eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"' >> /home/sena/.bashrc
  eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"
Install Homebrew's dependencies if you have sudo access:
  sudo apt-get install build-essential
For more information, see:
  https://docs.brew.sh/Homebrew-on-Linux
We recommend that you install GCC:
  brew install gcc
Run brew help to get started
Further documentation:
  https://docs.brew.sh

sena@k8s-sena:~$ echo 'eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"' >> /home/sena/.bashrc
sena@k8s-sena:~$ eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"
sena@k8s-sena:~$ brew --version
homebrew 4.5.4
sena@k8s-sena:~$
```

Başarı ile yüklendiğini test etmek için **brew --version** yazılır.

(Worker ve master için yapıldı.)

2.5 Homebrew ile Ubuntu Server'a Kubectl Kurulumu (Master & Worker)

brew install kubectl komutu ile yüklenir.

kubectl version --client komutu ile doğrulama sağlanır.

```
k8s-sena
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/8e11e90eb21a06e0f199af9d0e011e3693c77dd353b2477579d95c8471a5802--lz4-1.10.0-1.bottle_manifest
➔ Pouring lz4-1.10.0.x86_64_linux.bottle.1.tar.gz
➔ /home/linuxbrew/.linuxbrew/Cellar/lz4/1.10.0: 24 files, 798.6KB
➔ Installing gcc dependency: xz
➔ Downloading https://ghcr.io/v2/homebrew/core/xz/manifests/5.8.1
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/86a115cc1d43ff8a480fd907f812e70a403e1675d8a7223f61bb08cbbd2adc27--xz-5.8.1.bottle_manifest.
➔ Pouring xz-5.8.1.x86_64_linux.bottle.tar.gz
➔ /home/linuxbrew/.linuxbrew/Cellar/xz/5.8.1: 97 files, 2.6MB
➔ Installing gcc dependency: zlib
➔ Downloading https://ghcr.io/v2/homebrew/core/zlib/manifests/1.3.1
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/f68d0caf232d5f2aa586abefbfbfd7e958e384d84f3967008fa83de94b5f10ae--zlib-1.3.1.bottle_manifest
➔ Pouring zlib-1.3.1.x86_64_linux.bottle.tar.gz
➔ /home/linuxbrew/.linuxbrew/Cellar/zlib/1.3.1: 14 files, 476.0KB
➔ Installing gcc dependency: zstd
➔ Downloading https://ghcr.io/v2/homebrew/core/zstd/manifests/1.5.7
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/596a67b1677f62a0e246fd7c879891221cb826124f4355fb4deff1a2ec39--zstd-1.5.7.bottle_manifest
➔ Pouring zstd-1.5.7.x86_64_linux.bottle.tar.gz
➔ /home/linuxbrew/.linuxbrew/Cellar/zstd/1.5.7: 32 files, 2.9MB
➔ Installing gcc dependency: binutils
➔ Downloading https://ghcr.io/v2/homebrew/core/binutils/manifests/2.44
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/8dec3c476e7ac48bbb8a32ba3c2bc58546b09de0fb300ffe32425e1100f62285--binutils-2.44.bottle_manifest
➔ Pouring binutils-2.44.x86_64_linux.bottle.tar.gz
➔ /home/linuxbrew/.linuxbrew/Cellar/binutils/2.44: 6,462 files, 392.8MB
➔ Installing gcc
➔ Pouring gcc-15.1.0.x86_64_linux.bottle.tar.gz
Warning: The post-install step did not complete successfully
You can try again using:
  brew postinstall gcc
➔ Summary
➔ /home/linuxbrew/.linuxbrew/Cellar/gcc/15.1.0: 2,153 files, 591MB
➔ Fetching kubernetes-cni
➔ Downloading https://ghcr.io/v2/homebrew/core/kubernetes-cni/blobs/sha256:6f253773fdd67c1a68f07b59f0a924b5b88d3b091f22a6f9c7f377261688f3b4
➔ already downloaded: /home/sena/.cache/Homebrew/downloads/8dec3c476e7ac48bbb8a32ba3c2bc58546b09de0fb300ffe32425e1100f62285--binutils-2.44.bottle_manifest
➔ Pouring kubernetes-cni-1.33.1.x86_64_linux.bottle.tar.gz
➔ Caveats
bash completion has been installed to:
  /home/linuxbrew/.linuxbrew/etc/bash_completion.d
➔ Summary
➔ /home/linuxbrew/.linuxbrew/Cellar/kubernetes-cni/1.33.1: 255 files, 58.4MB
➔ Running 'brew cleanup kubernetes-cni...'
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see 'man brew').
➔ Caveats
➔ kubernetes-cni
bash completion has been installed to:
  /home/linuxbrew/.linuxbrew/etc/bash_completion.d
sena@k8s-sena:~$ kubectl version --client
Client Version: v1.33.1
Kustomize Version: v5.6.0
sena@k8s-sena:~$
```

2.6 Minikube Kurulumu (Alternatif Seçenek – Kubectl Kullanılmıyorsa Tercih Edilebilir)

Minikube, Kubernetes projesi kapsamında geliştirilen ve *Cluster Lifecycle Special Interest Group* (SIG) tarafından yönetilen bir yan projedir. Temel amacı, local ortamda single-node bir Kubernetes kümesi oluşturmaktır. (Birden fazla node'da eklenebilir) Minikube sayesinde, normalde yalnızca cloud sağlayıcılarında erişilebilen LoadBalancer türündeki servisler, bir eklenti (addon) aracılığıyla yerel ortamda da simüle edilebilir. Bunun yanı sıra, tek bir komutla cluster durdurulabilir ve yeniden başlatılabilir. Minikube; Docker, Hyper-V, KVM, VirtualBox gibi birçok sanallaştırma platformunu destekleyerek, farklı sistemler üzerinde cluster kurulumu için esneklik sağlar.

[minikube start](#) | [minikube](#) adresinde kurulum yönergeleri mevcuttur.

Minikube tek başına çalışmaz; çalışabilmesi için bir sanallaştırma platformunun da sisteme kurulmuş olması gerekir.

- Bunun için [Docker](#), [QEMU](#), [Hyperkit](#), [Hyper-V](#), [KVM](#), [Parallels](#), [Podman](#), [VirtualBox](#), or [VMware Fusion/Workstation](#) gibi bir container ya da virtual machine yöneticisine ihtiyaç duyulur.

Click on the buttons that describe your target platform. For other architectures, see [the release page](#) for a complete list of minikube binaries.

Operating system: **Linux** macOS Windows

Architecture: **x86-64** ARM64 ARMv7 ppc64 S390x

Release type: **Stable**

Installer type: Binary download **Debian package** RPM package

To install the latest minikube **stable** release on **x86-64 Linux** using **Debian package**:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
```

2.7 DOCKER KURULUMU (Master & Worker)

```
sudo apt update
sudo apt install -y docker.io
sudo usermod -aG docker $USER
newgrp docker
```

İşletim sistemi, mimari, sürüm türü ve kurulum biçimi seçenekleri seçilerek kullanıcıya uygun Minikube sürümü belirlenebilir.

```
sudo curl
LO https://storage.googleapis.com/minikube/releases/latest/minikube\_latest\_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
```

```
sena@k9s-sena:~$ minikube start
* minikube v1.36.0 on Ubuntu 24.04
* Unable to pick a default driver. Here is what was considered, in preference order:
* Alternatively you could install one of these drivers:
  - docker: Not installed: exec: "docker": executable file not found in $PATH
  - kvm2: Not installed: exec: "virqsh": executable file not found in $PATH
  - podman: Not installed: exec: "podman": executable file not found in $PATH
  - qemu2: Not installed: exec: "qemu-system-x86_64": executable file not found in $PATH
  - virtualbox: Not installed: unable to find VBoxManage in $PATH
X Exiting due to DRV_NOT_DETECTED: No possible driver was detected. Try specifying --driver, or see https://minikube.sigs.k8s.io/docs/start/
sena@k9s-sena:~$ _
```

Amacımız bir [Kubernetes cluster](#) oluşturmaktır. Bunun için **minikube start** komutunu çalıştırdığımızda, Minikube mevcut sistem üzerinde bir Kubernetes cluster başlatır ve gerekli yapılandırmaları yaparak **kubectl** aracıyla bu cluster'a erişip iletişim kurabilecek hale getirir.

```
k8s-sena
root@k9s-sena:~# curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 39.0M 100 39.0M 0 0 8306k 0 0:00:04 0:00:04 --:--:-- 9476k
root@k9s-sena:~# sudo dpkg -i minikube_latest_amd64.deb
selecting previously unselected package minikube.
(Reading database ... 122273 files and directories currently installed.)
Preparing to unpack minikube_latest_amd64.deb ...
Unpacking minikube (1.36.0-0) ...
Setting up minikube (1.36.0-0) ...
root@k9s-sena:~# _
```


Görselde görüldüğü gibi, **minikube start** komutu çalıştırıldığında sistemde herhangi bir sanallaştırma sürücüsü (Docker, KVM2, Podman, QEMU, VirtualBox) yüklü olmadığı için Minikube bir Kubernetes cluster başlatamıyor. Bu hatanın nedeni, Minikube'un çalışması için gerekli bir sanallaştırma altyapısının kurulu ve PATH değişkeninde tanımlı olmamasıdır. Sorunun çözümü için uygun bir driver kurulmalı ve **--driver** parametresiyle belirtilebilir.

Şimdi tekrar **--driver** parametresi vererek devam ediyorum.

minikube start --driver=docker

```
root@k9s-sena:/home/sena# minikube start --driver=docker
Error: unknown flag: --drive
See 'minikube start --help' for usage.
root@k9s-sena:/home/sena# minikube start --driver=docker
* minikube v1.36.0 on Ubuntu 24.04
* Using the docker driver based on user configuration
* The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
* If you are running minikube within a VM, consider using --driver=none:
*   https://minikube.sigs.k8s.io/docs/reference/drivers/none/

X Exiting due to DRV_AS_ROOT: The "docker" driver should not be used with root privileges.
root@k9s-sena:/home/sena# _
```

ALINAN HATA

Minikube'u docker driver'ı ile başlatmaya çalıştığımda, **root** kullanıcı ile çalıştırıldığı için hata aldım. Minikube, docker driver'ının root yetkileriyle kullanılmasına izin vermiyor; bu nedenle komutu normal kullanıcı ile çalıştırmam gerekti. Ardından **minikube start --driver=docker** komutunu tekrar dendiğimde, bu sefer de "permission denied while trying to connect to the Docker daemon socket" hatasıyla karşılaştım. Sena kullanıcısının Docker daemon'a erişim yetkisi olmamasından kaynaklanıyordu. Çözüm olarak, sena kullanıcısını docker grubuna şu komutla ekledim:

sudo usermod -aG docker \$USER && newgrp docker

Bu işlemin ardından minikube **start --driver=docker** komutunu tekrar çalıştırdım ve Kubernetes cluster başarılı bir şekilde oluşturulmaya başladı.

```
sena@k9s-sena:~$ minikube start --driver=docker
* minikube v1.36.0 on Ubuntu 24.04
* Using the docker driver based on user configuration
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.47 ...
* Downloading Kubernetes v1.33.1 preload ...
  > preloaded-images-k8s-v18-v1...: 347.04 MiB / 347.04 MiB 100.00% 5.92 Mi
  > gcr.io/k8s-minikube/kicbase...: 502.26 MiB / 502.26 MiB 100.00% 6.06 Mi
* Creating docker container (CPUs=2, Memory=2200MB) ...
* Preparing Kubernetes v1.33.1 on Docker 28.1.1 ...
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
sena@k9s-sena:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

minikube status komutu ile teyit edilebilir.

```
sena@k9s-sena:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE     VERSION
minikube    Ready     control-plane  7m12s   v1.33.1
sena@k9s-sena:~$
```

kubectl get nodes komutu ile görüldüğü üzere, Minikube başarılı bir şekilde bir Kubernetes cluster başlatmış ve kubectl aracıyla bu cluster'a erişim sağlanabilir duruma getirmiştir. Listelemede "minikube" isimli node'un Ready durumda olduğu ve control-plane rolüyle çalıştığı görülmektedir.

minikube delete komutu ile mevcut Kubernetes cluster tamamen silinir. Bu işlem, yapılandırmaları ve sanal makineyi sistemden kaldırır. Eğer yeniden bir cluster oluşturulmak istenirse, **minikube start** komutu kullanılarak sıfırdan yeni bir cluster başlatılabilir. Öte yandan, **minikube stop** komutu mevcut cluster'ı silmeden sadece duraklatır. Bu sayede daha sonra minikube start komutu ile kaldığı yerden tekrar çalıştırılabilir.

2.8 Kubeadm ile Kubernetes Cluster Kurulumu (Worker&Master)

Kubeadm, tıpkı Minikube gibi Kubernetes platformunun **Cluster Lifecycle Special Interest Group** tarafından yönetilen resmi bir alt projesidir. Kubeadm, **best practices** doğrultusunda, hızlı ve gerekli olan minimum konfigürasyonla Kubernetes cluster kurulumu sağlayan bir araçtır.

Birkaç makineyi ayağa kaldırıp, bu makineler üzerinde çalışan bir Kubernetes cluster oluşturmak istediğinizde kullanmanız gereken araç kubeadm'dir.

İki sanal makine oluşturacağız. Bunlardan biri Kubernetes **control plane** yani **master node** olarak çalışacak, diğeri ise **worker node** olarak yapılandırılacak.

Kurulum adımlarını içeren resmi sayfası: [Installing kubeadm | Kubernetes](#)

Kurulum adımlarını içeren Özgür Öztürk Github Reposu: [k8sfundamentals/setup at main · aytitech/k8sfundamentals · GitHub](#)

Esxi'a zaten önceden bir tane ubuntu server kurmuştum. Bir tane daha ayağa kaldırdım ve bunlardan biri master node, diğeri worker node olacak.

Kurulum adımları için her ikisine de aynı şeyleri yapmak şartı ile github reposundaki komutları takip ediyorum. (Master & Worker)

```
$ sudo modprobe overlay
```

```
$ sudo modprobe br_netfilter
```

```
$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
```

```
overlay
```

```
br_netfilter
```

```
EOF
```

```
sena@k9s-sena:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
> overlay
> br_netfilter
> EOF
overlay
br_netfilter
sena@k9s-sena:~$
```

```
$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
```

```
sena@k8s-master:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
> net.bridge.bridge-nf-call-iptables = 1
> net.bridge.bridge-nf-call-ip6tables = 1
> net.ipv4.ip_forward = 1
> EOF
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
sena@k8s-master:~$
```

\$ sudo sysctl --system ile ayarları kaydetmiş olduk.

Kubernetes (özellikle kubelet), **swap aktifken düzgün çalışmaz**. Bunun nedeni, kaynak yönetimi ve scheduler davranışlarının bozulmasıdır. Kubernetes, tüm RAM kullanımı üzerinde tam kontrol ister.

Bu adımların containerd'dan **önce** yapılması lazım çünkü Kubernetes, **swap belleğin devre dışı olmasını** şart koşar. (Master & Worker)

```
$ sudo swapoff -a
$ free -m
$ sudo sed -i 's/swap / s/^(.*)$/#\1/g' /etc/fstab
```

```
k8s-sena (worker)
sena@k9s-sena:~$ sudo swapoff -a
sena@k9s-sena:~$ free -m
              total        used         free      shared  buff/cache   available
Mem:           3914         1162          388           18         2657         2752
Swap:              0              0              0
sena@k9s-sena:~$ sudo sed -i 's/swap / s/^(.*)$/#\1/g' /etc/fstab
sena@k9s-sena:~$
```

Şimdi containerd kurulumuna geçeceğiz.

Kubernetes, **Container Runtime Interface (CRI)** uyumlu çeşitli container motorlarını destekler. Bunlar arasında en yaygın olarak kullanılanlar:

- **Docker**
- **containerd**
- **CRI-O**

Ancak günümüzde Kubernetes, doğrudan Docker'ı değil, onun arka planda kullandığı **containerd** bileşenini tercih eder.

2.9 containerd kurulumu (Master & Worker)

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
$ sudo apt install containerd.io
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now containerd
$ sudo systemctl start containerd
$ sudo mkdir -p /etc/containerd
$ sudo su -
$ containerd config default | tee /etc/containerd/config.toml
$ exit
$ sudo sed -i 's/          SystemdCgroup = false/          SystemdCgroup = true/'
/etc/containerd/config.toml
$ sudo systemctl restart containerd
```



```
sena@k9s-sena:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:5 https://download.docker.com/linux/ubuntu Codename: InRelease
Err:6 https://download.docker.com/linux/ubuntu Codename: Release
  404 Not Found [IP: 3.169.85.94 443]
Reading package lists... Done
E: The repository 'https://download.docker.com/linux/ubuntu Codename: Release' does not have a Release file.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
sena@k9s-sena:~$ sudo apt install containerd.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package containerd.io
E: Couldn't find any package by glob 'containerd.io'
```

ALINAN HATA

Böyle bir hata aldım Docker deposu henüz Ubuntu 24.04 (noble) için resmi destek sunmuyor. <https://download.docker.com/linux/ubuntu>

Codename: noble does not have a Release file.)

Çözüm: noble yerine jammy deposunu kullanmak

Ubuntu 22.04 (jammy) deposu, 24.04'te de çalışır çünkü ikisi ABI uyumludur.

Şu komutu çalıştırdım:

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Ardından başarılı bir şekilde de iki sanal makinemede (worker, master) aynı işlemleri yaparak containerd kurmayı başardım.

```
k8s-sena (worker)
[plugins."io.containerd.snapshotter.v1.zfs"]
  root_path = ""

[plugins."io.containerd.tracing.processor.v1.otlp"]

[plugins."io.containerd.transfer.v1.local"]
  config_path = ""
  max_concurrent_downloads = 3
  max_concurrent_uploaded_layers = 3

[[plugins."io.containerd.transfer.v1.local".unpack_config]]
  differ = ""
  platform = "linux/amd64"
  snapshotter = "overlayfs"

proxy_plugins]

stream_processors]

[stream_processors."io.containerd.ocicrypt.decoder.v1.tar"]
  accepts = ["application/vnd.oci.image.layer.v1.tar+encrypted"]
  args = ["--decryption-keys-path", "/etc/containerd/ocicrypt/keys"]
  env = ["OCICRYPT_KEYPROVIDER_CONFIG=/etc/containerd/ocicrypt/keyprovider.conf"]
  path = "ctd-decoder"
  returns = "application/vnd.oci.image.layer.v1.tar"

[stream_processors."io.containerd.ocicrypt.decoder.v1.tar+gzip"]
  accepts = ["application/vnd.oci.image.layer.v1.tar+gzip+encrypted"]
  args = ["--decryption-keys-path", "/etc/containerd/ocicrypt/keys"]
  env = ["OCICRYPT_KEYPROVIDER_CONFIG=/etc/containerd/ocicrypt/keyprovider.conf"]
  path = "ctd-decoder"
  returns = "application/vnd.oci.image.layer.v1.tar+gzip"

timeouts]
  "io.containerd.timeout.bolt.open" = "0s"
  "io.containerd.timeout.metrics.shimstats" = "2s"
  "io.containerd.timeout.shim.cleanup" = "5s"
  "io.containerd.timeout.shim.load" = "5s"
  "io.containerd.timeout.shim.shutdown" = "3s"
  "io.containerd.timeout.task.state" = "2s"

ttproc]
  address = ""
  gid = 0
  uid = 0
oot@k8s-sena:~# exit
logout
ene@k8s-sena:~$ sudo sed -i 's/SystemdGroup = false/SystemdGroup = true/' /etc/containerd/config.toml
ene@k8s-sena:~$ sudo systemctl restart containerd
ene@k8s-sena:~$
```

2.10 kubeadm Kurulumu (Master & Worker)

```
$ sudo ufw allow 6443/tcp
$ sudo ufw allow 2379:2380/tcp
$ sudo ufw allow 10250/tcp
$ sudo ufw allow 10259/tcp
$ sudo ufw allow 10257/tcp
$ sudo apt-get update
$ sudo apt-get install -y apt-transport-https ca-certificates curl
$ sudo mkdir -p -m 755 /etc/apt/keyrings
### Eğer Kubernetes 1.31'den başka bir versiyon yüklemek isterseniz aşağıdaki iki komuttaki
v1.31 kısımlarını düzeltin ####
$ curl -fsSL https://pkgs.k8s.io/core:stable/v1.31/deb/Release.key | sudo gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
$ echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-
keyring.gpg] https://pkgs.k8s.io/core:stable/v1.31/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list
$ sudo apt-get update
$ sudo apt-get install -y kubelet kubeadm kubectl
$ sudo apt-mark hold kubelet kubeadm kubectl
```

```

k8s-sena (worker)
Fetched 87.7 MB in 9s (9,337 kB/s)
Selecting previously unselected package conntrack.
(Reading database ... 122222 files and directories currently installed.)
Preparing to unpack .../0-conntrack_1:1.4.8-1ubuntu1_amd64.deb ...
Unpacking conntrack (1:1.4.8-1ubuntu1) ...
Selecting previously unselected package cri-tools.
Preparing to unpack .../1-cri-tools_1.31.1-1.1_amd64.deb ...
Unpacking cri-tools (1.31.1-1.1) ...
Selecting previously unselected package kubeadm.
Preparing to unpack .../2-kubeadm_1.31.9-1.1_amd64.deb ...
Unpacking kubeadm (1.31.9-1.1) ...
Selecting previously unselected package kubectl.
Preparing to unpack .../3-kubectl_1.31.9-1.1_amd64.deb ...
Unpacking kubectl (1.31.9-1.1) ...
Selecting previously unselected package kubernetes-cni.
Preparing to unpack .../4-kubernetes-cni_1.5.1-1.1_amd64.deb ...
Unpacking kubernetes-cni (1.5.1-1.1) ...
Selecting previously unselected package kubelet.
Preparing to unpack .../5-kubelet_1.31.9-1.1_amd64.deb ...
Unpacking kubelet (1.31.9-1.1) ...
Setting up conntrack (1:1.4.8-1ubuntu1) ...
Setting up kubectl (1.31.9-1.1) ...
Setting up cri-tools (1.31.1-1.1) ...
Setting up kubernetes-cni (1.5.1-1.1) ...
Setting up kubeadm (1.31.9-1.1) ...
Setting up kubelet (1.31.9-1.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
sena @ user manager service: systemd[1720]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
sena@k8s-sena:~$ sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
sena@k8s-sena:~$

```

2.10 kubernetes cluster kurulumu

Bu işlemleri master node üzerinde yapacağız. İki makinemiz vardı (master ve worker) Kubernetes'i önce master node'da ayağa kaldırmamız daha sonrasında worker node'u bu cluster'a dahil etmemiz gerekiyor.

\$ sudo kubeadm config images pull (Master)

\$ sudo kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=<ip> --control-plane-endpoint=<ip> (Master)

İlk olarak kubernetes'in kurulumu için gerekli olan docker image'lerini bu sisteme çekiyoruz. Kubernetes'in birçok component'i de container olarak kuruluyor. Yani kubernetes'in kendisi bir container orchestrator ama kubernetes'i oluşturan ana component'lerde birer container olarak çalışıyor.

```

k8s-sena (master)
sena@k8s-master:~$ sudo kubeadm config images pull
[0614 20:26:07.579187] 3265 version.go:261] remote version is much newer: v1.31.1; falling back to: stable-1.31
[config/images] Pulled registry.k8s.io/kube-apiserver:v1.31.9
[config/images] Pulled registry.k8s.io/kube-controller-manager:v1.31.9
[config/images] Pulled registry.k8s.io/kube-scheduler:v1.31.9
[config/images] Pulled registry.k8s.io/kube-proxy:v1.31.9
[config/images] Pulled registry.k8s.io/coredns/coredns:v1.11.3
[config/images] Pulled registry.k8s.io/pause:3.10
[config/images] Pulled registry.k8s.io/etcd:3.5.15-0
sena@k8s-master:~$ _

```

sudo kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=192.168.40.53 --control-plane-endpoint=192.168.40.53 (IP'leri master makine'im IP adresi ile değiştirim) (master'da çalıştırıcak)

You can now join any number of control-plane nodes by copying certificate authorities and service account keys on each node and then running the following as root:

```

kubeadm join 192.168.40.53:6443 --token xdsk4r.st94kxgkmf3zepih \
--discovery-token-ca-cert-hash
sha256:5f0925e411bef8a655c7a04f66843cd3fd472c37e9c1b23c04658a5b41ff64e7 \
--control-plane

```

Then you can join any number of worker nodes by running the following on each as root:

```

kubeadm join 192.168.40.53:6443 --token xdsk4r.st94kxgkmf3zepih \
--discovery-token-ca-cert-hash
sha256:5f0925e411bef8a655c7a04f66843cd3fd472c37e9c1b23c04658a5b41ff64e7 (worker)

```

```

k8s-sena (master)
[wait-control-plane] Waiting for the kubelet to boot up the control plane as static Pods from directory "/etc/kubernetes/manifests". This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.000998270s
[api-check] The API server is healthy after 0.002364043s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the configuration for the kubelets in the cluster
[upload-certs] Skipping phase. Please see --upload-certs
[mark-control-plane] Marking the node k8s-master as control-plane by adding the labels: [node-role.kubernetes.io/control-plane:{}]
[mark-control-plane] Marking the node k8s-master as control-plane by adding the taints [node-role.kubernetes.io/control-plane:NoSchedule]
[bootstrap-token] Using token: odox30.s44x4s9uhfjkccv
[bootstrap-token] Configured bootstrap tokens, cluster-info ConfigMap, RBAC Roles
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get nodes
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in order for nodes to get long term certificates
[bootstrap-token] Configured RBAC rules to allow the kubelet to automatically approve CSRs from a Node Bootstrap token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node client certificates in the cluster
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" namespace
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

kubeadm join 192.168.40.52:6443 --token odox30.s44x4s9uhfjkccv \
--discovery-token-ca-cert-hash sha256:c80249b397b964ed7696fdd2330e52d9e662172686f9e70aa63d748d3ebae50 \
--control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.40.52:6443 --token odox30.s44x4s9uhfjkccv \
--discovery-token-ca-cert-hash sha256:c80249b397b964ed7696fdd2330e52d9e662172686f9e70aa63d748d3ebae50
sena@k8s-master:~$ _

```

Artık, kubeadm bu makineyi kubernetes master haline getiriyor üzerine bir kubernetes cluster kurdu. (Master)

```
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alınan hata ve düzeltme

-f parametresi ile düzelttik

```
mkdir -p $HOME/.kube
sudo cp -f /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Bu adımlardan sonra artık başarılı bir şekilde kubectl ile iletişim kurulabilir.

```
sena@k8s-master:~$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    NotReady  control-plane  6m5s  v1.31.9
sena@k8s-master:~$
```

Yukarıdaki görselde de görebileceğimiz gibi worker node ve master node için iki komut var.

Artık cluster'a yeni node'lar ekleyeceğiz. Yeni bir control plane component'i yani ikinci bir master, üçüncü bir master eklemek istiyorsam kullanacağım komut:

```
kubeadm join 192.168.40.53:6443 --token ysdjvv.b72f3ukb7r6089vt \
--discovery-token-ca-cert-hash
sha256:be3623333299d42540990f68c508773c891083b2e541a5984ef1bc6c9de9c81a \
--control-plane
```

Eğer sisteme bir tane worker node eklemek istiyorsam yani container'larımın koşacağı, pod'larımın çalışacağı worker node'lardan bir tane eklemek istiyorsam da şu komutu orada çalıştıracağım. Bu komut sayesinde benim worker node'um bu cluster'a dahil olacak.

```
kubeadm join 192.168.40.53:6443 --token 8ijifp.if4tqssovya8c1h4 --discovery-token-ca-cert-hash
sha256:5f0925e411bef8a655c7a04f66843cd3fd472c37e9c1b23c04658a5b41ff64e7
```

token yenileme

```
sudo kubeadm token create --print-join-command (Master)
```

Şimdi worker makineye gelip yukarıdaki komutu root yetkisi ile çalıştırıyorum:

Bu işlemler esxi'da kopyalama desteği olmadığı için ssh ile bağlandım oradan devam edeceğim.

Yukarıdaki token'ın süreci geçtiği için kabul etmedi. Aşağıdaki komut ile yeni token istedim.

```
sena@k8s-master:~$ sudo kubeadm token create --print-join-command
[sudo] password for sena:
kubeadm join 192.168.40.52:6443 --token ecz111.dtwlgoyk6akuoln6 --discovery-token-ca-cert-hash sha256:c80249b397b964ed7696fd0d2330052d80662172686f9e70aa63d748d3ebae50
sena@k8s-master:~$
```

Bunu worker node'a yapıştırdım.

```

sena@k9s-sena:~$ sudo kubeadm join 192.168.40.52:6443 --token ecz111.dtwlgoyk6akuoln6 --discovery-token-ca-cert-hash sha256:c80249b397b964ed7696fd0d2330052d80662172686f9e70aa63d748d3ebae50
[sudo] password for sena:
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.501700913s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

sena@k9s-sena:~$
```

master node'da **kubectl get nodes** komutunu çalıştırarak teyit ediyorum.

kubectl get nodes

```

sena@k8s-master: ~
sena@k8s-master:~$ kubectl get nodes
NAME           STATUS    ROLES          AGE    VERSION
k8s-master     NotReady  control-plane   37m    v1.31.9
k9s-sena       NotReady  <none>          55s    v1.31.9
sena@k8s-master:~$
```

kubectl get pods -A ile de çalışan pod'ları görüntülüyorum.

```

sena@k8s-master: ~
sena@k8s-master:~$ kubectl get pods -A
NAMESPACE      NAME                                READY   STATUS    RESTARTS   AGE
kube-system    coredns-7c65d6cfc9-2qzcz          0/1     Pending   0           38m
kube-system    coredns-7c65d6cfc9-lz7gl          0/1     Pending   0           38m
kube-system    etcd-k8s-master                   1/1     Running   0           39m
kube-system    kube-apiserver-k8s-master          1/1     Running   0           39m
kube-system    kube-controller-manager-k8s-master 1/1     Running   0           39m
kube-system    kube-proxy-hxkpp                  1/1     Running   0           2m10s
kube-system    kube-proxy-pk85q                  1/1     Running   0           38m
kube-system    kube-scheduler-k8s-master          1/1     Running   0           39m
sena@k8s-master:~$
```

Status kısmında NotReady yazmasının sebebi ise biz bir cluster kurduğumuz zaman bu cluster varsayılan bir networking altyapısı ile gelmez. Bunu bizim seçerek kurmamız gerekir. Networking altyapısını yönetecek bir plugin kurmamız gerekir. En bilineni ise calico. Şimdi onları kuracağız.

\$ kubectl create -

f <https://raw.githubusercontent.com/projectcalico/calico/v3.25.1/manifests/tigera-operator.yaml>

\$ kubectl create -

f <https://raw.githubusercontent.com/projectcalico/calico/v3.25.1/manifests/custom-resources.yaml>

```
sena@k8s-master:~$ kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS             RESTARTS   AGE
calico-system   calico-kube-controllers-646dfcb69-8pnhf              0/1     ContainerCreating   0           67s
calico-system   calico-node-768lh                                     1/1     Running             0           67s
calico-system   calico-node-wrn5n                                     1/1     Running             0           67s
calico-system   calico-typha-6788b957cf-jn8dd                       1/1     Running             0           67s
calico-system   csi-node-driver-bz22r                                0/2     ContainerCreating   0           67s
calico-system   csi-node-driver-mtbr8                                0/2     ContainerCreating   0           67s
kube-system     coredns-7c65d6cfc9-2qzcz                             1/1     Running             0           44m
kube-system     coredns-7c65d6cfc9-lz7gl                             1/1     Running             0           44m
kube-system     etcd-k8s-master                                       1/1     Running             0           44m
kube-system     kube-apiserver-k8s-master                             1/1     Running             0           44m
kube-system     kube-controller-manager-k8s-master                   1/1     Running             0           44m
kube-system     kube-proxy-hxkpp                                     1/1     Running             0           8m8s
kube-system     kube-proxy-pk85q                                     1/1     Running             0           44m
kube-system     kube-scheduler-k8s-master                             1/1     Running             0           44m
tigera-operator tigera-operator-66df695c4-bt2fv                       1/1     Running             0           100s

sena@k8s-master:~$ kubectl get nodes -A
NAME        STATUS   ROLES    AGE     VERSION
k8s-master  Ready    control-plane  45m    v1.31.9
k9s-sena    Ready    <none>      8m39s  v1.31.9

sena@k8s-master:~$
```

Kurulum tamamlanıktan sonra artık status Ready oldu. Bu artık şu manaya geliyor kubeadm ile kubernetes kurulumu başarılı ile gerçekleşti.

SSH ile Bağlanmak İçin

sudo apt update

sudo apt install openssh-server -y

sudo systemctl enable ssh

sudo systemctl start ssh

1. clusters

Kubernetes API sunucusunun adresini ve sertifikasını içerir:

clusters: - cluster: server: <https://192.168.40.52:6443> certificate-authority-data: <base64 şifreli sertifika> name: kubernetes

- server: API sunucusunun IP'si ve portu (master node IP'imiz)
- certificate-authority-data: Bu cluster'a güvenmek için gereken CA sertifikası (base64 encoded)

2. users

Bu config dosyasını kullanan kullanıcının kimlik bilgileri:

users: - name: kubernetes-admin user: client-certificate-data: <base64> client-key-data: <base64>

- client-certificate-data: Kullanıcının public sertifikası
- client-key-data: Kullanıcının private key'i

Bu bilgiler sayesinde kubectl, API sunucusuna kimlik doğrulaması yapabilir.

3. contexts

Hangi kullanıcı hangi cluster'a bağlansın?

contexts: - context: cluster: kubernetes user: kubernetes-admin name: kubernetes-admin@kubernetes

Özetle: kubernetes-admin kullanıcısı, kubernetes adlı cluster'a bağlanacak.

4. current-context

kubectl komutlarının varsayılan olarak hangi cluster'la çalışacağını tanımlar:

current-context: kubernetes-admin@kubernetes

kubectl, bu dosyayı kullanarak:

- 192.168.40.52:6443 adresine bağlanır
- kubernetes-admin kullanıcısı olarak kendini tanıtır
- Gerekli sertifikaları sunar
- Bu yapı sayesinde kubectl get pods, kubectl apply gibi komutlar sorunsuz çalışır.

3.1 kubectl config dosyası

- Kubectl aracı, bağlanacağı Kubernetes cluster bilgilerine config dosyaları aracılığıyla erişir.
- Config dosyasının içerisinde Kubernetes cluster bağlantı bilgilerini ve oraya bağlanırken kullanmak istediğimiz kullanıcıları belirtiriz.
- Daha sonra bu bağlantı bilgileri ve kullanıcıları ve ek olarak namespace bilgilerini de oluşturarak context'ler yaratırız.
- Kubectl varsayılan olarak \$HOME/.kube/ altındaki config dosyasına bakar ama bunu KUBECONFIG environment variable değerini değiştirerek güncelleyebiliriz.

Yani önce hangi cluster'a bağlanacağımı, cluster bilgisini, cluster api server bilgisi ve cluster'ın certificate- authority datasını birleştirerek bir tane cluster tanımı yapıyorum. Daha sonra bu cluster üzerinde hangi kullanıcı buraya bağlanabilir diye kullanıcı tanımı yapıyorum. Bu cluster ve user bilgisini birleştirerek context diye bir şey aratıyor. context mevcut olarak bu cluster'a bu user ile bağlanacağım dediğim kısım.

kubectl config komutu ile kullanabileceğimiz sub command'leri görebiliyoruz.

```
sena@k8s-master:~$ kubectl config
Modify kubeconfig files using subcommands like "kubectl config set current-context my-context".

The loading order follows these rules:

1. If the --kubeconfig flag is set, then only that file is loaded. The flag may only be set once and no merging takes place.
2. If $KUBECONFIG environment variable is set, then it is used as a list of paths (normal path delimiting rules for your system). These paths are merged. When a value is modified, it is modified in the file that defines the stanza. When a value is created, it is created in the first file that exists. If no files in the chain exist, then it creates the last file in the list.
3. Otherwise, ${HOME}/.kube/config is used and no merging takes place.

Available Commands:
current-context  Display the current-context
delete-cluster   Delete the specified cluster from the kubeconfig
delete-context   Delete the specified context from the kubeconfig
delete-user      Delete the specified user from the kubeconfig
get-clusters     Display clusters defined in the kubeconfig
get-contexts     Describe one or many contexts
get-users        Display users defined in the kubeconfig
rename-context   Rename a context from the kubeconfig file
set              Set an individual value in a kubeconfig file
set-cluster      Set a cluster entry in kubeconfig
set-context      Set a context entry in kubeconfig
set-credentials  Set a user entry in kubeconfig
unset            Unset an individual value in a kubeconfig file
use-context      Set the current-context in a kubeconfig file
view             Display merged kubeconfig settings or a specified kubeconfig file

Usage:
  kubectl config SUBCOMMAND [options]

Use "kubectl config <command> --help" for more information about a given command.
Use "kubectl options" for a list of global command-line options (applies to all commands).
```

kubectl config get-contexts komutu kubectl'in baktığı config dosyasında mevcut context'leri listeler. Birden fazla context tanımlanabiliyor olsa da bunlardan birisi current context'tir. Yani mevcutta kullanılan ve atanan context anlamına gelir. Yanında yıldız (*) işareti bulunur. Kubectl ile girdiğimiz tüm komutlar bu mevcut context altında işlenecektir.

```
sena@k8s-master:~$ kubectl config get-contexts
CURRENT  NAME                                CLUSTER  AUTHINFO  NAMESPACE
*         kubernetes-admin@kubernetes        kubernetes  kubernetes-admin
```

İkinci adım olarakta **kubectl config current-context** komutunu girersek bu da şu an tanımlı olan current context'i getirir.

```
sena@k8s-master:~$ kubectl config current-context
kubernetes-admin@kubernetes
sena@k8s-master:~$
```

Peki, diyelim ki ben başka bir sunucu üzerinde işlem yapmak istiyorum böyle bir durumda context'imizi değiştirmemiz gerekiyor. Bu da **kubectl config use-context <hangi context'e geçilmek isteniyorsa o yazılır>** komutu ile sağlanır. Örnek komut:

```
sena@k8s-master:~$ kubectl config use-context docker-desktop
error: no context exists with the name: "docker-desktop"
sena@k8s-master:~$
```

Bundan sonraki tüm işlemler artık docker-desktop üzerinde bu context'e bağlı sunucu üzerinde işlenir.

4. kubectl kullanımı

kubectl yazıp enter'a bastığımız zaman ekrandaki gibi kubectl altında kullanabileceğimiz tüm seçenekleri görüyoruz.

```
sena@k8s-master:~$ kubectl
kubectl controls the Kubernetes cluster manager.

Find more information at: https://kubernetes.io/docs/reference/kubectl/

Basic Commands (Beginner):
  create      Create a resource from a file or from stdin
  expose      Take a replication controller, service, deployment or pod and expose it as a new Kubernetes service
  run         Run a particular image on the cluster
  set         Set specific features on objects

Basic Commands (Intermediate):
  explain     Get documentation for a resource
  get         Display one or many resources
  edit        Edit a resource on the server
  delete      Delete resources by file names, stdin, resources and names, or by resources and label selector

Deploy Commands:
  rollout     Manage the rollout of a resource
  scale       Set a new size for a deployment, replica set, or replication controller
  autoscale   Auto-scale a deployment, replica set, stateful set, or replication controller

Cluster Management Commands:
  certificate Modify certificate resources
  cluster-info Display cluster information
  top         Display resource (CPU/memory) usage
  cordon      Mark node as unschedulable
  uncordon    Mark node as schedulable
  drain       Drain node in preparation for maintenance
  taint       Update the taints on one or more nodes

Troubleshooting and Debugging Commands:
  describe    Show details of a specific resource or group of resources
  logs        Print the logs for a container in a pod
  attach      Attach to a running container
  exec        Execute a command in a container
  port-forward Forward one or more local ports to a pod
  proxy       Run a proxy to the Kubernetes API server
  cp          Copy files and directories to and from containers
  auth        Inspect authorization
```

kubectl cluster-info komutu üzerinde işlem yaptığımız kubernetes cluster ile ilgili temel bir bilgi edinmek istiyorsak kullanacağımız komuttur.

```
sena@k8s-master:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.40.52:6443
CoreDNS is running at https://192.168.40.52:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
sena@k8s-master:~$
```


Örneğin yukarıda birçok kullanabileceğimiz komut gördük. Bunları nasıl kullanabileceğimiz ile alakalı da **--help** komutundan yardım alabiliriz.

Örneğin: **kubectl cp --help**

```
sena@k8s-master:~$ kubectl cp --help
Copy files and directories to and from containers.

Examples:
# !!!Important Note!!!
# Requires that the 'tar' binary is present in your container
# image. If 'tar' is not present, 'kubectl cp' will fail.
#
# For advanced use cases, such as symlinks, wildcard expansion or
# file mode preservation, consider using 'kubectl exec'.

# Copy /tmp/foo local file to /tmp/bar in a remote pod in namespace <some-namespace>
tar cf - /tmp/foo | kubectl exec -i -n <some-namespace> <some-pod> -- tar xf - -C /tmp/bar

# Copy /tmp/foo from a remote pod to /tmp/bar locally
kubectl exec -n <some-namespace> <some-pod> -- tar cf - /tmp/foo | tar xf - -C /tmp/bar

# Copy /tmp/foo_dir local directory to /tmp/bar_dir in a remote pod in the default namespace
kubectl cp /tmp/foo_dir <some-pod>:/tmp/bar_dir

# Copy /tmp/foo local file to /tmp/bar in a remote pod in a specific container
kubectl cp /tmp/foo <some-pod>:/tmp/bar -c <specific-container>

# Copy /tmp/foo local file to /tmp/bar in a remote pod in namespace <some-namespace>
kubectl cp /tmp/foo <some-namespace>/<some-pod>:/tmp/bar

# Copy /tmp/foo from a remote pod to /tmp/bar locally
kubectl cp <some-namespace>/<some-pod>:/tmp/foo /tmp/bar

Options:
-c, --container='':
    Container name. If omitted, use the kubectl.kubernetes.io/default-container annotation for selecting the
    container to be attached or the first container in the pod will be chosen
--no-preserve=false:
```

kubectl ile aksini belirtmediğimiz sürece tüm komutlar context'de set edilmiş namespace üzerinde uygulanır. context'de namespace'de belirtilmediyse default isimli namespace üzerinde uygulanır.

Örneğin **kubectl get pods** komutu ile pod'ları listelemesini istedik. kubectl bu komutu **default** isimli namespace için çalıştırdı ve şu anda default isimli namespace'de deploy edilmiş herhangi bir pod olmadığı için de "No resources found in default namespace." mesajını döndü.

```
sena@k8s-master:~$ kubectl get pods
No resources found in default namespace.
sena@k8s-master:~$
```

Fakat ben istersem kubectl komutlarını hangi namespace için çalıştırmak istediğimi belirtebilirim. Örneğin **kubectl get pods -n kube-system** komutu sayesinde kube-system isimli namespace üzerinde çalıştıracaktır.

```

sena@k8s-master:~$ kubectl get pods
No resources found in default namespace.
sena@k8s-master:~$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-2qzcz           1/1     Running   0           17h
coredns-7c65d6cfc9-lz7gl           1/1     Running   0           17h
etcd-k8s-master                     1/1     Running   0           17h
kube-apiserver-k8s-master           1/1     Running   0           17h
kube-controller-manager-k8s-master 1/1     Running   0           17h
kube-proxy-hxkpp                    1/1     Running   0           17h
kube-proxy-pk85q                    1/1     Running   0           17h
kube-scheduler-k8s-master           1/1     Running   0           17h
sena@k8s-master:~$

```

Tüm namespace'lerde deploy edilmiş pod'ları listelemek için de **kubectl get pods -A** komutu kullanılır.

```

sena@k8s-master:~$ kubectl get pods -A
NAMESPACE   NAME                                READY   STATUS    RESTARTS   AGE
calico-apiserver  calico-apiserver-84cc67fdf6-f2bdj  1/1     Running   0           17h
calico-apiserver  calico-apiserver-84cc67fdf6-s7vlh  1/1     Running   0           17h
calico-kube-system  calico-kube-controllers-646dfcb69-8pnhf  1/1     Running   0           17h
calico-system      calico-node-768lh                    1/1     Running   0           17h
calico-system      calico-node-wrn5n                    1/1     Running   0           17h
calico-system      calico-typha-6788b957cf-jn8dd        1/1     Running   0           17h
calico-system      csi-node-driver-bz22r                2/2     Running   0           17h
calico-system      csi-node-driver-mtbr8                2/2     Running   0           17h
kube-system        coredns-7c65d6cfc9-2qzcz            1/1     Running   0           17h
kube-system        coredns-7c65d6cfc9-lz7gl            1/1     Running   0           17h
kube-system        etcd-k8s-master                      1/1     Running   0           17h
kube-system        kube-apiserver-k8s-master            1/1     Running   0           17h
kube-system        kube-controller-manager-k8s-master   1/1     Running   0           17h
kube-system        kube-proxy-hxkpp                     1/1     Running   0           17h
kube-system        kube-proxy-pk85q                     1/1     Running   0           17h
kube-system        kube-scheduler-k8s-master            1/1     Running   0           17h
tigera-operator    tigera-operator-66df695c4-bt2fv      1/1     Running   1 (17h ago)  17h
sena@k8s-master:~$

```

Çıktı formatını özelleştirmek için **-o (output)** parametresi kullanılır. Örneğin **-o wide** ifadesi eklendiğinde, standart çıktıya ek olarak node bilgisi, IP adresi gibi detaylar da gösterilir.

```

sena@k8s-master:~$ kubectl get pods -A -o wide
NAMESPACE   NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS GATES
calico-apiserver  calico-apiserver-84cc67fdf6-f2bdj  1/1     Running   0           17h   192.168.235.197  k8s-master    <none>            <none>
calico-apiserver  calico-apiserver-84cc67fdf6-s7vlh  1/1     Running   0           17h   192.168.16.130  k9s-sena      <none>            <none>
calico-kube-system  calico-kube-controllers-646dfcb69-8pnhf  1/1     Running   0           17h   192.168.235.194  k8s-master    <none>            <none>
calico-system      calico-node-768lh                    1/1     Running   0           17h   192.168.40.51   k9s-sena      <none>            <none>
calico-system      calico-node-wrn5n                    1/1     Running   0           17h   192.168.40.52   k8s-master    <none>            <none>
calico-system      calico-typha-6788b957cf-jn8dd        1/1     Running   0           17h   192.168.40.51   k9s-sena      <none>            <none>
calico-system      csi-node-driver-bz22r                2/2     Running   0           17h   192.168.235.192  k8s-master    <none>            <none>
calico-system      csi-node-driver-mtbr8                2/2     Running   0           17h   192.168.16.129  k9s-sena      <none>            <none>
kube-system        coredns-7c65d6cfc9-2qzcz            1/1     Running   0           18h   192.168.235.195  k8s-master    <none>            <none>
kube-system        coredns-7c65d6cfc9-lz7gl            1/1     Running   0           18h   192.168.235.193  k8s-master    <none>            <none>
kube-system        etcd-k8s-master                      1/1     Running   0           18h   192.168.40.52   k8s-master    <none>            <none>
kube-system        kube-apiserver-k8s-master            1/1     Running   0           18h   192.168.40.52   k8s-master    <none>            <none>
kube-system        kube-controller-manager-k8s-master   1/1     Running   0           18h   192.168.40.52   k8s-master    <none>            <none>
kube-system        kube-proxy-hxkpp                     1/1     Running   0           17h   192.168.40.51   k9s-sena      <none>            <none>
kube-system        kube-proxy-pk85q                     1/1     Running   0           18h   192.168.40.52   k8s-master    <none>            <none>
kube-system        kube-scheduler-k8s-master            1/1     Running   0           18h   192.168.40.52   k8s-master    <none>            <none>
tigera-operator    tigera-operator-66df695c4-bt2fv      1/1     Running   1 (17h ago)  17h   192.168.40.51   k9s-sena      <none>            <none>
sena@k8s-master:~$

```


Bir diğer örnek ise yaml formatıdır.

kubectl get pods -A -o yaml

```

sena@k8s-master:~$ kubectl get pods -A -o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: Pod
  metadata:
    annotations:
      cni.projectcalico.org/containerID: bf1e5428624d3f67bb4f64b17f4096ee554cb7db3938887d94c31a57a9a04ebc
      cni.projectcalico.org/podIP: 192.168.235.197/32
      cni.projectcalico.org/podIPs: 192.168.235.197/32
      hash.operator.tigera.io/calico-apiserver-certs: 6123cea75ee79737fef18b46612098311eb91bfd
      creationTimestamp: "2025-06-14T21:21:59Z"
      generateName: calico-apiserver-84cc67fdf6-
    labels:
      apiserver: "true"
      app.kubernetes.io/name: calico-apiserver
      k8s-app: calico-apiserver
      pod-template-hash: 84cc67fdf6
  name: calico-apiserver-84cc67fdf6-f2bdj
  namespace: calico-apiserver
  ownerReferences:
  - apiVersion: apps/v1
    blockOwnerDeletion: true
    controller: true
    kind: ReplicaSet
    name: calico-apiserver-84cc67fdf6
    uid: 3361478b-d58d-4163-bc4d-e08a32678dba
  resourceVersion: "5532"
  uid: bb4d56c0-957a-4c3c-83f6-5f744cc31aa8
  spec:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
        - podAffinityTerm:
            labelSelector:
              matchLabels:
                k8s-app: calico-apiserver
            namespaces:
            - calico-apiserver

```

json formati: **kubectl get pods -A -o json**

```

sena@k8s-master:~$ kubectl get pods -A -o json
{
  "apiVersion": "v1",
  "items": [
    {
      "apiVersion": "v1",
      "kind": "Pod",
      "metadata": {
        "annotations": {
          "cni.projectcalico.org/containerID": "bf1e5428624d3f67bb4f64b17f4096ee554cb7db3938887d94c31a57a9a04ebc",
          "cni.projectcalico.org/podIP": "192.168.235.197/32",
          "cni.projectcalico.org/podIPs": "192.168.235.197/32",
          "hash.operator.tigera.io/calico-apiserver-certs": "6123cea75ee79737fef18b46612098311eb91bfd"
        },
        "creationTimestamp": "2025-06-14T21:21:59Z",
        "generateName": "calico-apiserver-84cc67fdf6-",
        "labels": {
          "apiserver": "true",
          "app.kubernetes.io/name": "calico-apiserver",
          "k8s-app": "calico-apiserver",
          "pod-template-hash": "84cc67fdf6"
        },
        "name": "calico-apiserver-84cc67fdf6-f2bdj",
        "namespace": "calico-apiserver",
        "ownerReferences": [
          {
            "apiVersion": "apps/v1",
            "blockOwnerDeletion": true,
            "controller": true,
            "kind": "ReplicaSet",
            "name": "calico-apiserver-84cc67fdf6",
            "uid": "3361478b-d58d-4163-bc4d-e08a32678dba"
          }
        ],
        "resourceVersion": "5532",
        "uid": "bb4d56c0-957a-4c3c-83f6-5f744cc31aa8"
      },
      "spec": {
        "affinity": {
          "podAntiAffinity": {

```

-o wide	Varsayılan çıktıya ek bilgi (örn. node adı, pod IP) ekler
-o json	JSON formatında çıktı üretir
-o yaml	YAML formatında çıktı üretir
-o name	Yalnızca kaynak adını gösterir (örn. pod/my-app)
-o jsonpath= <expr>	JSONPath ifadesi ile çıktıyı özelleştirir
-o jsonpath-as-json	JSONPath sonucunu geçerli JSON formatında döndürür
-o jsonpath-file= <file>	JSONPath ifadesini bir dosyadan okur
-o custom-columns=...	Belirtilen sütunlara göre özelleştirilmiş tablo çıktısı üretir
-o custom-columns-file=	Sütun tanımlarını bir dosyadan alır
-o go-template=...	Go template kullanarak özelleştirilmiş çıktı üretir
-o go-template-file=	Go template tanımını dosyadan okur
-o template=...	Genel şablon tabanlı çıktı üretir (legacy)
-o templatefile=	Template tanımını dosyadan alır (legacy)

man komutu, "manual" (kılavuz) kelimesinin kısaltmasıdır ve Linux sistemlerinde komutların nasıl çalıştığını, hangi parametreleri desteklediğini ve örnek kullanımlarını öğrenmek için kullanılır. Benzer şekilde, Kubernetes dünyasında kubectl için bu işlevi sağlayan komut **kubectl explain**'dir.

kubectl explain, Kubernetes kaynak türlerinin (örneğin Pod, Deployment, Service) yapısını, alanlarını ve açıklamalarını incelemek için kullanılır ve man komutunun kubectl karşılığı olarak düşünülebilir.

Örneğin **kubectl explain pod** Pod nesnesinin açıklamasını ve field'larını getirir.

```

sena@k8s-master:~$ kubectl explain pod
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
    APIVersion defines the versioned schema of this representation of an object.
    Servers should convert recognized schemas to the latest internal value, and
    may reject unrecognized values. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources

  kind          <string>
    Kind is a string value representing the REST resource this object
    represents. Servers may infer this from the endpoint the client submits
    requests to. Cannot be updated. In CamelCase. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds

  metadata      <ObjectMeta>
    Standard object's metadata. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata

  spec          <PodSpec>
    Specification of the desired behavior of the pod. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status

  status        <PodStatus>
    Most recently observed status of the pod. This data may not be up to date.
    Populated by the system. Read-only. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status

```

5. Pod

Şu ana kadar hep kubernetes cluster ile ilgili bilgiler edindik. Ve amacımız kubernetes üstünde container image'ları haline getirdiğimiz iş yüklerimizi koşturmak istiyoruz, yani kubernetes üzerinde bir şeyler deploy etmek istiyoruz. Bunlara ise objeler denir. Kubernetes üzerinde çeşitli tiplerde objeler yaratır, siler ve modifiye ederiz. Kubernetes üzerinde her şey bir api objesidir ve bizler onun üzerinde koşturmak istediğimiz iş yüklerini bu objeler şeklinde koşturup ayarlarız.

- Pod'lar, Kubernetes'te oluşturabileceğiniz ve yönetebileceğiniz en küçük birimlerdir.
- Pod'lar bir ya da daha fazla container barındırabilir. Ama çoğu durumda pod tek container barındırır.
- Her pod'un eşsiz bir id'si "uid" bulunur.
- Her pod eşsiz bir ip adresine sahiptir.
- Aynı pod içerisindeki containerlar aynı node üstünde çalıştırılır ve bu containerlar birbirleriyle localhost üstünden haberleşebilir.

5.1 Pod Oluşturma

kubectl run firstpod --image=nginx --restart=Never komutu ile firstpod adında bir Pod oluşturduk. Bu Pod, resmi nginx image'ini kullanır. **--restart=Never** parametresi ise Pod'un tek seferlik çalıştırılmasını ifade eder; yani komut, yeniden başlatılmayan geçici bir Pod oluşturur.

```
sena@k8s-master:~$ kubectl run firstpod --image=nginx --restart=Never
pod/firstpod created
sena@k8s-master:~$
```

kubectl get pods -A -o wide komutu ile teyit ettik, başarılı bir şekilde oluşturulmuş.

```
sena@k8s-master:~$ kubectl get pods -A -o wide
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
calico-apiserver	calico-apiserver-84cc67fdf6-f2bdj	1/1	Running	0	34h	192.168.235.197	k8s-master	<none>	<none>
calico-apiserver	calico-apiserver-84cc67fdf6-s7vlh	1/1	Running	0	34h	192.168.16.130	k9s-sena	<none>	<none>
calico-system	calico-kube-controllers-646dfcb69-8pnhf	1/1	Running	0	34h	192.168.235.194	k8s-master	<none>	<none>
calico-system	calico-node-768lh	1/1	Running	0	34h	192.168.40.51	k9s-sena	<none>	<none>
calico-system	calico-node-wrn5n	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
calico-system	calico-typha-6788b957cf-jn8dd	1/1	Running	0	34h	192.168.40.51	k9s-sena	<none>	<none>
calico-system	csi-node-driver-bz22r	2/2	Running	0	34h	192.168.235.192	k8s-master	<none>	<none>
calico-system	csi-node-driver-mtbr8	2/2	Running	0	34h	192.168.16.129	k9s-sena	<none>	<none>
default	firstpod	1/1	Running	0	4m46s	192.168.16.131	k9s-sena	<none>	<none>
kube-system	coredns-7c65d6cfc9-2qzcz	1/1	Running	0	34h	192.168.235.195	k8s-master	<none>	<none>
kube-system	coredns-7c65d6cfc9-l27gl	1/1	Running	0	34h	192.168.235.193	k8s-master	<none>	<none>
kube-system	etcd-k8s-master	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
kube-system	kube-apiserver-k8s-master	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
kube-system	kube-controller-manager-k8s-master	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
kube-system	kube-proxy-hxkpp	1/1	Running	0	34h	192.168.40.51	k9s-sena	<none>	<none>
kube-system	kube-proxy-pk85q	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
kube-system	kube-scheduler-k8s-master	1/1	Running	0	34h	192.168.40.52	k8s-master	<none>	<none>
tigera-operator	tigera-operator-66df695c4-bt2fv	1/1	Running	1 (33h ago)	34h	192.168.40.51	k9s-sena	<none>	<none>

Not: **kubectl describe** komutu, bir Kubernetes objesinin detaylı bilgilerine erişmemizi sağlar. Örneğin, **kubectl describe pod firstpod** komutu ile firstpod adlı pod'a ait status, events, kullanılan image, node bilgisi ve diğer önemli ayrıntılar görüntülenebilir.

```

sena@k8s-master:~$ kubectl describe pod firstpod
Name:          firstpod
Namespace:     default
Priority:       0
Service Account: default
Node:          k9s-sena/192.168.40.51
Start Time:    Mon, 16 Jun 2025 07:23:50 +0000
Labels:        run=firstpod
Annotations:   cni.projectcalico.org/containerID: 1f988ebd74646680cc9159cf22ec360861ae802e8b54aabb5525e38b5e8afee
               cni.projectcalico.org/podIP: 192.168.16.131/32
               cni.projectcalico.org/podIPs: 192.168.16.131/32
Status:        Running
IP:            192.168.16.131
IPs:           IP: 192.168.16.131
Containers:
  firstpod:
    Container ID:   containerd://4a92372b898e5217caa9499fc37c60f0e7427cc7c1846bfff2001d507e0ec5cd8
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 16 Jun 2025 07:24:11 +0000
    Ready:          True
    Restart Count:   0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-l2xqz (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-l2xqz:

```

Komut çıktısının en altındaki **"Events"** bölümü, pod ile ilgili adım adım gerçekleşen olayları gösterir.

Pod'un ne zaman oluşturulduğu, hangi işlemlerin yapıldığı, hata olup olmadığı ve Kubernetes'in hangi aksiyonları aldığı gibi detaylı bilgiler yer alır.

```

Events:
Type     Reason      Age   From              Message
----     -
Normal   Scheduled   10m   default-scheduler Successfully assigned default/firstpod to k9s-sena
Normal   Pulling     10m   kubelet           Pulling image "nginx"
Normal   Pulled      9m54s kubelet           Successfully pulled image "nginx" in 20.215s (20.215s including waiting). Image size: 72406859 bytes.
Normal   Created     9m54s kubelet           Created container: firstpod
Normal   Started     9m54s kubelet           Started container firstpod
sena@k8s-master:~$

```

kubectl logs komutu, bir pod içerisindeki container'a ait log çıktısını görmemizi sağlar.

Örneğin, **kubectl logs firstpod** komutu ile firstpod adlı pod'un çalışması sırasında ürettiği loglara erişebiliriz.

Özellikle troubleshooting süreçlerinde bu loglar, hataların kaynağını tespit etmek açısından oldukça önemlidir.

```

sena@k8s-master:~$ kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/06/16 07:24:11 [notice] 1#1: using the "epoll" event method
2025/06/16 07:24:11 [notice] 1#1: nginx/1.27.5
2025/06/16 07:24:11 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2025/06/16 07:24:11 [notice] 1#1: OS: Linux 6.8.0-60-generic
2025/06/16 07:24:11 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/06/16 07:24:11 [notice] 1#1: start worker processes
2025/06/16 07:24:11 [notice] 1#1: start worker process 29
2025/06/16 07:24:11 [notice] 1#1: start worker process 30
2025/06/16 07:24:11 [notice] 1#1: start worker process 31
2025/06/16 07:24:11 [notice] 1#1: start worker process 32
sena@k8s-master:~$

```

Logları canlı olarak izlemek için **-f** parametresi kullanılır.

Örneğin, bir web server'in loglarını anlık olarak takip etmek isteyebilirsiniz. Bu durumda **kubectl logs -f firstpod** komutu ile loglar gerçek zamanlı olarak görüntülenebilir.

Özellikle sürekli güncellenen çıktıları izlemek ve service davranışlarını anlık takip etmek açısından oldukça faydalıdır.

```
sena@k8s-master:~$ kubectl logs -f firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/06/16 07:24:11 [notice] 1#1: using the "epoll" event method
2025/06/16 07:24:11 [notice] 1#1: nginx/1.27.5
2025/06/16 07:24:11 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2025/06/16 07:24:11 [notice] 1#1: OS: Linux 6.8.0-60-generic
2025/06/16 07:24:11 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/06/16 07:24:11 [notice] 1#1: start worker processes
2025/06/16 07:24:11 [notice] 1#1: start worker process 29
2025/06/16 07:24:11 [notice] 1#1: start worker process 30
2025/06/16 07:24:11 [notice] 1#1: start worker process 31
2025/06/16 07:24:11 [notice] 1#1: start worker process 32
```

kubectl exec komutu, bir pod içerisinde başka bir komut çalıştırmak istediğimizde kullanılır.

Örneğin, **kubectl exec firstpod -- hostname** komutu ile firstpod adlı pod içinde hostname komutunu çalıştırabilir ve pod'un barındığı container'ın hostname bilgisini alabiliriz.

```
sena@k8s-master:~$ kubectl exec firstpod -- hostname
firstpod
sena@k8s-master:~$
```

Bir pod'a etkileşimli olarak bağlanmak istediğimizde de **kubectl exec** komutunu kullanabiliriz.

Örneğin, **kubectl exec -it firstpod -- /bin/sh** komutu ile firstpod adlı pod'a bağlanarak terminal üzerinden komut çalıştırabiliriz. Pod içerisindeki container'a doğrudan erişim sağlamak için oldukça kullanışlıdır.

```
sena@k8s-master:~$ kubectl exec -it firstpod -- /bin/sh
# hostname
firstpod
# ls
bin boot dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
# printenv
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.96.0.1:443
HOSTNAME=firstpod
HOME=/root
PKG_RELEASE=1~bookworm
DYNPKG_RELEASE=1~bookworm
TERM=xterm
NGINX_VERSION=1.27.5
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NJS_VERSION=0.8.10
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_PROTO=tcp
NJS_RELEASE=1~bookworm
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_HOST=10.96.0.1
PWD=/
#
```

Oluşturduğumuz pod'u silmek için ise **delete** komutu kullanılır.

Örneğin, **kubectl delete pods firstpod** dediğimiz zaman firstpod isimli pod'u sistem üzerinden silmiş oluruz.

```
sena@k8s-master:~$ kubectl delete pods firstpod
pod "firstpod" deleted
sena@k8s-master:~$
```

6. YAML

YAML (YAML Ain't Markup Language) tüm programlama dilleri için okuması ve oluşturulması kolay bir veri serilizasyon standardıdır.

YAML İçin Faydalı Kaynaklar

- [Getting Started with Kubernetes Part 1: Understanding YAML Files in Context of Kubernetes](#)
- [The Official YAML Web Site](#)
- [Learn YAML tutorials with Cheatsheet examples](#)

Kubernetes'te herhangi bir kaynağı tanımlarken kullanılan YAML dosyalarında, her tanım mutlaka üç temel bileşen içermelidir: **apiVersion**, **kind** ve **metadata**. Bu alanlar, Kubernetes API'sine bu kaynağın türünü, ait olduğu versiyonu ve tanımlayıcı bilgilerini iletmek için zorunludur.

apiVersion

Tanımlamak istediğiniz kaynağın (örneğin bir Pod, Deployment ya da Service) Kubernetes API içerisindeki hangi versiyona ait olduğunu belirtir.

İlgili kaynağın hangi API group altında tanımlandığını ifade eder.

Örneğin:

- apiVersion: v1
- apiVersion: apps/v1

Doğru apiVersion kullanımı, YAML manifest'inin doğru yorumlanabilmesi açısından önemlidir. Kullanılabilecek versiyonlar **kubectl explain <resource>** komutu ile ya da Kubernetes resmi dokümantasyonu üzerinden öğrenilebilir.

kind

Oluşturmak istediğiniz kaynağın türünü belirtir (örneğin Pod, Service, Deployment).

Bu alan Kubernetes'e "hangi türde bir nesne oluşturulacağını" bildirir.

kind değeri değiştikçe, YAML dosyasının yapısı ve içeriği de farklılık gösterir.

metadata

Kaynağa ait tanımlayıcı bilgilerin yer aldığı bölümdür.

Genellikle aşağıdaki alt alanlar bulunur:

- **name:** Kaynağın ismi (benzersiz olmalıdır)
- **labels:** Kaynağı etiketlemek için kullanılan key-value çiftleri
- **namespace:** Kaynağın ait olduğu mantıksal alan

metadata bölümü, kaynakların gruplandırılması, filtrelenmesi ve ilişkilendirilmesi için büyük önem taşır. Özellikle labels, servis seçimleri (selector) gibi işlemler için önemlidir.

spec

spec, oluşturulacak kaynağın çalışma detaylarını (özelliklerini ve yapılandırmasını) tanımlar.

Her kaynak türü için spec alanında yer alacak bilgiler farklıdır:

- Pod tanımlarken: Container listesi, imajlar, portlar vb.
- Service tanımlarken: Tip (ClusterIP, NodePort, LoadBalancer), selector, hedef port gibi bilgiler.
- Deployment tanımlarken: Replica sayısı, template, strategy, selector vb.

Yani spec, tanımlanan kaynağın nasıl çalışacağını belirleyen alandır.

Örnek Pod Tanımı:

apiVersion: v1

kind: Pod

metadata:

name: firstpod

labels:

app: frontend

spec:

containers:

- **name: nginx**

image: nginx:latest

ports:

- **containerPort: 80**

- **kind** alanında bir Pod tanımlanmıştır.
- **metadata** alanında firstpod adında bir isim verilmiş ve app: frontend etiketi atanmıştır.
- **spec** alanında ise tek bir container tanımlanmıştır; bu container nginx:latest image'ini kullanmakta ve 80 numaralı portu dinlemektedir.

.yaml dosyasını **apply** etmek için kubernetes'de apply komutu kullanılır. Bu sayede YAML dosyasında tanımlı olan kaynaklar oluşturulur ya da mevcut kaynaklar güncellenir (**declarative configuration** yaklaşımı)

Örneğin, **kubectl apply -f pod1.yaml**

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f
objectbasetemplate.yaml podenv.yaml podlife1.yaml podlife4.yaml README.md
pod1.yaml podinitcontainer.yaml podlife2.yaml podmulticontainer.yaml service1.yaml
podannotation.yaml podlabel.yaml podlife3.yaml podnamespace.yaml
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f pod1.yaml
pod/firstpod created
sena@k8s-master:~/k8sfundamentals/pod$
```

Şimdi bu oluşturduğumuz objenin özelliklerine bakalım.

kubectl describe pods firstpod komutu ile istediğimiz özelliklerde bir pod oluşturulduğunu görüyoruz. (80 portu dış dünyaya açılmış, nginx image'ından oluşmuş, labels tanımlanmış)

```
Containers:
  nginx:
    Container ID:   containerd://e5f9263ec71a37fd01954dc728174fb27623d1660c7bea1f918f6d887ccb6b59
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:          80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Mon, 16 Jun 2025 08:38:11 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-drjpp (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-drjpp:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:       kube-root-ca.crt
    ConfigMapOptional:   <nil>
    DownwardAPI:         true
QoS Class:           BestEffort
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   2m11s  default-scheduler  Successfully assigned default/firstpod to k9s-sena
  Normal  Pulling     2m10s  kubelet        Pulling image "nginx:latest"
  Normal  Pulled      2m8s   kubelet        Successfully pulled image "nginx:latest" in 1.994s (1.994s including waiting). Image size: 72406859 bytes.
  Normal  Created     2m8s   kubelet        Created container: nginx
  Normal  Started     2m8s   kubelet        Started container nginx
```

YAML dosyaları sayesinde uygulamada yapılan değişiklikleri merkezi ve versiyonlanabilir bir şekilde yönetme imkânı elde ederiz.

Yani kaynakların tanımı kod haline getirildiği için, yapılan her güncelleme tek bir yerden takip edilebilir, kontrol edilebilir ve gerektiğinde kolayca geri alınabilir.

kubectl edit komutu herhangi bir kubernetes objesinin üstünde direkt bir değişiklik yapmak istediğimiz zaman kullanılır.

Örneğin, **kubectl edit pods firstpod** komutu çalıştırıldığında, **firstpod** adlı Pod'un mevcut konfigürasyonu .yaml formatında editörünüzde açılır.

Burada yapılan değişiklikler kaydedilip çıktığında, Kubernetes API bu güncellemeleri otomatik olarak uygular.


```

Please edit the object below. Lines beginning with a '#' will be ignored,
and an empty file will abort the edit. If an error occurs while saving this file will be
reopened with the relevant failures.
#
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cnf.projectcalico.org/containerID: 093211040adadbc1ed1a66e3e03d8951b5d634d50bfe345c9721384b0a85fd58
    cnf.projectcalico.org/podIP: 192.168.16.132/32
    cnf.projectcalico.org/podIPs: 192.168.16.132/32
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"labels":{"app":"frontend"},"name":"firstpod","namespace":"default"},"spec":{"containers":[{"image":"nginx:latest","name":"nginx","ports":[{"containerPort":80}]}]}}
  creationTimestamp: "2025-06-16T08:38:08Z"
  labels:
    app: frontend
  name: firstpod
  namespace: default
  resourceVersion: "145147"
  uid: 1a355905-d375-47b8-96ae-bdc024a64a33
spec:
  containers:
    - image: nginx:latest
      imagePullPolicy: Always
      name: nginx
      ports:
        - containerPort: 80
          protocol: TCP
      resources: {}
      terminationMessagePath: /dev/termination-log
      terminationMessagePolicy: File
      volumeMounts:
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
          name: kube-api-access-drjpp
          readOnly: true
      dnsPolicy: ClusterFirst
      enableServiceLinks: true
      nodeName: k9s-sena
'/tmp/kubect1-edit-566863217.yaml' 122L, 3689B
1,1 Top

```

6.1 Pod Yaşam Döngüsü

Pod oluşturmak için ya kubectl ile imperative bir şekilde komutla oluşturuyor ya da bir .yaml dosyası oluşturarak ve bunu kubectl apply komutu ile kubernetes cluster'a gönderiyoruz. Bu aşamadan itibaren kubernetes apiserver bu yaml dosyasını alır burada bizim belirttiğimiz ayarlarının yanında varsayılan olarak set edilmiş ayarları harmanlar, pod'a bir unique ID atar ve tüm bunları etcd'e kaydeder. Bu noktadan itibaren pod oluşturulmuştur. Ve yaşam döngüsünün ilk aşaması olan **pending** aşamasına geçer.

Eğer bir sistemde podun durumu pending'deyse bunun anlamı şudur: sistemde yeni bir pod oluşturuldu, podla ilgili tüm gerekli tanımlar yapıldı ve veritabanına kaydedildi. Ama pod henüz herhangi bir node üzerinde oluşturulmadı.

kubescheduler servisi, apiserver aracılığıyla etcd'i sürekli gözler. Eğer burada yeni yaratılmış ve herhangi bir node ataması yapılmamış bir pod görürse çalışmaya başlar. Algoritması ve seçme kriterlerine göre podun çalışmasının en uygun olacağı node'u seçer ve veritabanındaki pod objesine node bilgisini ekler. Bu noktadan itibaren pod yaşam döngüsünde **creating** aşamasına geçer.

Cluster içerisinde bulunan tüm node'larda **kubelet** adında bir servis çalışır. kubelet'de aynı kubescheduler gibi etcd veritabanını gözler ve bulunduğu node'a atanmış pod'ları tespit eder. Yeni yaratılmış pod'u görür ve hemen işlemlere başlar. İlk olarak pod tanımında oluşturulması istenen container'lara bakar ve bu container'ların oluşturulacağı image'ları sisteme indirmeye başlar.

Eğer bir şekilde image indirilemezse pod öncelikle **image pull** ve ardından da **image pull backoff** aşamasına geçer. Eğer pod'un status'unda image pull backoff görürseniz bu node'un image'ı repository'den çekemediği ve bunu tekrar tekrar denemeye devam ettiği anlamına gelir.

En sık karşılaşılan neden ise image isminin pod isminde yanlış yazılmasıdır.

Bir diğer sık rastlanılan sebep ise image'ı çekmek için registry'e authoneticate olmak gerekmesi fakat kullanıcı adı ve şifre gibi bilgilerde hata yapılmasıdır. Bu durumda da image çekilemeyecek ve pod **ImagePullBackOff** aşamasında takılı kalacaktır.

Fakat bunların olmadığını ve image'in düzgün çekildiğini varsayalım. İşte bu noktadan itibaren, kubelet o node'da bulunan container engine ile haberleşir ve ilgili container'ların oluşturulmasını sağlar. Container'lar çalışmaya başladığı anda da pod **Running** durumuna geçer. Bu noktada artık pod oluşturulmuş olur.

Container imajlarında, container oluşturulup çalıştırıldığı zaman başlatılması belirlenen bir uygulama bulunur. Bu uygulama çalıştığı sürece container da çalışır durumdadır. Fakat uygulama çalışmayı bırakırsa container da durdurulur. Uygulamanın çalışmayı bırakması da 3 şekilde olur:

- Uygulama görevlerini tamamlayıp otomatik şekilde hata vermeden kapanır.
- Kullanıcı ya da sistem uygulamaya kapanma sinyali gönderir ve hata vermeden kapanır.
- Ya da hata verir, çöker, sıkıntı çıkar ve hata kodu oluşturarak kapanır.

Pod'un içerisinde oluşturulan container'lara pod tanımı içerisinde bir **RestartPolicy** tanımlanır ve RestartPolicy üç değer alabilir.

1. **Always:** Varsayılan değerdir ve siz aksini belirtmediğiniz sürece always olarak ayarlanır. Always şu manaya gelir: Pod'un içerisinde yaratılan container ister hata vererek isterse de hata vermeden hangi koşulda durdurulursa durdurulsun o container'ı yeniden başlat demektir.
2. **On-failure:** Sadece hata alıp kapanırsa yeniden başlatılır. Yani container görevini başarıyla (exit code 0) tamamladıysa yeniden başlatılmaz.
3. **Never:** Bu değer seçildiğinde container herhangi bir nedenle durduğunda yeniden başlatılmaz. Genellikle test veya tek seferlik işlemler için kullanılır.

Bu noktadan itibaren, pod'un altında tanımlı container'lar çalışmaya devam ettikçe pod **running** olarak devam eder. Eğer container'ların hepsi hata vermeden doğal olarak kapanırsa ve RestartPolicy'de Never ya da On-Failure olarak set edildiyse pod'un status'u **Succeeded'a** ya da diğer bir adıyla **Completed'a** döner ve pod başarılı bir şekilde yaşam döngüsünü tamamlar.

Yine **RestartPolicy** never ya da on-failure olarak set edildiği durumlarda container'lardan biri hata verip kapanırsa bu sefer pod'un statusu **Failed** olarak işaretlenir ve yaşam döngüsünü böyle tamamlar.

Fakat **restartPolicy** değeri **Always** olarak ayarlandığında, container'lar ister hatayla kapansın ister normal şekilde kapatılsın, her durumda yeniden başlatılacaklarını söylemiştik. Dolayısıyla Pod hiçbir zaman **Completed** ya da **Failed** durumuna geçmez. Bunun yerine, Pod'un içerisindeki container yeniden başlatılır ve **Running** durumunda kalmaya devam eder.

Fakat Kubernetes bu yeniden başlatma işlemini belirli bir sıklıkla yapıyorsa, bazı şeylerin ters gittiğine kanaat getirir ve Pod'u **CrashLoopBackOff** adını verdiğimiz bir duruma sokar.

CrashLoopBackOff, bir container'ın sürekli olarak çökmesi (crash) ve Kubernetes'in bu çökmeden sonra onu defalarca yeniden başlatmaya çalışması sonucu oluşan bir **geri çekilme (backoff)** durumudur.

Bu durumda Kubernetes, yeniden başlatma denemeleri arasına kademeli olarak daha uzun bekleme süreleri koyar (**exponential backoff**).

Bu durum genellikle aşağıdaki gibi senaryolarda oluşur:

- Uygulama hemen çöküyor (örneğin yanlış yapılandırılmış environment)
- Uygulama çalışıyor ama birkaç saniye içinde exit ediyor
- Gerekli bağımlılıklar (örneğin veritabanı bağlantısı) hazır değil

6.2 Pod Yaşam Döngüsü Uygulama

Öncelikle, canlı izleme amacıyla bir terminalde **kubectl get pods -w** komutunu çalıştırarak sistemdeki Pod durumlarını anlık olarak takip etmeye başladım. Ardından başka bir terminalde hatalı bir **.yaml** dosyasını **kubectl apply -f podlife1.yaml** komutuyla apply ettim. Bu dosya içinde image tanımı bilinçli olarak yanlış girilmişti. Böylece Kubernetes'in bu hatalı duruma nasıl tepki verdiğini adım adım gözlemledim.

```
sena@k8s-master:~/k8sfundamentals/pod$ cat podlife1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: imageerrorpod
spec:
  containers:
  - name: imageerrorcontainer
    image: enignx
```

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podlife1.yaml
pod/imageerrorpod created
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	3h23m
imageerrorpod	0/1	Pending	0	0s
imageerrorpod	0/1	Pending	0	0s
imageerrorpod	0/1	ContainerCreating	0	0s
imageerrorpod	0/1	ContainerCreating	0	0s
imageerrorpod	0/1	ErrImagePull	0	2s
imageerrorpod	0/1	ImagePullBackOff	0	14s
imageerrorpod	0/1	ErrImagePull	0	29s
imageerrorpod	0/1	ImagePullBackOff	0	43s
imageerrorpod	0/1	ErrImagePull	0	58s
imageerrorpod	0/1	ImagePullBackOff	0	73s

- İlk olarak, **imageerrorpod** isimli Pod başarıyla oluşturuldu ve **Pending** durumuna geçti. Bu aşamada Pod planlandı, tanımı **etcd** veritabanına kaydedildi, ancak henüz bir node üzerinde container çalıştırılmamıştı.
- Kısa bir süre sonra Pod, **ContainerCreating** aşamasına geçti. Bu, Kubernetes'in container'ı oluşturmak için belirtilen image'i çekmeye çalıştığını gösteriyordu.
- Ancak tanımda yer alan image hatalı olduğundan, sistem **ErrImagePull** hatasıyla karşılaştı. Bu durum, Kubernetes'in image'i çekemediğini ve işlemin başarısız olduğunu ifade eder.
- Bu hatanın ardından Kubernetes, belirli aralıklarla image'i tekrar çekmeye çalıştı. Ancak her deneme yine başarısız oldu ve Pod bu sefer **ImagePullBackOff** durumuna geçti. Bu, sistemin başarısız denemelerden sonra otomatik olarak beklemeye geçmesi anlamına gelir. Her yeni denemede bekleme süresi biraz daha uzar.

- Takip ettiğim terminal ekranında bu iki durum – **ErrImagePull** ve **ImagePullBackOff** – dönüşümlü olarak tekrarlandı. Pod hiçbir zaman Running duruma gelemedi çünkü image baştan hatalı tanımlanmıştı.

İkinci örneğimizde ise, terminalde çalışan kısa süreli bir uygulamanın Pod içerisinde nasıl davrandığını gözlemlemek için **podlife2.yaml** adlı bir YAML dosyası kullandım. Uygulama, **echo** komutuyla bir çıktı verdikten sonra 20 saniye bekleyip kapanacak şekilde tanımlanmıştı.

Pod tanımı içerisinde **restartPolicy: Never** olarak ayarlandığı için uygulama bir kere çalıştıktan sonra kapanacak ve kubelet tarafından yeniden başlatılmayacaktı.

İlk olarak **kubectl apply -f podlife2.yaml** komutuyla Pod'u oluşturduktan sonra, farklı bir terminal üzerinden **kubectl get pods -w** komutuyla süreci canlı olarak takip ettim.

Olaylar şu şekilde gelişti:

1. Pod **succeededpod**, oluşturulduktan sonra önce **Pending** durumuna geçti.
2. Ardından container oluşturulmaya başlandı ve Pod durumu **ContainerCreating** olarak değişti.
3. Kısa süre sonra uygulama çalışmaya başladı ve Pod durumu **Running** oldu.
4. Belirlenen komut (**echo "Hello, Kubernetes!" && sleep 20**) çalıştırıldı. Uygulama görevini başarıyla tamamladı.
5. Uygulama doğal olarak kapandı ve Pod durumu **Completed** olarak işaretlendi.
6. Çünkü **restartPolicy: Never** olduğu için Kubernetes bu container'ı yeniden başlatmadı.

```
sena@k8s-master:~/k8sfundamentals/pod$ cat podlife2.yaml
apiVersion: v1
kind: Pod
metadata:
  name: succeededpod
spec:
  restartPolicy: Never
  containers:
  - name: succeedcontainer
    image: ubuntu:latest
    command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 20']
sena@k8s-master:~/k8sfundamentals/pod$
```

Üçüncü örnekte ise, Pod içerisinde bilinçli olarak hatalı bir komut tanımlayarak uygulamanın başarısız olmasını ve Kubernetes'in buna nasıl tepki verdiğini inceledim.

podlife3.yaml adlı YAML dosyasında, **ubuntu:latest** image'i kullanılarak bir container tanımlandı. Ancak command satırında geçersiz bir komut (**abc**) verildi. Ayrıca **restartPolicy: Never** olarak ayarlandığı için container kapandığında yeniden başlatılmayacaktı.

Terminalden **kubectl apply -f podlife3.yaml** komutunu çalıştırarak Pod'u oluşturduktan sonra, **kubectl get pods -w** komutuyla süreci canlı olarak izledim.

```
sena@k8s-master:~/k8sfundamentals/pod$ cat podlife3.yaml
apiVersion: v1
kind: Pod
metadata:
  name: failedpod
spec:
  restartPolicy: Never
  containers:
  - name: failcontainer
    image: ubuntu:latest
    command: ['sh', '-c', 'abc']sena@k8s-master:~/k8sfundamentals/pod$
```

```
sena@k8s-master: ~/k8sfundamentals/pod
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podlife2.yaml
pod/succeededpod created
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podlife3.yaml
pod/failedpod created
sena@k8s-master:~/k8sfundamentals/pod$
```

```
sena@k8s-master: ~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
firstpod      1/1     Running   0           3h40m
succeededpod  0/1     Completed 0           3m57s
failedpod     0/1     Pending   0           0s
failedpod     0/1     Pending   0           0s
failedpod     0/1     ContainerCreating 0           0s
failedpod     0/1     ContainerCreating 0           1s
failedpod     0/1     Error      0           4s
failedpod     0/1     Error      0           5s
failedpod     0/1     Error      0           5s
```

Olaylar şu şekilde gerçekleşti:

1. **failedpod** isimli Pod başarıyla oluşturuldu ve ilk olarak Pending durumuna geçti.
2. Ardından Pod, container başlatma aşamasına geçti ve durumu **ContainerCreating** oldu.
3. Ancak container içindeki komut geçersiz olduğu için sistem komutu çalıştıramadı ve uygulama anında hata verdi.
4. Bu nedenle Pod'un durumu çok kısa sürede **Error** olarak işaretlendi.
5. **restartPolicy: Never** olduğu için Kubernetes herhangi bir yeniden başlatma denemesi yapmadı. Pod, hata verdikten sonra bu şekilde kaldı.

Son .yaml dosyamızda ise, daha önce kullandığımız **echo** komutunu içeren kısa ömürlü bir container uygulamasını bu kez **restartPolicy: Always** ile tanımlayarak Kubernetes'in yeniden başlatma davranışını gözlemledim.

podlife4.yaml adlı YAML dosyasında, **ubuntu:latest** image'i ile çalışan bir Pod tanımlandı. Uygulama terminal çıktısı verdikten sonra 20 saniye bekleyip kapanacak şekilde tasarlanmıştı. Ancak bu kez **restartPolicy: Always** olarak ayarlanmıştı, yani container ne şekilde kapanırsa kapersin kubelet tarafından **her defasında yeniden başlatılacaktı**.

Uygulamayı başlatmak için **kubectl apply -f podlife4.yaml** komutunu çalıştırdım. Ardından farklı bir terminalde **kubectl get pods -w** ile Pod'un durumunu canlı olarak izlemeye başladım.

```
sena@k8s-master:~/k8sfundamentals/pod$ cat podlife4.yaml
apiVersion: v1
kind: Pod
metadata:
  name: crashloopbackpod
spec:
  restartPolicy: Always
  containers:
  - name: crashloopbackcontainer
    image: ubuntu:latest
    command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 20']
sena@k8s-master:~/k8sfundamentals/pod$
```

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podlife4.yaml
pod/crashloopbackpod created
sena@k8s-master:~/k8sfundamentals/pod$
```

```
sena@k8s-master:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
firstpod      1/1     Running   0           3h46m
crashloopbackpod 0/1     Pending   0           0s
crashloopbackpod 0/1     Pending   0           0s
crashloopbackpod 0/1     ContainerCreating 0           0s
crashloopbackpod 0/1     ContainerCreating 0           1s
crashloopbackpod 1/1     Running   0           3s
crashloopbackpod 0/1     Completed 0           24s
crashloopbackpod 1/1     Running   1 (3s ago)  26s
crashloopbackpod 0/1     Completed 1 (23s ago) 46s
crashloopbackpod 0/1     CrashLoopBackOff 1 (16s ago) 61s
crashloopbackpod 1/1     Running   2 (19s ago) 64s
crashloopbackpod 0/1     Completed 2 (39s ago) 84s
crashloopbackpod 0/1     CrashLoopBackOff 2 (15s ago) 98s
```

1. **crashloopbackpod** adlı Pod oluşturuldu ve **Pending** durumuna geçti.
2. Kısa süre sonra Kubernetes, container'ı başlatmak için çalıştı ve durum **ContainerCreating** oldu.
3. Uygulama başarıyla başlatıldı, **echo** komutu çalıştı ve 20 saniye sonra container sonlandı. Pod bu noktada Completed durumuna geçti.
4. **restartPolicy: Always** olduğu için kubelet, container'ı yeniden başlattı. Pod tekrar **Running** durumuna döndü.
5. Uygulama yeniden çalışıp kapandıktan sonra yine **Completed** oldu.
6. Kubernetes bu döngüyü sürdürdü. Ancak arka arkaya birkaç defa aynı şekilde çalışıp kapanan container'lar kubelet tarafından "sorun olabilir" şeklinde yorumlandı.
7. Bu noktadan itibaren sistem bu durumu **CrashLoopBackOff** olarak değerlendirdi. Pod, artık yeniden başlatılmadan önce bekleme süresine girdi (exponential backoff).
8. Her yeniden başlatma girişiminden sonra aynı döngü tekrarlandı: **Running** → **Completed** → **CrashLoopBackOff**

7.Multi Container Pod

7.1 Çoklu Container Pod

Örneğin WordPress uygulamasını deploy etmek istediğimizi düşünelim. Bildiğiniz üzere WordPress, PHP tabanlı bir frontend katmanına ve tüm verilerin tutulduğu bir MySQL veritabanına ihtiyaç duyar. Bu iki bileşeni container ortamında çalıştırmak istiyoruz. Peki şu soruyu soralım:

WordPress ve MySQL'i tek bir container içinde barındırabilir miyiz?

Teknik olarak evet, her iki uygulamayı da aynı container image'ına dahil edebilirsiniz. Ancak bu, konteyner mimarisinin temel prensiplerine aykırıdır ve bu nedenle genellikle tercih edilmez.

Bunun yerine, WordPress ve MySQL'i **ayrı container'lar** olarak çalıştırmak en doğru yaklaşımdır. Bu container'lar aynı **Pod** içerisinde yer alabilirler ancak her biri kendi ortamında, izole şekilde çalışır.

Peki neden bu ayrımı yapıyoruz?

Cevap oldukça net: **İzolasyon**.

Container teknolojisinin temel avantajlarından biri uygulamaların birbirinden izole çalışabilmesidir. Böylece her bileşenin yaşam döngüsü ayrı yönetilir, kaynak kullanımı ayrılır ve uygulamalar birbiriyle doğrudan bağımlı hale gelmez. Bu sayede ölçeklenebilirlik, güvenlik ve bakım kolaylığı sağlanmış olur.

Uygulama

Senaryomuz şu olacak: Nginx image'inden oluşan bir web uygulamamız olacak ve bunu kubernetes'e bir pod olarak deploy edeceğiz. Bu nginx container bizim web sayfamızı publish edecek. Bunun yanında bu pod içinde bir tane daha container oluşturacağız. Bu iki container'a da bir volume bağlı olacak. Bu sayede her iki container'dan da erişilebilen bir depolama çözümüne sahip olacaklar.

Bu ikinci sidecar olarak adlandıracağımız container'ın ana görevi şu olacak: bizler bu web sayfasını oluşturan dosyaları dışarıda bir github repository'de tutuyoruz. Bu sidecar container'da bulunan uygulama, github repository'i belirli aralıklarla kontrol edecek ve web sayfamızda bir güncelleme varsa bunu download ederek volume'un mount edildiği klasöre kaydedecektir.

Nginx image'inden oluşturduğumuz container'da bu volume'e erişebildiği için buradaki dosyaları sunacak. Böylelikle web sayfamız güncellenmiş olacak.

Öncelikle daha önce birçok pod oluşturduğumuz için daha önce oluşturduğumuz cluster'ı siliyorum kafa karışıklığı yaratmaması için

minikube delete komutu ile. Daha sonra tekrar **minikube start** komutu ile sıfırdan bir kubernetes cluster'ı oluşturduk.

/k8sfundamentals/pod dizinin altında **podmulticontainer.yaml** dosyasını nano ile açtım.

([GitHub - aytitech/k8sfundamentals](https://github.com/aytitech/k8sfundamentals): Ayti.tech tarafından sunulan "Kubernetes Temelleri" eğitiminde gerekli dosyaların barındırılması için oluşturulan GitHub repository'si.)

```
apiVersion: v1
kind: Pod
metadata:
  name: multicontainer
spec:
  containers:
    - name: webcontainer
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - name: sharedvolume
          mountPath: /usr/share/nginx/html
    - name: sidecarcontainer
      image: busybox
      command: ["/bin/sh"]
      args: ["-c", "while true; do wget -O /var/log/index.html https://raw.githubusercontent.com/ozgurozturknet/hello-world/master/index.html; sleep 15; done"]
      volumeMounts:
        - name: sharedvolume
          mountPath: /var/log
  volumes:
    - name: sharedvolume
      emptyDir: {}
```

Daha önceden gördüğümüz şeylerle benzer aslında fakat burada iki tane container tanımlıyoruz. Ve bunun yanında **sharedvolume** adında bir volume tanımlıyoruz. İlk uygulamamız bizim ana uygulamamız. nginx image'inden oluşturuyoruz, 80 portun expose ediyor ve oluşturulacak volume'u **/usr/share/nginx/html** altına mount ediyoruz. Bu klasör, nginx'in web sayfalarında sunduğu klasör. nginx uygulaması çalıştığı zaman bu klasöre bakar ve içerisinde index.html adında bir dosya arar. Kısacası bizim sayfamızın konuşlanması gereken yer burası. Biz volume'u bu folder'a mount ediyoruz.

İkinci container'ımız ise **sidecarcontainer** olarak adlandırılıyor ve **busybox** adlı image'dan oluşturuyoruz. Aynı volume'de bu container'ın **/var/log** klasörüne mount ediyoruz.

Dolayısıyla nginx container'ı da **/usr/share/nginx/html** ve bu sidecarcontainer'daki **/var/log** klasörü aynı yere bakıyor olacak. Bu klasörde oluşturulacak her şeyi iki container'da bu path'lerden görebilecek.

(**sidecarcontainer**: Her 15 saniyede bir belirtilen index.html dosyasını indirip /var/log/index.html olarak kaydeder.)

Ortak Volume: sharedvolume

Kısaca Ne Oluyor?

- sidecarcontainer, internetten dosya indirir ve ortak volume'a yazar.
- nginx, bu dosyayı servis eder.
- Böylece iki container aynı Pod içinde, ortak dosya üzerinden iletişim kurar.

Şimdi bunu deploy edelim.

kubectl apply -f podmulticontainer.yaml komutu ile created ediyorum.

kubectl get pods komutu ile teyit ediyorum.

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
multicontainer 2/2     Running   0           32s
sena@k8s-master:~/k8sfundamentals/pod$
```

İki ayrı terminal açtım ve iki farklı container'a bağlanacağım.

Hangi container üzerinde işlem yapacağımızı belirtmek için **-c** parametresi kullanılır.

kubectl exec -it multicontainer -c webcontainer -- /bin/sh

```
sena@k8s-master: ~
sena@k8s-master:~$ kubectl exec -it multicontainer -c webcontainer -- /bin/sh
# hostname
multicontainer
#
```

Diğer terminalde ise aşağıdaki komutu çalıştırdım:

kubectl exec -it multicontainer -c sidecarcontainer -- /bin/sh

Öncelikle webcontainer'da ifconfig komutu içinde olmadığı için kuralım.

apt update

apt install net-tools

Daha sonra iki terminalde de **ifconfig** komutunu çalıştırıyorum.

```
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.244.0.3 netmask 255.255.0.0 broadcast 10.244.255.255
    inet6 fe80::9050:67ff:fea1:1710 prefixlen 64 scopeid 0x20<link>
    ether 92:50:67:a1:17:10 txqueuelen 0 (Ethernet)
    RX packets 5059 bytes 10650820 (10.1 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3859 bytes 328114 (320.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
#
```

```
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 92:50:67:A1:17:10
          inet addr:10.244.0.3  Bcast:10.244.255.255  Mask:255.255.0.0
          inet6 addr: fe80::9050:67ff:fea1:1710/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5145 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3938 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:10686304 (10.1 MiB)  TX bytes:335864 (327.9 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128  Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/ #
```

Gördüğünüz üzere her ikisininde IP adresi aynı (10.244.0.3)

Aynı pod içerisinde oluşturulan container'larda network izolasyonu olmaz.

Sidecar ve web container'ları birbirlerinin process'lerine localhost üstünden erişebilirler. Network bakımından aynı makinede çalışıyor gibiler.

Şimdi folder'lara geçiş yapıyorum.

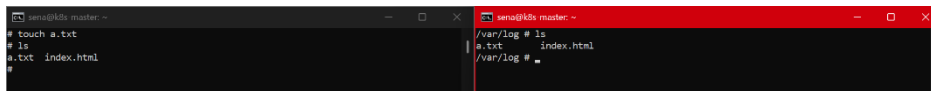
webcontainer → **/usr/share/nginx/html**

sidecarcontainer → **/var/log**

Bu iki klasörü de aynı volume'e mount etmiştik.

Peki bu ne demek?

Örneğin web container'da touch komutu ile a.txt diye bir dosya oluşturduğum zaman sidecar container'da da /var/log'un altında da görünür.



```
sema@k8s-master:~$ touch a.txt
sema@k8s-master:~$ ls
a.txt  index.html

sema@k8s-master:~$ cd /var/log
sema@k8s-master:/var/log$ ls
a.txt  index.html
```

sidecarcontainer'ın loglarına bakmak istediğim zaman

kubectl logs multicontainer -c sidecarcontainer

8.Init Container

Init container, bir pod içerisinde birden fazla container tanımlanmasını sağlayan bir yapıdır. Ancak uygulama container'lardan farklı olarak, init container'lar yalnızca pod'un başlangıç aşamasında çalışır ve görevini tamamladıktan sonra durur.

- Pod başlatıldığında **ilk olarak init container çalıştırılır.**
- Init container, kendisine verilen görevleri tamamladıktan sonra kapanır.
- **Tüm init container'lar başarıyla tamamlanmadan uygulama container'ları başlatılmaz.**

8.1 Init Container Neden Kullanılır?

Bazı durumlarda, uygulama container'ı başlamadan önce yerine getirilmesi gereken hazırlık adımları olabilir. Örneğin, bir web uygulamanızın sağlıklı çalışabilmesi için önce PostgreSQL veritabanının erişilebilir durumda olması gerekiyorsa, init container kullanarak pod başlamadan önce bu bağlantıyı kontrol edebilirsiniz.

Ayrıca uygulamanın ihtiyaç duyduğu konfigürasyon dosyalarını uzaktaki bir sunucudan çekmek, dosya sistemi üzerinde belirli klasörleri oluşturmak ya da bir DNS çözümlemesini test etmek gibi işlemler init container'lar aracılığıyla güvenli ve kontrollü şekilde gerçekleştirilebilir. Böylece uygulama container'ı yalnızca sistemin hazır olduğu durumlarda devreye girer.

8.2 Init Container Uygulama

Eğitim repository'si altında yer alan **podinitcontainer.yaml** dosyasını inceleyelim:

YAML tanımında görüldüğü üzere, pod içerisinde iki farklı container tanımlanmıştır:

- **appcontainer** adlı ana uygulama container'ı
- **initcontainer** adlı ön hazırlık görevini üstlenen init container

```

sena@k8s-master: ~/k8sfundamentals/pod
GNU nano 7.2 podinitcontainer.yaml
apiVersion: v1
kind: Pod
metadata:
  name: initcontainerpod
spec:
  containers:
    - name: appcontainer
      image: busybox
      command: ['sh', '-c', 'echo The app is running! && sleep 3600']
    initContainers:
      - name: initcontainer
        image: busybox
        command: ['sh', '-c', 'until nslookup myservice; do echo waiting for myservice; sleep 2; done']

```

Multicontainer yapılarında tüm container'lar **containers** alanı altında tanımlanırken, init container'lar **initContainers** başlığı altında ayrı olarak tanımlanır.

Init container içerisinde tanımlı komut kısmında, öncelikle bir **nslookup** işlemi yapılmaktadır. Bu işlem, DNS üzerinde **myservice** isimli servisin çözülüp çözülmediğini kontrol eder. Eğer

bu servis henüz erişilebilir değilse, komut ekrana **"waiting for myservice"** mesajını yazdırır ve 2 saniye bekledikten sonra işlemi tekrar eder.

Bu döngü, **nslookup myservice** komutundan başarılı bir sonuç alıncaya kadar devam eder. Servis çözümlendiğinde, init container görevini tamamlamış sayılır ve kapanır. Ardından, **appcontainer** çalışmaya başlar.

kubectl apply -f podinitcontainer.yaml komutu ile container'ımızı oluşturalım.

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podinitcontainer.yaml
pod/initcontainerpod created
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
initcontainerpod    0/1     Init:0/1   0           11s
multicontainer      2/2     Running   0           90m
sena@k8s-master:~/k8sfundamentals/pod$
```

Görsel de görüldüğü üzere kubernetes, init container'ın başarılı bir şekilde tamamlanmasını bekler. Bu işlem bitmeden **appcontainer çalıştırılmaz**. Init container'ın görevi bittikten sonra **STATUS** alanı Running olarak güncellenir ve app container'ı devreye girer.

kubectl logs -f initcontainerpod -c initcontainer komutu ile loglarına bakıyorum.

```
sena@k8s-master: ~/k8sfundamentals/pod
** server can't find myservice.default.svc.cluster.local: NXDOMAIN
** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.cluster.local: NXDOMAIN
** server can't find myservice.default.svc.cluster.local: NXDOMAIN
** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.cluster.local: NXDOMAIN

waiting for myservice
Server:      10.96.0.10
Address:     10.96.0.10:53

** server can't find myservice.cluster.local: NXDOMAIN
** server can't find myservice.default.svc.cluster.local: NXDOMAIN
** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.default.svc.cluster.local: NXDOMAIN
** server can't find myservice.cluster.local: NXDOMAIN

waiting for myservice
```

** server can't find myservice.default.svc.cluster.local: NXDOMAIN hatasını görüyoruz.

nslookup komutunun myservice adında bir DNS kaydı bulamadığını göstermektedir.

Init container içinde tanımlı olan komut şu şekildeydi:

command: ['sh', '-c', 'until nslookup myservice; do echo waiting for myservice; sleep 2; done']

myservice isimli DNS kaydı çözülene kadar nslookup sorgusunu her 2 saniyede bir tekrar eder. Eğer cevap alınamazsa "waiting for myservice" mesajı ekrana yazdırılır ve döngü devam eder.

Bu yüzden de bu işlem tamamlanmadığı için ana appcontainer'ımızı başlatmadı.

Aynı klasör içerisinde **service1.yaml** dosyası var.

```

sena@k8s-master: ~/k8sfundamentals/pod
GNU nano 7.2 service1.yaml *
apiVersion: v1
kind: Service
metadata:
  name: myservice
spec:
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376

```

YAML dosyasına baktığımızda **myservice** adında bir **Service** objesi tanımlanmaktadır. Bu servis, pod'lar arasında ağ üzerinden erişim sağlamak amacıyla kullanılır.

Bu tanımla birlikte, Kubernetes DNS sistemi artık **myservice** adını çözümleyebilir hale gelir. Böylece, init container içerisindeki nslookup myservice komutu başarılı şekilde sonuçlanır. Bu da init container'ın kapanmasına ve ana uygulama container'ının (appcontainer) başlatılmasına olanak tanır.

Şimdi bu .yaml dosyasını **kubectl apply -f service1.yaml** komutu ile apply edelim.

```

sena@k8s-master: ~/k8sfundamentals/pod
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f service1.yaml
service/myservice created
sena@k8s-master:~/k8sfundamentals/pod$ _

```

Logları canlı olarak izliyorduk, loglara baktığımız zaman nslookup komutuna bir cevap döndüğünü ve bu sayede IP adresini öğrenebildik.

```

** server can't find myservice.cluster.local: NXDOMAIN
waiting for myservice
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:  myservice.default.svc.cluster.local
Address: 10.111.108.49

** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.cluster.local: NXDOMAIN

** server can't find myservice.svc.cluster.local: NXDOMAIN
** server can't find myservice.cluster.local: NXDOMAIN

waiting for myservice

```

```

sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
initcontainerpod    1/1     Running   0           33s
multicontainer      2/2     Running   0           4h7m
sena@k8s-master:~/k8sfundamentals/pod$ _

```

Görüldüğü üzere init container, **myservice.default.svc.cluster.local** adresini başarılı şekilde çözümlendikten sonra görevini tamamladı. Ardından pod yaşam döngüsünde bir sonraki adıma geçildi ve **appcontainer** çalışmaya başladı. **kubectl get pods** çıktısında da bu durum net şekilde doğrulanmakta: Pod'un STATUS alanı artık Running olarak görünmekte ve READY sütununda 1/1 ifadesi yer almakta, yani ana container aktif olarak çalışıyor.

```

sena@k8s-master: ~
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.108.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.108.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.108.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.111.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.108.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.108.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
Connecting to raw.githubusercontent.com (185.199.109.133:443)
wget: note: TLS certificate validation not implemented
saving to '/var/log/index.html'
index.html 100% |*****| 165 0:00:00 ETA
'/var/log/index.html' saved
sena@k8s-master:~$

```

Gördüğünüz üzere sidecar container, her 15 saniyede bir GitHub üzerinden **index.html** dosyasını indirerek shared volume'a yazmaktadır.

8.3 Pod Üzerinden Web Servise Erişim (Port Yönlendirme)

kubectl port-forward pod/multicontainer 8080:80 (Pod içindeki Nginx'in dinlediği 80 portunu, local makinedeki 8080 portuna yönlendirir. Bu sayede tarayıcı üzerinden <http://localhost:8080> ile erişim sağlanabilir.)

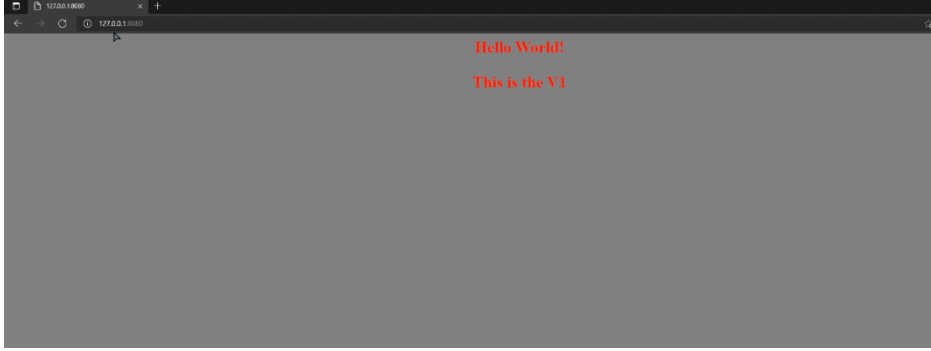
kubectl exec -it multicontainer -c webcontainer -- netstat -tlnp ile dinlediği port kontrol edilebilir. Port 80 görülmeli.

```

sena@k8s-master:~$ kubectl port-forward pod/multicontainer 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80

```


Not: Ubuntu server'a Windows makinem üzerinden SSH ile bağlandığım ve kurum ağı içerisindeki bir IP adresini kullandığım için, port yönlendirme adımlarını atlardım. Ancak local bir kurulum senaryosunda, aşağıdaki ekran görüntüsünün alınması beklenir:



GitHub üzerindeki **index.html** dosyasında herhangi bir değişiklik yapılması durumunda, sidecar container tarafından her 15 saniyede bir gerçekleştirilen **wget** çağrısı sayesinde bu değişiklik Pod içerisindeki shared volume'a otomatik olarak yansıtılır.

9.Label ve Selector

Tag'ler kubernetes'de her türlü objeye atayabildiğimiz anahtar-değer eşlenikleridir. Etiketler sayesinde biz oluşturduğumuz objeleri bizlerin anlayacağı ve gruplama yaparken kullanabileceğimiz bilgiler eklemiş oluruz. Bu sayede kubernetes tarafından bizlere core özellik olarak sunulmayan objelere belirli bir aidiyet ataması işlemini gerçekleştirebiliriz.

Bir Kubernetes label'ı genellikle şu yapıya sahiptir:

[example.com/tier](#): front-end

Bu yapı üç bileşenden oluşur:

- **Prefix (Önek):** [example.com/](#)
 - Opsiyoneldir, yani kullanılması zorunlu değildir.
 - [kubernetes.io/](#) ve [k8s.io/](#) gibi önekler, Kubernetes çekirdek bileşenleri için ayrılmıştır ve sadece sistem bileşenleri tarafından kullanılır.
- **Anahtar:** tier
 - Anahtar adı en fazla 63 karakter olabilir.
 - Alfaniümerik bir karakterle başlamalı ve bitmelidir (a-z, 0-9, A-Z).
 - Tire (-), alt çizgi (_) ve nokta (.) karakterlerine izin verilir.
- **Değer:** front-end
 - Değer kısmı da en fazla 63 karakter olabilir.
 - Boş bırakılabilir (değer olmayabilir).
 - Alfaniümerik karakterler ve belirli semboller içerebilir.

Örnek dosyamız: **podlabel.yaml**

```

sena@k8s-master: ~/k8sfundamentals/pod
GNU nano 7.2 podlabel.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  labels:
    app: firstapp
    tier: frontend
    mycluster.local/team: team1
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: pod2
  labels:
    app: firstapp
    tier: frontend
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: pod3
  labels:
    app: firstapp
    tier: backend

```

Bu dosyada 11 farklı pod tanımı var ve hepsinin üzerinde çeşitli label'lar tanımlanmış. Objelere istediğimiz kadar label atayabiliriz fakat tek bir kuralımız var: bir pod'a aynı anahtarı iki defa atayamayız.

Şimdi bu dosyayı **kubectl apply -f podlabel.yaml** komutu ile apply edelim. Toplamda 11 tane pod oluşturuldu.

```

sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podlabel.yaml
pod/pod1 created
pod/pod2 created
pod/pod3 created
pod/pod4 created
pod/pod5 created
pod/pod6 created
pod/pod7 created
pod/pod8 created
pod/pod9 created
pod/pod10 created
pod/pod11 created
sena@k8s-master:~/k8sfundamentals/pod$

```

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
initcontainerpod    1/1     Running   1 (30m ago) 90m
multicontainer      2/2     Running   0           5h37m
pod1                 1/1     Running   0           3m59s
pod10                1/1     Running   0           3m59s
pod11                0/1     Pending   0           3m59s
pod2                 1/1     Running   0           3m59s
pod3                 1/1     Running   0           3m59s
pod4                 1/1     Running   0           3m59s
pod5                 1/1     Running   0           3m59s
pod6                 1/1     Running   0           3m59s
pod7                 1/1     Running   0           3m59s
pod8                 1/1     Running   0           3m59s
pod9                 1/1     Running   0           3m59s
sena@k8s-master:~/k8sfundamentals/pod$
```

kubectl get pods -l "app" --show-labels komutu ile üzerinde app label'ı tanımlı olan Pod'ları getirdik. Yani hangi Pod'ların içinde app adında bir label olduğunu sorguladık. --show-labels parametresi sayesinde sadece app değil, o Pod üzerinde tanımlı tüm label bilgilerini de görmüş olduk.

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods -l "app" --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
pod1    1/1     Running   0           6m30s app=firstapp,mycluster.local/team=team1,tier=frontend
pod2    1/1     Running   0           6m30s app=firstapp,tier=frontend
pod3    1/1     Running   0           6m30s app=firstapp,tier=backend
pod4    1/1     Running   0           6m30s app=firstapp,tier=backend
pod5    1/1     Running   0           6m30s app=secondapp,tier=frontend
pod6    1/1     Running   0           6m30s app=secondapp,tier=frontend
pod7    1/1     Running   0           6m30s app=secondapp,tier=backend
pod8    1/1     Running   0           6m30s app=secondapp,tier=backend
sena@k8s-master:~/k8sfundamentals/pod$
```

kubectl get pods -l "app=firstapp, tier=frontend" --show-labels

Bu komutla birlikte, üzerinde hem **app=firstapp** hem de **tier=frontend** label'ı bulunan Pod'ları filtrelemiş olduk. **-l** parametresi ile bu iki label'ı yazarken, araya **virgül (,) koyduk**. Bu virgül, bize "ve" anlamı kazandırdı. Yani:

"app label'ı firstapp olacak ve aynı zamanda tier label'ı frontend olacak" dedik.

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods -l "app=firstapp, tier=frontend" --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
pod1    1/1     Running   0           12m   app=firstapp,mycluster.local/team=team1,tier=frontend
pod2    1/1     Running   0           12m   app=firstapp,tier=frontend
sena@k8s-master:~/k8sfundamentals/pod$
```

kubectl get pods -l "app=firstapp, tier!=frontend" --show-labels

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods -l "app=firstapp, tier!=frontend" --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
pod3    1/1     Running   0           17m   app=firstapp,tier=backend
pod4    1/1     Running   0           17m   app=firstapp,tier=backend
sena@k8s-master:~/k8sfundamentals/pod$
```

Bu komutu çalıştırarak, **label'ları içinde app=firstapp olan fakat tier label'ı frontend olmayan Pod'ları** listelemiş olduk.

Burada önemli olan kısım **tier!=frontend** ifadesidir. Bu ifade sayesinde, belirli bir değere sahip olmayan Pod'ları filtreleyebiliyoruz. Yani:

app label'ı firstapp olan, ama tier label'ı kesinlikle frontend olmayan Pod'lar gelsin dedik.

Virgül (,) operatörü burada da yine bir **"ve" bağlacı gibi** çalıştı. İki koşul aynı anda geçerli olmalı dedik:

- Pod'un **app=firstapp** olması gerekiyor.
- **tier=frontend olmaması** gerekiyor.

Bu komutla, **negation (dışlama) filtreleme işlemi** yapmış olduk.

kubectl get pods -l "app, tier=frontend" --show-labels komutu sayesinde ise:

app label'ı tanımlı olacak ve aynı zamanda **tier=frontend** olan Pod'lar listelenecek.

```
sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods -l "app, tier=frontend" --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
pod1    1/1     Running   0           20m   app=firstapp,mycluster.local/team=team1,tier=frontend
pod2    1/1     Running   0           20m   app=firstapp,tier=frontend
pod5    1/1     Running   0           20m   app=secondapp,tier=frontend
pod6    1/1     Running   0           20m   app=secondapp,tier=frontend
sena@k8s-master:~/k8sfundamentals/pod$
```

kubectl get pods -l 'app in (firstapp, secondapp)' --show-labels

```
sena@k8s-master:~$ kubectl get pods -l 'app in (firstapp, secondapp)' --show-labels
NAME    READY   STATUS    RESTARTS   AGE   LABELS
pod1    1/1     Running   0           17h   app=firstapp,mycluster.local/team=team1,tier=frontend
pod2    1/1     Running   0           17h   app=firstapp,tier=frontend
pod3    1/1     Running   0           17h   app=firstapp,tier=backend
pod4    1/1     Running   0           17h   app=firstapp,tier=backend
pod5    1/1     Running   0           17h   app=secondapp,tier=frontend
pod6    1/1     Running   0           17h   app=secondapp,tier=frontend
pod7    1/1     Running   0           17h   app=secondapp,tier=backend
pod8    1/1     Running   0           17h   app=secondapp,tier=backend
sena@k8s-master:~$
```

Bu komutla birlikte, **label selector ifadesi kullanarak app** label'ı değeri **firstapp** veya **secondapp** olan tüm Pod'ları listeledik.

Buradaki **in** ifadesi sayesinde:

- app=firstapp
 - app=secondapp
- label'lerinden herhangi birine sahip olan Pod'lar filtrelenmiş oldu.

9.1 Label

kubectl label pods pod9 app=thirdapp komutu ile pod9 isimli Pod'a app=thirdapp şeklinde yeni bir label ekledik.

```
sena@k8s-master:~$ kubectl label pods pod9 app=thirdapp
pod/pod9 labeled
sena@k8s-master:~$ kubectl get pods --show-labels
NAME                READY   STATUS    RESTARTS   AGE   LABELS
initcontainerpod    1/1     Running   0           18h   <none>
multicontainer       2/2     Running   0           22h   <none>
pod1                 1/1     Running   0           17h   app=firstapp,mycluster.local/team=team1,tier=frontend
pod10                1/1     Running   0           17h   team=team2
pod11                0/1     Pending   0           17h   <none>
pod2                 1/1     Running   0           17h   app=firstapp,tier=frontend
pod3                 1/1     Running   0           17h   app=firstapp,tier=backend
pod4                 1/1     Running   0           17h   app=firstapp,tier=backend
pod5                 1/1     Running   0           17h   app=secondapp,tier=frontend
pod6                 1/1     Running   0           17h   app=secondapp,tier=frontend
pod7                 1/1     Running   0           17h   app=secondapp,tier=backend
pod8                 1/1     Running   0           17h   app=secondapp,tier=backend
pod9                 1/1     Running   0           17h   app=thirdapp,team=team1
```

Kubernetes'te **normal şartlar altında kube-scheduler**, Pod'ların hangi Node üzerinde çalışacağını kendi algoritmasına göre otomatik olarak belirler. Ancak bazı durumlarda, belirli bir Pod'un sadece **özellleştirilmiş özelliklere sahip bir Node üzerinde çalışmasını** isteyebiliriz. İşte bu tür durumlarda **nodeSelector** alanını kullanırız.

```
GNU nano 7.2 podlabel.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod10
  labels:
    team: team2
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: pod11
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
  nodeSelector:
    hddtype: ssd
```

nodeSelector alanında, pod11'in hangi node üzerinde çalışması gerektiği açıkça belirtilmiştir.

Bu sayede, yalnızca **hddtype=ssd** label'ına sahip olan node'lar hedef alınmış olur. Kubernetes scheduler, bu şartı taşıyan bir node bulana kadar Pod'u çalıştırmaz.

Eğer belirtilen label'a sahip bir node mevcut değilse, Pod **Pending** durumunda kalır ve çalıştırılmaz.

```

sena@k8s-master:~/k8sfundamentals/pod$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
initcontainerpod    1/1     Running   19 (20s ago)  19h
multicontainer       2/2     Running   0           23h
pod1                 1/1     Running   0           17h
pod10                1/1     Running   0           17h
pod11                0/1     Pending   0           17h
pod2                 1/1     Running   0           17h
pod3                 1/1     Running   0           17h
pod4                 1/1     Running   0           17h
pod5                 1/1     Running   0           17h
pod6                 1/1     Running   0           17h
pod7                 1/1     Running   0           17h
pod8                 1/1     Running   0           17h
pod9                 1/1     Running   0           17h
sena@k8s-master:~/k8sfundamentals/pod$

```

Bizim kubernetes cluster'ımızda böyle bir node olmadığı için kubescheduler bu kritere sahip node bulamadığı için bu podu oluşturmadı ve pending durumda kaldı.

Şimdi node'umuza bu label'i ekleyeceğiz.

kubectl get nodes --show-labels ile varsayılan node'u sıralıyorum ve **kubectl get pods -w** ile canlı olarak izliyorum.

```

sena@k8s-master:~/k8sfundamentals/pod$ kubectl get nodes --show-labels
NAME        STATUS    ROLES    AGE   VERSION   LABELS
minikube    Ready     control-plane  23h   v1.33.1   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=minikube,kubernetes.io/os=linux,minikube.k8s.io/commit=f8f52f5de11fc6ad8244afac475e1d0f96841df1-dirty,minikube.k8s.io/name=minikube,minikube.k8s.io/primary=true,minikube.k8s.io/updated_at=2025_06_17T07_20_16_0700,minikube.k8s.io/version=v1.36.0,node-role.kubernetes.io/control-plane=node.kubernetes.io/exclude-from-external-load-balancers=
sena@k8s-master:~/k8sfundamentals/pod$

```

kubectl labels nodes minikube hddtype=ssd ile bir tane label ekliyorum ve sonra Kubernetes scheduler devreye girdi, **nodeSelector** şartını sağlayan bir node olduğunu fark etti ve daha önce **Pending** durumda kalan Pod otomatik olarak bu node üzerinde çalıştırıldı.

```

sena@k8s-master:~$ kubectl get pods -w
NAME                READY   STATUS    RESTARTS   AGE
initcontainerpod    1/1     Running   19 (6m14s ago)  19h
multicontainer       2/2     Running   0           23h
pod1                 1/1     Running   0           17h
pod10                1/1     Running   0           17h
pod11                0/1     Pending   0           17h
pod2                 1/1     Running   0           17h
pod3                 1/1     Running   0           17h
pod4                 1/1     Running   0           17h
pod5                 1/1     Running   0           17h
pod6                 1/1     Running   0           17h
pod7                 1/1     Running   0           17h
pod8                 1/1     Running   0           17h
pod9                 1/1     Running   0           17h
pod11                0/1     Pending   0           17h
pod11                0/1     ContainerCreating  0           17h
pod11                1/1     Running   0           17h

```

10. Annotation

Annotation'lar, bir objeyi tanımlamak için kullanılır fakat bu bilgilerin label olarak eklenmesi uygun olmadığında devreye girer. Yani objeye ait açıklayıcı ama filtreleme ya da seçim için kullanılmayan bilgileri annotation olarak ekleriz.

example.com/notification-email: admin@k8sfundamentals.com

Bu yapı üç temel bileşenden oluşur:

- **Prefix (Önek):** example.com/ gibi. Bu kısım opsiyoneldir, yani zorunlu değildir.
- **Anahtar (Key):** **notification-email** gibi. Anahtar alanı en fazla 63 karakter olmalıdır.
- **Değer (Value):** admin@k8sfundamentals.com gibi. Annotation değeri herhangi bir karakter içerebilir, alfanümerik olmak zorunda değildir.

Anahtar alanı için geçerli kurallar:

- Anahtar alfanümerik bir karakterle (a-z, A-Z, 0-9) başlamalı ve bitmelidir.
- Tire (-), alt çizgi (_) ve nokta (.) karakterleri arasında alfanümerik değerler olabilir.

11. Namespace

Diyelim ki kurumumuzda toplam **10 farklı ekip** (ekip1, ekip2, ... ekip10) aktif olarak çalışıyor ve hepsi ortak bir dosya sunucusu olan **fileserver** üzerinden kendi klasörlerine erişiyor. Bu dosya sunucusu içinde /share dizini altında her ekibin ayrı bir klasörü var: /share/ekip1, /share/ekip2, ..., /share/ekip10

Bu yapıda her ekip aynı sistem üzerinde çalışıyor ve teorik olarak sadece kendi dizinine erişmesi gerekiyor. Ancak uygulamada çeşitli problemlerle karşılaşabiliyoruz:

Karşılaşılan Zorluklar

- Tüm ekipler aynı sunucuyu kullanıyor. Yanlışlıkla başka bir ekibin klasörüne erişim veya dosya üzerine yazma gibi durumlar yaşanabiliyor.
- Erişim yetkilerini tek tek dizin bazında ayarlamak zaman alıyor ve merkezi olarak yönetmek zorlaşıyor.
- Hangi ekip ne yapmış, hangi dosyaya erişmiş gibi bilgileri takip etmek zorlaşıyor.
- Ortak bir ortamda çalışırken ekiplerin log dosyaları, temporary dosyaları veya cache dizinleri çakışabiliyor.

İşte tam da bu noktada **Kubernetes Namespace** kavramı devreye giriyor. Her ekibe bir klasör vermek yerine, Kubernetes dünyasında **her ekibe ayrı bir namespace** tanımlıyoruz.

Namespace'ler, adlar için bir kapsam sağlar. Kaynak adlarının bir namespace içinde benzersiz olması gerekir.

Namespace'ler birbirinin içine yerleştirilemez ve her Kubernetes kaynağı yalnızca bir namespace içinde olabilir.

Namespace'ler, bir Kubernetes cluster'ındaki kaynakları birden çok kullanıcı arasında bölmenin bir yoludur.

Her kubernetes kurulumunda varsayılan olarak dört namespace oluşturulur.

```
sena@k8s-master:~$ kubectl get namespaces
NAME                STATUS    AGE
default              Active    24h
kube-node-lease      Active    24h
kube-public          Active    24h
kube-system          Active    24h
sena@k8s-master:~$
```

Varsayılan Namespace'ler:

- **default:**
Kullanıcılar tarafından oluşturulan kaynaklar, başka bir namespace belirtilmediği sürece bu namespace içinde yer alır. Yani kubectl komutlarıyla bir kaynak oluşturduğunuzda namespace belirtmezseniz, kaynaklar otomatik olarak default namespace'e yerleştirilir.
- **kube-system:**
Kubernetes'in kendi bileşenlerinin (kube-dns, kube-proxy, vs.) çalıştığı alandır. Sistem tarafından yönetilen ve genellikle cluster içi teknik servislerin bulunduğu namespace'tir.
- **kube-public:**
Tüm kullanıcıların (kimliği doğrulanmamış olanlar dahil) erişimine açık global bir namespace'tir. Örneğin bazı genel bilgilendirme veya erişim verileri bu alanda bulunabilir.
- **kube-node-lease:**
Kubernetes 1.13 sürümünden itibaren gelen bu namespace, node'ların "lease" (kira) bilgilerini tutar. Bu sayede node sağlığı daha hızlı ve verimli bir şekilde izlenebilir.

kubectl create namespace app1 komutu ile **app1** adında yeni bir namespace oluşturduk.

Komutu çalıştırdıktan sonra **kubectl get namespaces** komutu ile mevcut namespace'leri listelediğimizde, app1 namespace'inin aktif olarak eklendiğini görebiliyoruz.


```

sena@k8s-master:~$ kubectl create namespace app1
namespace/app1 created
sena@k8s-master:~$ kubectl get namespaces
NAME                STATUS    AGE
app1                 Active    8s
default              Active    24h
kube-node-lease      Active    24h
kube-public          Active    24h
kube-system          Active    24h
sena@k8s-master:~$

```

kubectl describe namespace <namespace-ismi>

Belirtilen namespace hakkında detaylı bilgi verir.

kubectl delete namespace <namespace-ismi>

Belirtilen namespace'i tamamen siler.

kubectl get all -n <namespace-ismi>

Belirtilen namespace içindeki tüm objeleri listeler (pod, service, deployment, vs).

kubectl config set-context --current --namespace=<namespace-ismi>

Varsayılan olarak çalışılan namespace'i değiştirir.

kubectl get pods --all-namespaces

Tüm namespace'lerdeki pod'ları listelemek için kullanılır.

kubectl label namespace <namespace-ismi> <etiket-adi>=<deger>

12. Deployment

Kubernetes mimarisinde çalıştırılabilir en küçük birim **Pod**'dur. Ancak doğrudan pod'ları yönetmek pratik değildir; bu yüzden genellikle üst seviye bir kontrol katmanı olan **Deployment** objesiyle pod'lar yönetilir.

Deployment sayesinde:

- Pod'lar istenilen sayıda replika ile çoğaltılabilir,
- Güncellemeler kesintisiz bir şekilde yapılabilir (rolling update),
- Kapanan pod'lar otomatik olarak yeniden oluşturulur (self-healing).

Şimdi bu yönetimsel katmanı daha yakından inceleyeceğiz.

12.1 Singleton pod

Singleton Pod'lar yalnızca bir node üzerinde çalışır. O node çökerse pod da kaybedilir. Bu yüzden üretim ortamlarında genellikle **Deployment** veya **ReplicaSet** ile birden fazla pod tanımı yapılarak yüksek erişilebilirlik sağlanır.

Örneğin, elimizde **üç adet Node** bulunan bir Kubernetes cluster'ı var. **firstpod** adında tek bir Pod tanımlanmış ve bu pod'un nasıl çalıştığını gözlemliyoruz.

1. **Pod Tanımlanıyor**
Kullanıcı, **firstpod** isimli pod'u bir YAML dosyasıyla tanımlıyor. Bu pod **nginx:latest** imajını kullanacak şekilde oluşturulmuş ve 80 numaralı portu expose edecek şekilde yapılandırılmış.
2. **Scheduler Devreye Giriyor**
Pod tanımı API Server'a ulaştıktan sonra Kubernetes scheduler devreye giriyor. Cluster'daki node'ları tarayarak uygun olanı seçiyor. Bu örnekte scheduler, **node3**'ü seçmiş.
3. **Image Pull ve Running Durumu**
Seçilen node (**node3**), Docker Hub üzerinden **nginx:latest** imajını indiriyor ve pod'u başlatıyor. Pod **Running** aşamasına geçtiğinde artık hizmet vermeye hazır hale geliyor.
4. **Node Arızası Durumu**
Ancak **node3** arızalanıyor veya erişilemez hale geliyor. Bu durumda pod'un çalıştığı node yok olduğu için Kubernetes, pod'un durumunu Terminating olarak işaretliyor.
5. **RestartPolicy Devreye Girer mi?**
Eğer **RestartPolicy: Always** tanımlıysa Kubernetes bu pod'u başka bir uygun node'da yeniden başlatmaya çalışır. Ama **Never** tanımlıysa bu pod başka bir node'a taşınmaz. Yani tekil olarak tanımlanan bir pod'un fault tolerance yeteneği yoktur.

Başlangıçta sistemimizde üç adet worker node vardı ve her birine **nodeSelector** aracılığıyla aynı uygulamadan birer tane Pod dağıttık. Bu pod'ların önüne bir LoadBalancer koyduk ve böylece herhangi bir node'daki pod çalışmaz hale gelse bile diğer iki node'daki pod'lar üzerinden sistemimiz çalışmaya devam edebildi.

Ancak zamanla işler karmaşık hale geldi:

Yeni bir uygulama versiyonu yayınladım. Bu versiyonu kullanmak için her node'daki pod'u **ayrı ayrı** güncellemem gerekti. Her pod'un YAML dosyasındaki image alanını tek tek değiştirmek zorunda kaldım. Bu süreç hem zaman aldı hem de hataya açık hale geldi.

Yeni versiyon bazı sorunlara yol açtı. Bu defa rollout yapmak değil rollback yapmak istedim ama tek tek düzenlenmiş pod YAML dosyaları nedeniyle bu da oldukça zor oldu.

Ayrıca uygulamanın farklı parçalarına özel **label'lar** eklemem gerekti. Ancak her pod'u manuel yönettiğim için bu label'ları da tek tek her pod tanımına yazmak durumunda kaldım. Bu da yönetimi daha da zorlaştırdı.

Aynı uygulamayı birden fazla node'a tekrar tekrar deploy ederken YAML dosyalarının içeriğini kopyala-yapıştır yöntemiyle çoğalttım. Hem tekrar eden işler oluştu hem de hatalı konfigürasyon riski arttı. Güncellemeler ve yönetim oldukça zahmetli hale geldi.

Oysa ki Deployment Kullansaydım...

Eğer bu yapıyı bir Deployment objesi ile yönetseydim;

- Tüm pod'lar tek bir tanım üzerinden yönetilebilirdi.
- Sadece replicas: 3 diyerek 3 node'da pod ayağa kalkardı.
- Yeni versiyon çıktığında tek bir image alanını güncelleyerek kubectl apply ile rollout yapabilirdim.
- Gerekirse kubectl rollout undo ile kolayca rollback işlemi gerçekleştirebilirdim.
- Label güncellemeleri tek tanım üzerinden yapılır, her pod'a tek tek gitmem gerekmezdi.
- kubectl scale ve güncelleme işlemleri daha hızlı, daha temiz ve güvenli olurdu.

Bir Deployment objesi, Kubernetes'te istenen pod durumunu tanımlamak için kullanılır. **Deployment Controller**, sistemdeki mevcut durumu bu tanımlanan hedef durumla karşılaştırır ve fark varsa gerekli aksiyonları otomatik olarak alır.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
  labels:
    team: development
    app: front-end
spec:
  replicas: 3
  selector:
    matchLabels:
      app: front-end
  template:
    metadata:
      labels:
        app: front-end
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Örneğin; belirtilen replicas: 3 değeriyle 3 adet pod çalışması gerektiği tanımlanmışsa ve sistemde sadece 2 pod çalışıyorsa, eksik olan pod otomatik olarak yeniden başlatılır. Yani Deployment, sürekli olarak kümedeki durumu izler ve hedefe ulaşmak için düzeltici işlemler yapar.

12.2 Deployment Uygulama

kubectl create deployment firstdeployment --image=nginx:latest --replicas=2

Bu yapı sayesinde elimizde 2 replica halinde çalışan bir nginx uygulaması oldu.

```
sena@k8s-master:~$ kubectl create deployment firstdeployment --image=nginx:latest --replicas=2
deployment.apps/firstdeployment created
sena@k8s-master:~$ kubectl get deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
firstdeployment	2/2	2	2	8s

```
sena@k8s-master:~$
```

```
sena@k8s-master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
firstdeployment-5b665bb68f-9knf7	1/1	Running	0	4m5s
firstdeployment-5b665bb68f-phshg	1/1	Running	0	4m5s
initcontainerpod	1/1	Running	20 (56m ago)	20h

```
sena@k8s-master:~$
```

kubectl delete pods firstdeployment-5b665bb68f-9knf7 komutu ile pod'lardan birini sildik

Ancak hemen ardından, Deployment kontrol mekanizması devreye girdi ve silinen pod'un yerine **otomatik olarak** yeni bir pod oluşturuldu.

- Yeni oluşturulan pod'un ismi farklı bir hash ile gelir.

```
sena@k8s-master: ~
Every 2.0s: kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
firstdeployment-5b665bb68f-cfcdq	1/1	Running	0	16s
firstdeployment-5b665bb68f-phshg	1/1	Running	0	9m1s
initcontainerpod	1/1	Running	21 (96s ago)	21h

```
sena@k8s-master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
firstdeployment-5b665bb68f-9knf7	1/1	Running	0	8m19s
firstdeployment-5b665bb68f-phshg	1/1	Running	0	8m19s
initcontainerpod	1/1	Running	21 (54s ago)	21h

```
sena@k8s-master:~$ kubectl get pods firstdeployment-5b665bb68f-9knf7
```

NAME	READY	STATUS	RESTARTS	AGE
firstdeployment-5b665bb68f-9knf7	1/1	Running	0	8m31s

```
sena@k8s-master:~$ kubectl delete pods firstdeployment-5b665bb68f-9knf7
pod "firstdeployment-5b665bb68f-9knf7" deleted
sena@k8s-master:~$
```

kubectl set image deployment/firstdeployment nginx=httpd

- Deployment içindeki **containers** bölümünde nginx adını verdiğimiz container vardı.
- Bu container ilk başta **nginx:latest** imajıyla çalışıyordu.

- Şimdi bu komutla, **aynı container adı korunarak**, kullandığı imajı **httpd** (Apache HTTP Server) olarak değiştirebiliyoruz.

kubectl scale deployment firstdeployment --replicas=5 komutla birlikte, firstdeployment isimli deployment'ın replica sayısı 5'e çıkarılıyor.

```

sena@k8s-master: ~
Every 2.0s: kubectl get pods                                k8s-mas
NAME                                READY   STATUS    RESTARTS   AGE
firstdeployment-5b665bb68f-cfcdq    1/1     Running   0           10m
firstdeployment-5b665bb68f-d9tnl    1/1     Running   0           3m42s
firstdeployment-5b665bb68f-hhqkh    1/1     Running   0           13s
firstdeployment-5b665bb68f-phshg    1/1     Running   0           19m
firstdeployment-5b665bb68f-xcmqt    1/1     Running   0           3m40s
initcontainerpod                    1/1     Running   21 (11m ago) 21h

sena@k8s-master: ~
sena@k8s-master:~$ kubectl scale deployment firstdeployment --replicas=5
deployment.apps/firstdeployment scaled

```

kubectl delete deployments firstdeployment komutu ile de oluşturulan deployment silinebilir.

(GitHub repository'si: [GitHub - aytitech/k8sfundamentals](https://github.com/aytitech/k8sfundamentals): Ayti.tech tarafından sunulan "Kubernetes Temelleri" eğitiminde gerekli dosyaların barındırılması için oluşturulan GitHub repository'si.)

Deployment.yaml dosyasını oluşturmak için öncelikle elimizde bir **podexample.yaml** dosyası var. Bu dosyada pod oluşturmak için gerekli temel alanlar tanımlanmış: **metadata** kısmında pod adı ve label bilgisi, **spec** altında ise container adı, kullanılacak imaj ve port bilgileri yer alıyor. Bu bilgiler pod'u manuel olarak çalıştırmak için yeterli. Ancak biz bu yapıyı Deployment seviyesinde tanımlamak istiyoruz.

```

sena@k8s-master: ~/k8sfundamentals/deployment
GNU nano 7.2                                podexample.yaml
apiVersion: v1
kind: Pod
metadata:
  name: examplepod
  labels:
    app: frontend
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80

```

deploymenttemplate.yaml dosyamızdaki template alanına geçiyoruz. Template altına ise elimizdeki podexample.yaml dosyasının metadata'dan sonraki kısmını yapıştırıyoruz. Ancak pod **name** artık gereksiz olduğu için bu kısmı siliyoruz. Pod'ların isimlendirmesini artık Deployment kendisi yapacağı için bunu manuel belirtmemize gerek kalmıyor.

Bu şekilde template altına container bilgilerini yerleştiriyoruz. image, port gibi değerleri aynen alıp taşıyoruz. Tab'lamalara dikkat ederek bu alanları doğru bir şekilde hizalıyoruz.

```

sena@k8s-master: ~/k8sfundamentals/deployment
GNU nano 7.2 deploymenttemplate.yaml *
apiVersion: apps/v1
kind: Deployment
metadata:
  name: firstdeployment
  labels:
    team: development
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80

```

Sonuç olarak elimizde 3 replica ile çalışan, **nginx** imajına sahip, app: frontend etiketli pod'ları yöneten bir deployment tanımı oluyor. Artık tek tek pod oluşturmakla uğraşmak yerine, tek bir deployment üzerinden hem yönetebilir hem de gerektiğinde güncelleyebilir hale geliyoruz.

kubectl apply -f deploymenttemplate.yaml komutu ile apply ediyorum.

13. ReplicaSet

ReplicaSet, belirli bir anda sistemde kaç tane pod çalışması gerektiğini kontrol eder ve bu sayıyı garanti altına alır.

Yani biz burada 3 pod tanımladığımızda, Kubernetes bu podlardan biri silinse bile otomatik olarak yeniden oluşturur.

Daha önce **deploymenttemplate.yaml** dosyası ile oluşturduğumuz deployment'ın başarıyla çalışıp çalışmadığını kontrol etmek için **kubectl get deployments** komutunu kullanıyorum. Çıktıya baktığımızda firstdeployment adında bir deployment'ın 3 pod'u hazır (READY) olduğunu, hepsinin güncel (UP-TO-DATE) ve erişilebilir (AVAILABLE) durumda olduğunu görüyoruz.

```

sena@k8s-master:~/k8sfundamentals/deployment$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
firstdeployment     3/3     3            3           53s
sena@k8s-master:~/k8sfundamentals/deployment$

```

Ayrıca **kubectl get replicaset** komutunu çalıştırarak, ilgili deployment'ın arka planda oluşturduğu ReplicaSet kaydını da görüntülüyorum.

Görüldüğü gibi firstdeployment-54758c4c55 isimli ReplicaSet, 3 pod istedi (DESIRED), 3 pod oluşturdu (CURRENT) ve bu pod'ların 3'ü de çalışır durumda (READY).

```
sena@k8s-master:~/k8sfundamentals/deployment$ kubectl get replicaset
NAME                                DESIRED  CURRENT  READY  AGE
firstdeployment-54758c4c55          3         3        3      2m36s
sena@k8s-master:~/k8sfundamentals/deployment$
```

Diğer komutlar:

kubectl rollout status deployment <deployment-adi>

Bir deployment'ın rollout (dağıtım) durumunu gösterir. (Uygulamanın güncellenme süreci devam ediyorsa bu komutla canlı olarak durumunu takip edebiliyorum.)

kubectl rollout history deployment <deployment-adi>

Deployment'ın geçmiş revizyonlarını listeler. Bu komutla hangi image'lar veya yapılandırmalarla kaç defa değişiklik yapıldığını takip ediyorum.

kubectl rollout undo deployment <deployment-adi>

Deployment'ı bir önceki başarılı versiyona döndürür. (Yeni sürümde hata çıkarsa bu komutla uygulamayı önceki stabil versiyona hızlıca geri alabiliyorum.)

kubectl edit deployment <deployment-adi>

Canlı olarak YAML dosyasını editler. Örneğin image'ı değiştirebilirsin.

kubectl delete deployment <deployment-adi>

Deployment'ı ve ona bağlı ReplicaSet ile pod'ları siler.

kubectl get rs veya **kubectl get replicaset**

ReplicaSet'leri listeler.

Deployment ve ReplicaSet Kullanmanın Farkları

ReplicaSet sadece pod'ları **sabit sayıda tutar**, fakat:

- **Güncelleme (rolling update)** yapamaz.
- **Versiyon geçmişini** tutmaz.
- Yeni bir image deploy etmek istersen eski pod'ları silip yenilerini manuel oluşturman gerekir.
- **Rollback** gibi bir mekanizması yoktur.

Yani yönetimsel işlemleri tamamen manuel yapman gerekir. Bu da hataya açık, zahmetli ve risklidir.

ReplicaSet **daha düşük seviyeli** bir nesnedir. Deployment ise onun üzerine inşa edilmiş, **uygulamayı gerçek hayatta idare edilebilir hale getiren üst seviye yapıdır**.

Örnekle açıklamak gerekirse,

Diyelim ki bir **nginx** uygulamasını yayınladın. Sonra **nginx:1.25** versiyonuna geçmek istiyorsun.

- **ReplicaSet kullansaydın**, eski pod'ları silip yenilerini sen oluşturacaktın.
- **Deployment kullandığında**, sadece image'ı değiştiriyorsun, kalan her şeyi Kubernetes otomatik hallediyor: eskiyi kaldırıyor, yeniye getiriyor, hata olursa geri dönüyor.

14. Rollout ve Rollback

deployrecreate.yaml dosyasını incelediğimizde, **spec** altında **strategy** adında yeni bir alan olduğunu ve burada **type** olarak **Recreate** seçildiğini görüyoruz. Yani bu YAML dosyası, Deployment güncellendiğinde **rollout** işlemlerinin nasıl yönetileceğini belirliyor.

```

sena@k8s-master: ~/k8sfundamentals/deployment
GNU nano 7.2                                deployrecreate.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rcdeployment
  labels:
    team: development
spec:
  replicas: 3
  selector:
    matchLabels:
      app: recreate
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: recreate
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80

```

Kubernetes'te **rollout strategy** olarak kullanabileceğimiz iki tip seçenek mevcut: **RollingUpdate** ve **Recreate**. Bu örnekte ise Recreate strategy'si tercih edilmiş. Recreate şu anlama gelir: Eğer bu Deployment üzerinde bir değişiklik yapılırsa, önce mevcut tüm pod'lar silinir ve bu işlem tamamlandıktan sonra yeni pod'lar oluşturulur.

Recreate strategy'si genellikle **hardcore migration** gibi eski versiyonla yeni versiyonun aynı anda çalışmasının istenmediği, hatta sakıncalı olduğu senaryolarda tercih edilir. Örneğin bir uygulamada **major** güncelleme yapılıyorsa, eski versiyon tamamen sistemden kaldırıldıktan sonra yeni versiyonun başlatılması daha güvenli olur.

14.1 Recreate Ugulama

Öncelikle minikube cluster'ımızı **minikube delete** komutu ile delete edelim ve **minikube start** ile sıfırdan yeni bir kubernetes cluster kuralım.

İki tab açtım. Birinde **watch kubectl get pods** komutunu diğesinde **watch kubectl get rs** çalıştırıyorum.

Birinde pod'ları diğesinde replicaset'leri sıraladım.

kubectl apply -f deployrecreate.yaml komutu ile deployment objemizi oluşturalım.

```

sena@k8s-master: ~
Every 2.0s: kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
rcdeployment-687df9bcb7-2vrjx        0/1     ContainerCreating   0           21s
rcdeployment-687df9bcb7-9w652        0/1     ContainerCreating   0           21s
rcdeployment-687df9bcb7-gr8d6        0/1     ContainerCreating   0           21s

sena@k8s-master: ~
Every 2.0s: kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
rcdeployment-687df9bcb7              3          3         0       22s

```

- Bir adet ReplicaSet oluşturdu.
- Bu ReplicaSet üzerinden 3 adet pod ayağa kaldırıldı.

Sonrasında bu Deployment üzerinde güncelleme yapmak için kubectl set image komutunu kullanıyoruz. Burada uygulamanın imajını nginx'ten httpd'ye güncelliyoruz:

kubectl set image deployment rcdeployment nginx=httpd

```

sena@k8s-master: ~
Every 2.0s: kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
rcdeployment-588d5d854c-bm99p        0/1     ContainerCreating   0           12s
rcdeployment-588d5d854c-cgjr7        0/1     ContainerCreating   0           12s
rcdeployment-588d5d854c-x4t24        0/1     ContainerCreating   0           12s

sena@k8s-master: ~
Every 2.0s: kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
rcdeployment-588d5d854c              3          3         0       12s
rcdeployment-687df9bcb7              0          0         0       17m

```

Bu işlem gerçekleştiğinde **Recreate** strategy devreye giriyor. Yeni ReplicaSet oluşturulmadan önce:

- Mevcut ReplicaSet'in **desired** değeri 0'a çekiliyor.
- Var olan tüm pod'lar terminate ediliyor.
- Yeni ReplicaSet oluşturuluyor ve bu set üzerinden yeni pod'lar ayağa kaldırılıyor.

Sonuç olarak, eski pod'lar tamamen silindikten sonra yeni versiyonun pod'ları oluşturuluyor. **Recreate** stratejisinin mantığı da tam olarak budur: eski sürüm tamamen yok edilmeden yeni sürüm çalıştırılmaz.

14.2 RollingUpdate

Kullanabileceğimiz ikinci tip strategy ise RollingUpdate'dir. YAML dosyamızda **strategy** alanında herhangi bir değer belirtmezsek Kubernetes varsayılan olarak **RollingUpdate** stratejisini kullanır. Bu strateji, **Recreate'in** tam tersidir. Recreate'te yeni pod'lar oluşturulmadan önce mevcut olanlar silinirken, RollingUpdate'te bu işlem aşamalı olarak gerçekleştirilir. Yani eski pod'lar silinirken aynı anda yeni pod'lar devreye alınır. Böylece kesinti yaşanmaz.

Bu güncellemenin ne kadar kontrollü ilerleyeceğini iki parametre belirler:

- **maxUnavailable**: Güncelleme sırasında aynı anda en fazla kaç pod'un geçici olarak devre dışı kalabileceğini belirler.
- **maxSurge**: Aynı anda kaç yeni pod'un mevcut pod sayısına ek olarak devreye alınabileceğini tanımlar.

```

sena@k8s-master: ~/k8sfundamentals/deployment
GNU nano 7.2 deployrolling.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rolldeployment
  labels:
    team: development
spec:
  replicas: 10
  selector:
    matchLabels:
      app: rolling
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 2
      maxSurge: 2
  template:
    metadata:
      labels:
        app: rolling
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80

```

Örneğin görselimizdeki **deployrolling.yaml** dosyasındaki Deployment nesnesinde:

- **replicas**: 10 olarak belirlenmiş, yani sistemde aynı anda 10 pod çalışmalı.
- **maxUnavailable**: 2 sayesinde güncelleme başladığında aynı anda en fazla 2 pod devre dışı bırakılır.
- **maxSurge**: 2 ise aynı anda en fazla 2 yeni pod'un devreye girebileceğini ifade eder.

Bu durumda güncelleme aşamalı bir şekilde ilerler:

İlk olarak 2 pod devre dışı kalır ve 2 yeni pod oluşturulur. Yeni pod'lar hazır hale geldiğinde diğer 2 eski pod devre dışı kalır ve bu döngü tüm pod'lar için tamamlanır.

Sonuç olarak sistemde herhangi bir servis kesintisi yaşanmadan yeni versiyona geçiş sağlanmış olur.

14.3 RollingUpdate Uygulama

kubectl apply -f deployrolling.yaml komutunu çalıştırarak tanımlı Deployment'ı uyguluyorum ve toplam 10 pod başarılı şekilde oluşturuluyor.

kubectl get pods komutu ile pod'ların **Running** durumda olduğunu,

kubectl get rs komutu ile de tüm bu pod'ların tek bir **ReplicaSet** altında toplandığını teyit ediyorum.

```

sena@k8s-master: ~
Every 2.0s: kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
rolldeployment-6b947d555-2gt6c      1/1     Running   0           2m44s
rolldeployment-6b947d555-4jf6h      1/1     Running   0           2m44s
rolldeployment-6b947d555-5kkjh      1/1     Running   0           2m44s
rolldeployment-6b947d555-6wgs4      1/1     Running   0           2m44s
rolldeployment-6b947d555-cn59z      1/1     Running   0           2m44s
rolldeployment-6b947d555-dt2jn      1/1     Running   0           2m44s
rolldeployment-6b947d555-kbbrn      1/1     Running   0           2m44s
rolldeployment-6b947d555-pr9vh      1/1     Running   0           2m44s
rolldeployment-6b947d555-wfjxb      1/1     Running   0           2m44s
rolldeployment-6b947d555-wtvcv      1/1     Running   0           2m44s

sena@k8s-master: ~
Every 2.0s: kubectl get rs

NAME                                DESIRED   CURRENT   READY   AGE
rolldeployment-6b947d555            10         10        10       2m45s

```

Şimdi bu Deployment objesi üzerinde bir güncelleme yapalım.

kubectl edit deployment rolldeployment --save-config komutunu kullanarak YAML dosyasını açıyorum.

İlgili alanda klavyeden **i** tuşuna basarak insert mod'a geçiyorum ve nginx olan imaj ismini "httpd" olarak değiştiriyorum.

Dosyayı kaydedip çıktığımda, Deployment objesi otomatik olarak güncellenmiş oluyor.

```

sena@k8s-master: ~/k8sfundamentals/deployment
progressDeadlineSeconds: 600
replicas: 10
revisionHistoryLimit: 10
selector:
  matchLabels:
    app: rolling
strategy:
  rollingUpdate:
    maxSurge: 2
    maxUnavailable: 2
    type: RollingUpdate
template:
  metadata:
    creationTimestamp: null
    labels:
      app: rolling
  spec:
    containers:
      - image: nginx_
        imagePullPolicy: Always
        name: nginx
        ports:
          - containerPort: 80
            protocol: TCP
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
    dnsPolicy: ClusterFirst
    restartPolicy: Always
    schedulerName: default-scheduler
    securityContext: {}
    terminationGracePeriodSeconds: 30
status:
  availableReplicas: 10
  conditions:
    - lastTransitionTime: "2025-06-20T08:17:10Z"
      lastUpdateTime: "2025-06-20T08:17:10Z"
      message: Deployment has minimum availability.
      reason: MinimumReplicasAvailable
-- INSERT --

```

klavyede i'ye basarak insert mod'a geçiş yapılır

ardından nginx olan kısım httpd ile değiştirilir

- Hemen yeni bir replicaset objesi oluşturuldu ve yeni pod'larda yavaş yavaş oluşturuluyor.
- Burada dikkat çeken nokta; **desired** sayısı hiçbir zaman 12'yi geçmiyor, yani sistemde aynı anda çalışan pod sayısı tanımladığımız sınırları aşmıyor. Ayrıca başlangıçta minimum 8 pod her zaman aktif kalıyor. Bu da sistemin kesintisiz devam etmesini sağlıyor.
- İşte **rollingUpdate strategy** bize zero downtime geçiş imkanı verir.

```

C:\ sena@k8s-master: ~
Every 2.0s: kubectl get pods

NAME                                READY   STATUS              RESTARTS   AGE
rolldeployment-55574b86c-2dm2f      1/1     Running             0           19s
rolldeployment-55574b86c-9x6ns      1/1     Running             0           19s
rolldeployment-55574b86c-fg5zh      1/1     Running             0           19s
rolldeployment-55574b86c-jwt5l      0/1     ContainerCreating   0           7s
rolldeployment-55574b86c-lpn2f      0/1     ContainerCreating   0           11s
rolldeployment-55574b86c-nt6lp      0/1     ContainerCreating   0           3s
rolldeployment-55574b86c-pnj6t      1/1     Running             0           19s
rolldeployment-55574b86c-qpw5m      0/1     ContainerCreating   0           0s
rolldeployment-55574b86c-vb57g      1/1     Running             0           14s
rolldeployment-6b947d555-6wgs4      1/1     Running             0           20m
rolldeployment-6b947d555-kbbrn      1/1     Terminating        0           20m
rolldeployment-6b947d555-pr9vh      1/1     Running             0           20m
rolldeployment-6b947d555-wfjxb      1/1     Running             0           20m

C:\ sena@k8s-master: ~
Every 2.0s: kubectl get rs

NAME                                DESIRED   CURRENT   READY   AGE
rolldeployment-55574b86c            9         9         5       20s
rolldeployment-6b947d555            3         3         3       20m

```

Diğer komutlar:

kubectl rollout history deployment <name> → Deployment geçmişini gösterir

kubectl set image deployment/<name> <container-name>=<yeni-image> → Container imajını günceller (canary/release senaryoları için)

kubectl rollout status deployment <name> → Deployment güncelleme sürecini takip eder

kubectl rollout undo deployment <name> → Son yapılan güncellemeyi geri alır

kubectl scale deployment <name> --replicas=N → Pod sayısını günceller

15. Kubernetes Ağ Altyapısı

Şimdiye kadar Kubernetes'in temel objeleri olan **Pod**, **ReplicaSet** ve **Deployment** kavramlarını ele aldık. Bu objelerin nasıl tanımlandığını ve temel yapılandırmalarını öğrendik. Özünde yaptığımız şey, pod'lar oluşturmak ve bu pod'ları yönetmekti.

Ancak, bu pod'lar kendi aralarında ya da dış dünyayla **iletişim kuramazsa**, yaptığımız konfigürasyonların pek de bir anlamı kalmaz. Sonuçta amacımız, uygulamalarımızı Kubernetes üzerinde yayınlamak ve bu uygulamalara **kullanıcıların erişmesini sağlamak**. Aynı şekilde uygulamaların da birbirleriyle ya da dış kaynaklarla haberleşebilmesi gerekir.

Bu noktada Kubernetes'in **ağ yapısının nasıl çalıştığını** bilmek önemli. Şimdi Kubernetes'in ağ konusuna nasıl yaklaştığını inceleyelim.

15.1 Kubernetes Ağ Kuralları

- Kubernetes kurulumunda pod'lara IP dağıtılması için bir IP adres aralığı ya da Kubernetes terminolojisinde bilinen adıyla **--pod-network-cidr** belirlenir.
- Kubernetes'te her pod bu CIDR bloğundan atanacak bir eşsiz IP adresine sahip olur.
- Aynı cluster içerisindeki tüm podlar varsayılan olarak birbirleriyle herhangi bir kısıtlama olmadan ve NAT yani network address translation olmadan haberleşebilirler.

15.2 Kubernetes Ağ İşleyişi

Evimizde dört fiziksel makine olduğunu varsayalım ve bu makinelerle bir Kubernetes cluster kurmak istiyoruz. İnternet bağlantımız bir router üzerinden sağlanıyor ve router'a bağlı dört makineyi bir switch'e bağladık.

Bu makinelerin hepsine 192.168.1.x IP bloğundan adresler atandı. Default gateway olarak router'ı gösteriyoruz. Yani makineler hem kendi aralarında hem de dış dünyayla iletişim kurabiliyor.

Tüm makinelerde Linux işletim sistemi kurulu. Master node olarak belirlediğimiz makineden Kubernetes kurulumuna başlayacağız. Diğer makineleri ise worker node olarak cluster'a dahil edeceğiz.

Kubernetes kurulumunda **--pod-network-cidr** parametresiyle pod'lara atanacak IP adres aralığını belirlememiz gerekiyor. Bu örnekte 10.10.0.0/16 IP bloğunu kullandık.

Kurulum tamamlandığında worker node'larda aşağıdaki işlemler gerçekleşir:

- Her bir worker node üzerinde bir adet **virtual bridge** oluşturulur. Bu bridge, belirlenen **pod-network-cidr** üzerinden pod'lar arası iletişimi sağlar.
- Yeni bir pod oluşturulduğunda, bu pod için Linux kernel içinde ayrı bir **network namespace** tanımlanır. Pod'a ait sanal **eth0** interface'e, belirlenen IP bloğundan bir adres atanır.

- Atanan bu IP, pod'un bulunduğu worker node üzerindeki sanal bridge'e bağlanarak iletişim kurulmasını sağlar.



Görselde gördüğünüz üzere cluster'ımızda toplam 6 adet pod var ve her biri **pod-cidr** bloğundan IP adresini almış durumda. Bu bölümde ilk olarak bir pod'un dış dünyadaki bir IP adresine erişmeye çalıştığında network trafiğinin nasıl aktığını adım adım inceleyeceğiz.

15.3 Pod'dan Dış Dünyaya Erişim – NAT ile Çıkış (Senaryo 1)

Pod-a isimli bir pod'un **1.1.1.1** IP adresine sahip dış dünyadaki bir sunucuya erişmek istediğini varsayalım.

1. Pod-a'dan çıkan paket, önce bulunduğu node'un üzerindeki **bridge** arayüzüne teslim edilir.
2. Bridge, bu paketi **worker node**'un fiziksel **eth0** arayüzüne iletir.
3. Paket burada **NAT** işlemine tabi tutulur. Pod'un IP'si yerine worker node'un IP'si ile çıkış yapılır.
4. NAT'lanmış paket, **default gateway** olarak tanımlı router'a iletilir.
5. Router paketi alır ve hedef IP olan **1.1.1.1** adresine yönlendirir.
6. Sunucudan gelen yanıt paketi tekrar router üzerinden geri döner, NAT tablosu sayesinde hangi pod'a ait olduğu çözülür.
7. Paket, NAT işlemi tersine uygulanarak tekrar pod-a'ya ulaştırılır.

Bu mekanizma sayesinde dış dünyaya doğru olan tüm çıkışlar worker node üzerinden gerçekleşir ve geri gelen paketlerin doğru pod'a yönlendirilmesi NAT tablosu sayesinde sağlanır.

15.4 Pod'lar Arası İletişim – Aynı Node Üzerinde (Senaryo 2)

Bu senaryoda yine **pod-a**'yı ele alıyoruz. Ancak bu sefer pod-a'nın aynı **worker node** üzerinde çalışan **pod-b** ile iletişim kurmak istediğini varsayalım.

Her iki pod da:

- Aynı **bridge** arayüzüne bağlıdır.
- Aynı **pod CIDR** bloğundan atanmış IP adreslerine sahiptir.

Bu durumda iletişim şu şekilde gerçekleşir:

1. Pod-a, pod-b'ye göndermek istediği ağı paketi **bridge** arayüzüne iletir.
2. Bridge, paketi doğrudan aynı node üzerinde çalışan pod-b'ye yönlendirir.
3. Pod-b yanıt paketini aynı şekilde bridge'e gönderir, yanıt yine aynı yoldan pod-a'ya ulaşır.

İki pod aynı worker node üzerinde olduğu ve aynı bridge'i kullandığı için aralarındaki trafik **NAT gerektirmez**. İletişim, aynı fiziksel ağ üzerindeki iki makine gibi doğrudan ve sorunsuz şekilde gerçekleşir.

15.5 Farklı Node'lar Arası Pod İletişimi (Senaryo 3)

Bu senaryoda, bir pod'un başka bir worker node üzerindeki pod ile haberleşmesi gerekir. Örneğin, worker1'deki pod-a, worker2 üzerindeki pod-c ile iletişim kurmak istiyor.

Başlangıçta belirttiğimiz gibi Kubernetes'te tüm pod'ların birbirleriyle NAT gerekmeksizin haberleşebilmesi beklenir. Ancak burada pod-a'dan çıkan paket, pod-c'ye ulaşmak isterken doğrudan bridge'e gider. Fakat bridge, pod-c'nin IP adresini tanımadığı için paket kaybolur. Çünkü bu IP adresi routing tablosuna dahil değildir.

Bu tür durumlarda çözüm olarak Kubernetes, varsayılan olarak bir **CNI (Container Network Interface)** eklentisi kullanır. Bu eklenti sayesinde farklı node'larda bulunan pod'lar arasında sanal bir overlay network kurulur. Böylece farklı node'larda bulunan pod'lar sanki aynı ağdaymış gibi haberleşebilir.

Bu yapıyı sağlamak için genellikle **VXLAN** gibi **overlay** teknolojileri kullanılır. Örneğin, 10.10.0.0/16 ağı overlay network olarak tanımlanır ve bu ağ sayesinde NAT yapılmadan farklı node'lardaki pod'lar birbirine ulaşabilir.

Geçmişte Kubernetes'e doğrudan route tanımlamak gibi çözümler uygulanmış olsa da bu yöntemler karmaşıktı ve sürdürülebilir değildi. Bu nedenle, bu ağ problemini çözmek için CNI standardı geliştirildi.

CNI sayesinde her Kubernetes kurulumu, çalıştığı ortama uygun bir **CNI plugin** seçerek ağ altyapısını kolayca kurabilir ve yönetebilir. Bu altyapı sayesinde farklı node'lardaki pod'lar arasında güvenli ve kesintisiz bir iletişim mümkün olur.

Daha Fazla Bilgi İçin: [GitHub - containernetworking/cni: Container Network Interface - networking for Linux containers](https://github.com/kubernetes/cni)

Kubernetes Resmi Sayfası: [Cluster Networking | Kubernetes](https://kubernetes.io/docs/concepts/cluster-networking/)

Örneğin biz bu ortamda **CALICO CNI plugin**'sini yüklersek, bu noktadan itibaren pod'ların IP adreslerinin atanması, IP table kurallarının düzenlenmesi ve node'lar arasında pod CIDR bloğunun NAT'sız çalışması için gereken overlay tanımlarının yapılarak VXLAN interface'lerinin oluşturulması dahil tüm görevler **CALICO** tarafından hallediliyor.

Kubernetes'te bir pod oluşturduğunuz zaman **scheduler**, bunun çalışacağı node'u seçer ve **kubelet** burada devreye girerek pod'u oluşturur.

Pod için atanacak IP adresinin belirlenmesi, bunun pod'a atanması ve altyapıda ayarlanması gereken tüm işlerin halledilmesi ise **CALICO** tarafından sağlanır. Böylece pod'lar farklı node'larda olsalar bile birbirleriyle NAT olmadan haberleşebilirler.

16. Service

Şimdi şöyle bir senaryo düşünelim:

Bir web uygulaması oluşturduk. Bu uygulamaya kullanıcılar bağlanıp resim yükleyebiliyor ve yüklenen resimlere filtre eklenip geriye dönülüyor. Uygulamayı iki ayrı service olarak tasarladık: biri frontend, diğeri backend olacak şekilde. Kullanıcılar frontend üzerinden erişim sağlıyor. Bu katmanı JavaScript ile geliştirdik.

Backend kısmı ise frontend'in gönderdiği resmi alıp filtreleyen yapıyı içeriyor. Bu kısmı .NET ile yazdık.

Her iki bileşeni container image haline getirip Kubernetes üzerinde çalıştırmaya hazır hale getirdik. İki ayrı deployment.yaml dosyası oluşturarak frontend ve backend katmanlarını üçer pod olacak şekilde Kubernetes üzerinde deploy ettik.

Bu senaryoda erişim kısmını inceleyelim:

İlk olarak, frontend pod'larına dış dünyadan erişim sağlanması gerekiyor. Bu en temel problem. Bir şekilde bu erişimi sağladığımızı düşünelim. Kullanıcı frontend pod'lardan birine bağlanıp resmi yüklediğinde, frontend uygulamasının bu resmi backend pod'lardan birine iletmesi gerekiyor. Kubernetes içerisinde her pod benzersiz bir IP adresine sahip ve varsayılan olarak birbirleriyle haberleşebiliyor.

Toplamda 3 frontend ve 3 backend pod oluşturduk. Frontend pod'larının, backend pod'lara erişmesi gerekiyor. Bir kullanıcı frontend'e resim yüklediğinde, ilgili frontend pod bu resmi herhangi bir backend pod'a iletip işlemenin yapılmasını ve geri alınmasını sağlıyor.

Bu iletişimin kurulabilmesi için frontend pod'ların backend pod'lara nasıl ulaşacaklarını bilmesi gerekiyor.

Bunun için backend pod'ların IP adreslerini listeledim:

- **backend-pod-1:** 10.10.10.1
- **backend-pod-2:** 10.10.10.2
- **backend-pod-3:** 10.10.10.3

Sonrasında frontend pod'lara tek tek bağlandım ve bu IP adreslerini yazılımın içine gömerek erişimi sağladım. Bu geçici olarak sorunu çözdü.

Ancak node'lardan biri sorun çıkardı ve devre dışı kaldı. Deployment objesi bunu fark ederek ilgili node'daki backend pod'ları, sağlıklı node'larda yeniden oluşturdu.

Yeni pod'lar oluşturulduğunda ise yeni IP adresleri almış oldular. Bu durumda yeniden frontend pod'lara bağlanıp yazılım içerisindeki IP adreslerini güncellemem gerekti. Bu süreç sıkça tekrarlanabileceği için sürdürülebilir bir yöntem değil.

Ayrıca bir scale yaptım ve backend için 4 pod oluşturdum. Bunların IP'lerini de frontend tarafına manuel eklemem gerekiyordu. Tüm bu işlemler ciddi bir yük oluşturuyor.

İşte bu noktada Kubernetes içerisinde **service** devreye giriyor. Service objeleri bu gibi problemleri çözmek için kullanılıyor.

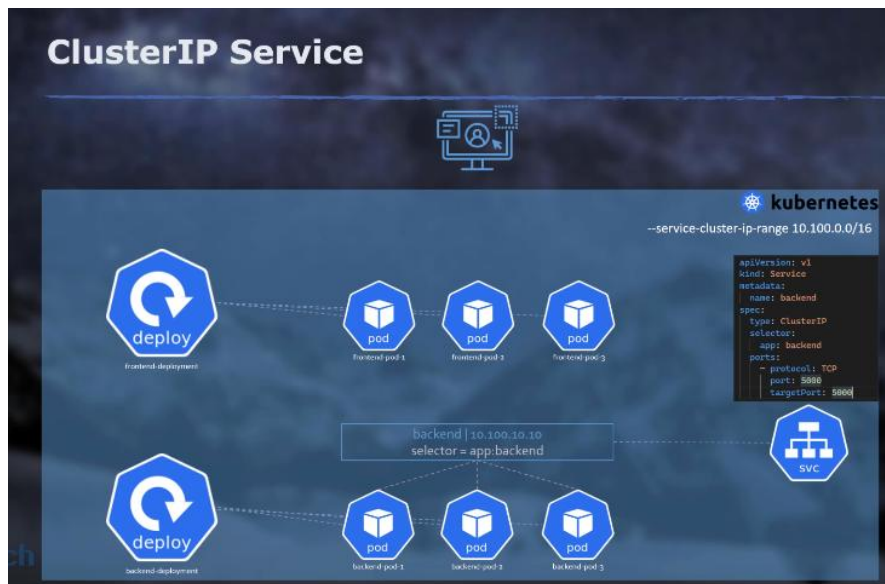
Kubernetes'te dört farklı service tipi tanımlayabiliriz. Bu service objeleri, uygulamaların birbirleriyle ve dış dünya ile sağlıklı bir şekilde haberleşmesini sağlar.

16.1 Cluster IP

Bunlardan ilki **ClusterIP** servisi. Bu servis tipi, varsayılan olarak gelir ve yalnızca cluster içi iletişime izin verir. label selector aracılığıyla backend pod'ları ile ilişkilendirilir. Service objesine bir DNS ismi verilir ve sistem, bu servise ait sanal bir IP adresi atar. Atanan IP ve DNS ismi, cluster'ın DNS mekanizmasında kayıtlıdır.

Bu IP adresi fiziksel bir **interface'e** bağlı değildir, yani doğrudan erişilemez. IP, sanaldır ve yalnızca **kube-proxy** tarafından yönetilir. **kube-proxy**, her node üzerinde bu IP'ye gelen trafiği uygun backend pod'lara yönlendirecek şekilde iptables kuralları oluşturur.

Bu yönlendirme işlemi genellikle **round-robin** algoritması ile yapılır; yani istekler, ilgili pod'lara sırayla dağıtılır. Böylece frontend pod'ları backend pod'lara IP'lerine ihtiyaç duymadan erişebilir.



16.2 NodePort Service

NodePort da tıpkı ClusterIP gibi bir service objesidir. Bu servis objesini oluşturduğunuzda, **label selector** aracılığıyla ilgili pod'larla eşleştirebilirsiniz. NodePort servisi oluşturulduğunda Kubernetes, 30.000–32.767 aralığında bir port seçer ve bu port tüm worker node'larda açılır.

Bu port, ilgili servise **kube-proxy** tarafından yönlendirilir. Yani, worker node'ların IP adreslerine bu port üzerinden gelen tüm trafik, servis ile eşleşmiş pod'lara yönlendirilir. Böylece dış dünyadan gelen bağlantılar, herhangi bir worker node'un IP'si ve belirtilen port üzerinden uygulamanıza ulaşabilir.

Eğer bu node'ların önüne bir load balancer ya da reverse proxy koyarsanız, gelen trafiği node'lara dağıtabilir, oradan da servise bağlı uygun pod'lara ulaştırabilirsiniz. Bu sayede kullanıcılar, uygulamanıza dış dünyadan erişim sağlayabilir.

16.3 LoadBalancer Service

LoadBalancer tipi service objeleri, yalnızca AWS, Google Kubernetes Engine veya Azure Kubernetes Service gibi cloud servis sağlayıcılarının yönettiği Kubernetes ortamlarında kullanılabilir.

Bu tip bir service objesi oluşturulduğunda, cloud servis sağlayıcısı dış dünyadan erişilebilecek public bir IP adresine sahip bir load balancer oluşturur ve bu load balancer'ı sizin service objenizle ilişkilendirir. Böylece, bu public IP adresine gelen tüm trafik, service aracılığıyla ilgili pod'lara yönlendirilir.

NodePort için demiştik ya, “worker node'un önüne bir load balancer ya da reverse proxy koyarsınız” diye. İşte cloud servis sağlayıcıları bu işlemi sizin yerinize otomatik olarak yapar. Bu otomatikleştirilmiş yapı, LoadBalancer tipi service objesiyle sağlanır.

16.4 Service Uygulama

k8sfundamentals/service klasörünün altındaki deploy.yaml dosyasını inceleyelim.

```

sena@k8s-master: ~/k8sfundamentals/service
GNU nano 7.2 deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    team: development
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: ozgurozturknet/k8s:latest
          ports:
            - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    team: development
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend

```

Görüldüğü üzere iki deployment yapıyoruz. Birincisi frontend pod'ları için diğeri backend pod'ları için 3 replica oluşturacağız.

İlk olarak bunu bir deploy edelim.

kubectl apply -f deploy.yaml

```

sena@k8s-master:~/k8sfundamentals/service$ kubectl apply -f deploy.yaml
deployment.apps/frontend created
deployment.apps/backend created
sena@k8s-master:~/k8sfundamentals/service$

```

watch kubectl get pods komutu ile oluşturulmuş mu diye bakalım. Görüldüğü üzere 3 tane frontend için 3 tane backend için toplamda 6 tane pod ayağa kaldırılmış.

```

sena@k8s-master: ~
Every 2.0s: kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
backend-7586b7b49-h7hhq             1/1     Running   0           3m15s
backend-7586b7b49-h86dn             1/1     Running   0           3m15s
backend-7586b7b49-qfdr6             1/1     Running   0           3m15s
frontend-fb99cc5df-2jlj8            1/1     Running   0           3m15s
frontend-fb99cc5df-nr594            1/1     Running   0           3m15s
frontend-fb99cc5df-rbnsq            1/1     Running   0           3m15s

```

serviceClusterIP.yaml dosyamızı inceleyelim. Servislerde diğer tüm kubernetes objeleri gibi yaml dosyaları aracılığıyla oluşturulabilir.

```

GNU nano 7.2 serviceClusterIP.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000

```

- **v1 api'i** üzerinde tipi Service olan bir obje oluşturmak istediğimizi söylüyoruz.
- **metadata** kısmında name anahtarı ile ismini tanımlıyor daha sonra da **spec** kısmında gerekli ayarları tanımlıyoruz. Buradaki isim kısmı önemli çünkü bu isim artık bizim cluster içerisinde bu servise ulaşacağımız isim olacak. Bunun bir clusterIP olduğunu **type** kısmında tanımlıyoruz.
- **selector** kısmında dikkat edin, labeller konusunda bunu anlatmıştık. Label'lar objeler arasında bağlantı kurmakta kullanılır. selector kısmında app:backend key value'sini tanımlayarak bu servise şunu diyoruz: git bu anahtar-değer eşliğine sahip olan podları bul ve trafiği onlara yönlendir. Yani servis hangi podlara hizmet edeceğini selector ile seçer.
- **port**, servisin Cluster içindeki diğer bileşenlerin erişeceği portu belirtir. **targetPort** ise trafiğin yönlendirileceği backend pod içindeki uygulamanın dinlediği porttur.

Şimdi bu .yaml dosyasını **kubectl apply -f serviceClusterIP.yaml** komutu ile deploy edelim.

kubectl get services komutu ile teyit edelim.

```
sena@k8s-master:~/k8sfundamentals/service$ kubectl apply -f serviceClusterIP.yaml
service/backend created
sena@k8s-master:~/k8sfundamentals/service$ kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
backend       ClusterIP     10.103.144.108 <none>       5000/TCP    12s
kubernetes    ClusterIP     10.96.0.1      <none>       443/TCP    10d
sena@k8s-master:~/k8sfundamentals/service$
```

Görüldüğü üzere, backend isimli bir service oluşturulmuş, buna bir virtual IP atanmış; bu IP sadece cluster içinde geçerlidir. Cluster içinde herhangi bir kaynak bu IP adresinin 5000 portuna istek gönderdiği anda bu istek bu service'in selector ile seçtiği pod'lardan bir tanesinin IP adresine aynı port üzerinden yönlendirilir.

Mesela bir pod'a bağlanalım.

kubectl get pods dedim.

```
sena@k8s-master:~/k8sfundamentals/service$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-7586b7b49-h7hhq             1/1     Running   0           6d1h
backend-7586b7b49-h86dn             1/1     Running   0           6d1h
backend-7586b7b49-qfdr6             1/1     Running   0           6d1h
frontend-fb99cc5df-2jlj8            1/1     Running   0           6d1h
frontend-fb99cc5df-nr594            1/1     Running   0           6d1h
frontend-fb99cc5df-rbnsq            1/1     Running   0           6d1h
sena@k8s-master:~/k8sfundamentals/service$
```

Herhangi birine bağlanacağım.

kubectl exec -it frontend-fb99cc5df-2jlj8 -- bash

```
sena@k8s-master: ~/k8sfundamentals/service
sena@k8s-master:~/k8sfundamentals/service$ kubectl exec -it frontend-fb99cc5df-2jlj8 -- bash
bash-5.0# nslookup backend
Server:      10.96.0.10
Address:     10.96.0.10:53

** server can't find backend.cluster.local: NXDOMAIN

** server can't find backend.svc.cluster.local: NXDOMAIN

Name:   backend.default.svc.cluster.local
Address: 10.103.144.108

** server can't find backend.svc.cluster.local: NXDOMAIN

** server can't find backend.cluster.local: NXDOMAIN

bash-5.0#
```

Görüldüğü üzere, burada **DNS çözümlemesi** devreye girmektedir.

Kubernetes içerisinde, CoreDNS tabanlı bir DNS hizmeti yer alır ve tüm **pod**'lar, DNS sorgularını bu hizmete yönlendirir. Bir **Service** oluşturduğunuzda, bu servis için bir DNS kaydı oluşturulur. Bu sayede, diğer pod'lar servis adına karşılık gelen IP adresini DNS üzerinden öğrenebilirler.

Örneğin bu örnekte **backend** adında bir servis oluşturulmuş ve aynı namespace içinde çalıştığı için sadece backend ismiyle erişim sağlanabilmektedir.

Eğer servis başka bir namespace içinde olsaydı, tam adıyla yani **backend.default.svc.cluster.local** şeklinde yazılması gerekirdi.

```

sena@k8s-master: ~/k8sfundamentals/service
bash-5.0# curl backend:5000
Pod: backend-7586b7b49-qfdr6 ---Backend Service is up and running---
bash-5.0# curl backend:5000
Pod: backend-7586b7b49-h7hhq ---Backend Service is up and running---
bash-5.0# curl backend:5000
Pod: backend-7586b7b49-qfdr6 ---Backend Service is up and running---
bash-5.0# curl backend:5000
Pod: backend-7586b7b49-h7hhq ---Backend Service is up and running---
bash-5.0# curl backend:5000
Pod: backend-7586b7b49-qfdr6 ---Backend Service is up and running---
bash-5.0#

```

curl komutuyla backend servisine ait 5000 portuna istek göndermek istiyorum. Gelen trafik, servis tanımında belirtilen selector aracılığıyla eşleşen pod'lardan birine yönlendirilir.

Komutu birkaç kez çalıştırdığımda, her seferinde olmasa da isteklerin farklı pod'lara yönlendirildiğini görüyorum. Bu durum, Kubernetes servislerinde varsayılan olarak kullanılan **round-robin** yük dengeleme algoritmasından kaynaklanır. Bu sayede servis, trafiği pod'lar arasında dengeli bir şekilde dağıtarak basit bir **load balancer** işlevi görür.

```

sena@k8s-master: ~/k8sfundamentals/service
GNU nano 7.2 serviceNodePort.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

```

Şimdi **kubectl apply -f serviceNodePort.yaml** komutu ile YAML dosyamızı deploy edelim. Bu servis tanımında **type** olarak **NodePort** seçildiğini görüyoruz.

kubectl get service komutu ile servisleri listelediğimizde, oluşturduğumuz **frontend** servisinin **NodePort** tipinde olduğunu görüyoruz.

Bu servis tipi sayesinde, Kubernetes iç ağı dışında kalan dış istemciler de servise erişebilir. Servisin **targetPort** olarak tanımlanan **80** numaralı porta gönderilen istekler, otomatik olarak **30571** gibi belirli bir port üzerinden Kubernetes node'larının dış ağ arabirimlerine yönlendirilir.

Kubernetes, tüm **worker node**'ların dış bacaklarında bu portu dinlemeye başlar. Böylece, herhangi bir worker node'un IP adresi üzerinden **http://<node-ip>:30571** şeklinde erişim sağladığınızda, istek **frontend** pod'larından birine yönlendirilmiş olur.

Not: Hangi node'a istek gönderildiğinin bir önemi yoktur, çünkü arka planda istek ilgili pod'a otomatik olarak yönlendirilir.

```
sena@k8s-master:~/k8sfundamentals/service$ kubectl get service
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
backend      ClusterIP   10.103.144.108 <none>       5000/TCP   18m
frontend     NodePort    10.98.34.149   <none>       80:30571/TCP 5s
kubernetes   ClusterIP   10.96.0.1     <none>       443/TCP    10d
sena@k8s-master:~/k8sfundamentals/service$
```

minikube service --url frontend komutu, **NodePort** tipi servis için bir tünel oluşturur ve bize farklı bir port üzerinden bu servise dışarıdan erişim imkânı sağlar.

Minikube, bu komut sayesinde local IP adresi ve dışa açılmış NodePort bilgisiyle birlikte erişim URL'sini oluşturur. Böylece tarayıcıdan veya **curl** komutu ile bu URL üzerinden doğrudan **frontend** servisine ulaşılabilir.

Bizim örneğimizde <http://192.168.49.2:30571> IP adresine gidilerek servise ulaşılabilir.

```
sena@k8s-master:~/k8sfundamentals/service$ minikube service --url frontend
http://192.168.49.2:30571
sena@k8s-master:~/k8sfundamentals/service$ minikube service list
-----
| NAMESPACE | NAME      | TARGET PORT | URL                               |
|-----|-----|-----|-----|
| default    | backend   | No node port |                                   |
| default    | frontend  | 80          | http://192.168.49.2:30571        |
| default    | kubernetes | No node port |                                   |
| kube-system | kube-dns  | No node port |                                   |
|-----|-----|-----|-----|
sena@k8s-master:~/k8sfundamentals/service$
```

curl <http://192.168.49.2:30571> komutu ile NodePort üzerinden frontend servisine başarılı bir şekilde istek gönderdik ve HTML yanıtını simüle etmiş olduk.

```
sena@k8s-master:~/k8sfundamentals/service$ curl http://192.168.49.2:30571
<html>
<body style="background-color:gray;">

<center><h1 style="color:darkblue;"> Version 1
  <p>K8S Fundamentals</p>
  <p>Pod: frontend-fb99cc5df-2jlj8</p></h1></center>

</body>
</html>sena@k8s-master:~/k8sfundamentals/service$
```


Üçüncü ve son servis tanımımızı da deploy edelim. Bu YAML dosyasında **type: LoadBalancer** olarak tanımlandığını görüyoruz.

Ancak bu yapı, **lokal bir Kubernetes cluster** üzerinde (örneğin Minikube veya kubeadm ile kurulan ortamlarda) **doğrudan çalışmaz**, çünkü LoadBalancer tipi servisler genellikle bir **cloud provider** (örn. AWS, GCP, Azure) tarafından otomatik olarak dış IP atamasıyla desteklenir.

Bu tür servisler, altyapı sağlayıcısı tarafından bir **external load balancer** oluşturularak çalışır. Örneğin: GKE (Google Kubernetes Engine), AKS (Azure Kubernetes Service) veya EKS (Amazon Elastic Kubernetes Service) ortamlarında bu servis tipi gerçek anlamda dış erişim sağlar.

Dolayısıyla, lokal ortamda **LoadBalancer** tipi bir servis oluşturduğunuzda dış IP ataması yapılmaz ve istekleri dışarıdan almanız mümkün olmaz.

```

sena@k8s-master: ~/k8sfundamentals/service
GNU nano 7.2 serviceLoadBalancer.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontendlb
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

```

Bunu minikube ile test edelim.

kubectl apply -f serviceLoadBalancer.yaml ile önce serviceLoadBalancer.yaml dosyanı apply edelim.

minikube tunnel komutu ile devam edelim. (Minikube, LoadBalancer tipi servislere dış IP atayamaz. Ama minikube tunnel komutu, bu tür servisler için sanal bir LoadBalancer sağlar.)

NOT: Terminal açık kalmalı, başka bir terminalde devam edeceğiz.

```

sena@k8s-master: ~/k8sfundamentals/service
sena@k8s-master:~/k8sfundamentals/service$ kubectl apply -f serviceLoadBalancer.yaml
service/frontendlb created
sena@k8s-master:~/k8sfundamentals/service$ minikube tunnel
[sudo] password for sena:
Status:
  machine: minikube
  pid: 3408678
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [frontendlb]
  errors:
    minikube: no errors
    router: no errors
    loadbalancer emulator: no errors

```

Sonrasında başka bir terminal açarak servisin durumunu kontrol ettim: **kubectl get service frontendlb**

curl <http://192.168.49.2> ile test ettim.

```

sena@k8s-master: ~
sena@k8s-master:~$ kubectl get service frontendlb
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
frontendlb    LoadBalancer  10.98.155.47   10.98.155.47   80:31259/TCP     58s
sena@k8s-master:~$ curl http://10.98.155.47
<html>
<body style="background-color:gray;">

<center><h1 style="color:darkblue;"> Version 1
  <p>K8S Fundamentals</p>
  <p>Pod: frontend-fb99cc5df-nr594</p></h1></center>

</body>
</html>sena@k8s-master:~$

```

kubectl get service komutu ile listelediğimizde, daha önce oluşturduğumuz **ClusterIP**, **NodePort** ve **LoadBalancer** tipinde servisleri görebiliyoruz.

```

sena@k8s-master: ~/k8sfundamentals/service
sena@k8s-master:~/k8sfundamentals/service$ kubectl get service
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
backend       ClusterIP     10.103.144.108 <none>         5000/TCP         3h40m
frontend      NodePort      10.98.34.149   <none>         80:30571/TCP     3h21m
frontendlb    LoadBalancer 10.98.155.47   <pending>      80:31259/TCP     49m
kubernetes    ClusterIP     10.96.0.1      <none>         443/TCP          10d
sena@k8s-master:~/k8sfundamentals/service$

```

Şimdiye kadar servisleri hep **declarative** yani **.yaml** dosyaları ile tanımlayarak oluşturduk. Bu dosyalarda **type** alanı ile servisin türünü **ClusterIP**, **NodePort** ya da **LoadBalancer** olarak belirtmiştik.

Ancak servisleri sadece YAML dosyası ile değil, **imperative** yani doğrudan komut satırı üzerinden de oluşturmak mümkündür.

Aşağıdaki komutla **backend** isimli deployment için bir **ClusterIP** servisi oluşturabiliriz:

kubectl expose deployment backend --type=ClusterIP --name=backend

Not: --type belirtmediğinizde varsayılan olarak zaten ClusterIP kullanılır.

```
sema@k8s-master:~/k8sfundamentals/service$ kubectl expose deployment backend --type=ClusterIP --name=backend
service/backend exposed
sema@k8s-master:~/k8sfundamentals/service$ kubectl get service
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
backend      ClusterIP   10.106.26.113 <none>         5000/TCP    5s
kubernetes   ClusterIP   10.96.0.1     <none>         443/TCP    10d
sema@k8s-master:~/k8sfundamentals/service$
```

backend için de aynı işlemleri yaptık. (nodeport için)

kubectl expose deployment frontend --type=NodePort --name=frontend

```
sema@k8s-master:~/k8sfundamentals/service$ kubectl expose deployment frontend --type=NodePort --name=frontend
service/frontend exposed
sema@k8s-master:~/k8sfundamentals/service$ kubectl get service
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
backend      ClusterIP   10.106.26.113 <none>         5000/TCP    3m39s
frontend     NodePort    10.96.188.76   <none>         80:31892/TCP 3s
kubernetes   ClusterIP   10.96.0.1     <none>         443/TCP    10d
sema@k8s-master:~/k8sfundamentals/service$
```

kubectl get endpoints komutu ile mevcut servislerin hangi pod IP'lerine yönlendirme yaptığı (yani hangi endpoint'lere sahip olduğu) görüntülenebilir. Servislerin arka planda hangi pod'larla ilişkili olduğunu göstermesi açısından oldukça önemlidir.

```
sema@k8s-master:~/k8sfundamentals/service$ kubectl get endpoints
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME         ENDPOINTS                                     AGE
backend      10.244.0.30:5000,10.244.0.32:5000,10.244.0.33:5000 4m36s
frontend     10.244.0.29:80,10.244.0.31:80,10.244.0.34:80        60s
kubernetes   192.168.49.2:8443                                   10d
sema@k8s-master:~/k8sfundamentals/service$
```

- **backend** servisi 3 farklı pod'un 5000 numaralı portuna yönlendirilmiş.
- **frontend** servisi ise 3 farklı pod'un **80** numaralı portuna yönlendirilmiş.

(Bu IP adresleri, servis tanımındaki selector ile eşleşen pod'ların IP adresleridir.)

kubectl describe endpoints frontend komutu ile frontend servisine ait endpoint'ler hakkında daha detaylı bilgi alıyoruz:

- **Subsets** altında, servis tarafından yönlendirilen pod IP'leri listeleniyor.
- Bu IP'ler pod'ların **internal IP**'leridir ve servis, gelen trafiği bu IP'lere yönlendirir.
- Port olarak **80/TCP** gösterilmiş, bu da targetPort alanındaki değeri ifade eder.
- **NotReadyAddresses** boş olduğu için, bu pod'ların hepsi ready durumda.

```

sena@k8s-master:~/k8sfundamentals/service$ kubectl describe endpoints frontend
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
Name:          frontend
Namespace:     default
Labels:        endpoints.kubernetes.io/managed-by=endpoint-controller
               team=development
Annotations:   endpoints.kubernetes.io/last-change-trigger-time: 2025-06-30T12:33:52Z
Subsets:
  Addresses:          10.244.0.29,10.244.0.31,10.244.0.34
  NotReadyAddresses:  <none>
  Ports:
    Name      Port      Protocol
    ----      -
    <unset>   80       TCP
Events:      <none>

```

17. Liveness probes

Pod yaşam döngüsü sırasında, bir pod oluşturulduğunda bu pod'un koştığı node üzerindeki **kubelet** bileşeni devreye girer. Kubelet, pod'un içindeki container'ları oluşturur ve daha sonra bu container'ları izlemeye başlar.

Eğer **restartPolicy** değeri **Always** ya da **OnFailure** olarak ayarlanmışsa ve container içindeki uygulama çalışmayı durdurur ya da yanıt veremez hale gelirse, **kubelet** bunu algılar ve ilgili container'ı yeniden başlatır.

Ancak bazı durumlarda, container hala çalışır durumda görünse de içindeki uygulama (örneğin bir web sunucusu) işlevini yerine getirmiyor olabilir.

Örneğin:

- Web uygulaması barındıran bir pod'umuz olduğunu düşünelim.
- HTTP daemon (örn. Nginx) hala çalışıyor gibi görünüyor, bu nedenle container "Running" durumunda.
- Ancak servis, içerik sunmayı durdurmuş olabilir.

Bu durumda kubelet, container çalıştığı için bir sorun olduğunu fark edemez ve pod'u yeniden başlatmaz.

İşte bu gibi senaryolarda **liveness probe** devreye girer.

Liveness probe, pod'un gerçekten "canlı" olup olmadığını anlamaya yarayan bir mekanizmadır. Belirli aralıklarla pod'a yapılan HTTP, TCP ya da komut tabanlı kontroller ile uygulamanın gerçekten yanıt verip vermediği kontrol edilir. Eğer uygulama başarısız bir şekilde yanıt verirse, kubelet bu container'ı yeniden başlatır.

17.1 Uygulama

/k8sfundamentals/liveready altındaki **liveness.yaml** dosyamızı inceleyelim.

Burada 3 tane pod tanımı yapılmış, (liveness-http, liveness-exec ve goproxy)

```

sen@k8s-master: ~/k8sfundamentals/liveready
GNU nano 7.2 liveness.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/liveness
    args:
    - /server
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
        httpHeaders:
        - name: Custom-Header
          value: Awesome
      initialDelaySeconds: 3
      periodSeconds: 3
---
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-exec
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/busybox
    args:
    - /bin/sh
    - -c
    - touch /tmp/healthy; sleep 30; rm -rf /tmp/healthy; sleep 600

```

```

---
apiVersion: v1
kind: Pod
metadata:
  name: goproxy
  labels:
    app: goproxy
spec:
  containers:
  - name: goproxy
    image: k8s.gcr.io/goproxy:0.1
    ports:
    - containerPort: 8080
    livenessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20

```

liveness-http: Burada liveness adlı container için, /server komutu çalıştırılarak başlatılan bir pod tanımlanmış; Kubernetes bu container'ın canlılığını /healthz path'ine 8080 portundan HTTP GET isteği göndererek ve **Custom-Header: Awesome** kullanarak her 3 saniyede bir kontrol edecek şekilde livenessProbe yapılandırmasıyla izleyecek şekilde ayarlanmıştır.

liveness-exec: Burada ise **liveness** container'ı için **/bin/sh -c 'touch /tmp/healthy; sleep 30; rm -rf /tmp/healthy; sleep 600'** komutu ile çalışan bir pod tanımlanmış; Bu container'ın canlılığını 5 saniye sonra başlayıp her 5 saniyede bir **/tmp/healthy** dosyasını cat ile okuyarak kontrol eder, dosya silindikten sonra probe başarısız olur ve container yeniden başlatılır.

goproxy: Son olarak **goproxy** adlı container için k8s.gcr.io/goproxy:0.1 imajı kullanılarak 8080 portundan TCP bağlantısı kurulabilen bir pod tanımlanmış; bu container'ın canlılığını 15 saniye sonra başlayıp her 20 saniyede bir 8080 portuna TCP bağlantısı kurmaya çalışarak tcpSocket türünde liveness kontrolüyle izlemektedir.

kubectl apply -f liveness.yaml komutu ile .yaml dosyası apply edilerek **watch kubectl get pods** komutu ile canlı olarak izlenebilir.

18. Readiness probes

Örnek bir senaryo üzerinden gidelim: Bir frontend uygulamanın var ve bu uygulamanın yeni versiyonunu (v2) production ortamına dağıtıyorsun. Bu uygulama frontend-deployment üzerinden 3 adet pod olarak dağıtılıyor: frontend-pod-1, frontend-pod-2 ve frontend-pod-3.

Pod'lar başlatıldığında, container'lar teknik olarak "çalışıyor" gözüküyor olabilir, ancak uygulamanın içinde yer alan servislerin (örneğin: bir HTTP sunucusunun ayaklanması, önbelleğin yüklenmesi veya konfigürasyonun okunması gibi) hazır hale gelmesi birkaç saniye sürebilir.

İşte bu aşamada LoadBalancer veya Service, pod'ların çalıştığını görüp trafik yönlendirmeye başlar. Eğer uygulama iç servisler henüz hazır değilse, gelen istekler başarısız olur ve son kullanıcılar hatalarla karşılaşır.

Kubernetes, pod'u Running olarak görür ama uygulama hazır olmadığı için kullanıcılara hata döner. Load balancer sağlıklı bir pod'a trafik gönderir.

Bu gibi durumları engellemek için **readiness probe** kullanılır. Readiness probe, pod'un dış dünyadan (örneğin load balancer veya service) trafik alabilecek kadar **hazır olup olmadığını** kontrol eder.

Readiness probe sayesinde Kubernetes, pod tamamen hazır olana kadar onu **servis discovery sistemine dahil etmez**, yani trafik almasına izin vermez. Uygulama gerçekten hazır olduğunda probe başarılı olur ve pod trafiğe açılır. Böylece kullanıcıya hata gitmez, sistem daha kararlı çalışır.

Readiness probes Nedir?

Kubelet, bir container'ın **trafiği kabul etmeye hazır olup olmadığını** anlamak için readiness probe kullanır.

Bir pod'un içinde birden fazla container olabilir ve pod'un **"hazır" (Ready)** olarak kabul edilmesi için tüm container'ların readiness probe'lerinin başarılı olması gerekir.

Eğer bir container çalışıyor ama uygulama henüz servis verecek durumda değilse, Kubernetes bu pod'a trafik yönlendirmez.

Readiness probe sayesinde pod tamamen hazır olmadan önce **service arkasına eklenmez**.

terminationGracePeriodSeconds: 30 → Pod'a düzgünce kapanması için verilen mühlet.

19. Resource limits

Container'lar, siz aksi bir sınır belirtmediğiniz sürece, çalıştıkları host'un CPU ve memory kaynaklarına sınırsız şekilde erişebilirler.

Örneğin, 8 GB RAM'e sahip bir sistemde bir container çalıştırdığınızda, içindeki uygulama bu 8 GB RAM'in tamamını ve tüm CPU gücünü kullanabilir. Bu durum, özellikle Kubernetes gibi aynı anda yüzlerce container'ın çalıştığı ortamlarda ciddi sorunlara yol açabilir.

Eğer bir pod, bulunduğu node'daki kaynakları aşırı kullanıyorsa, aynı node üzerinde çalışan diğer pod'ların düzgün çalışmasını engelleyebilir.

Bu nedenle, bir pod içindeki container'ların ne kadar CPU ve ne kadar memory kullanabileceğini kısıtlamamız gerekir.

İşte bu noktada, pod tanımlarında kullandığımız **requests** ve **limits** seçenekleri bize bu kontrolü sağlar.

CPU

Kubernetes ortamında her **worker node**'un sahip olduğu belirli miktarda CPU kaynağı vardır. Bu kaynaklar; cloud ortamlarında genellikle **vCPU (virtual CPU)** olarak, fiziksel ortamlarda ise **core** veya **hyper-thread** şeklinde tanımlanır.

Kubernetes, 1 CPU değerini genellikle şu kaynaklara eşit kabul eder:

- 1 AWS vCPU
- 1 GCP Core
- 1 Azure vCore
- 1 Hyper-threaded core (bare-metal)

CPU Kaynağını Nasıl Belirleriz?

CPU miktarı tam sayı (örn: "1", "1000m") veya kesirli (örn: "0.1", "100m") olarak tanımlanabilir:

Anlamı	Tanım
1 CPU core	cpu: "1" veya cpu: "1000m"
1 CPU'nin %10'u	cpu: "0.1" veya cpu: "100m"

Memory

Kubernetes ortamında bir container, cluster'da o anda kullanılabilir memory varsa kendi **requests.memory** değerinden fazlasını geçici olarak kullanabilir. Ancak bir container'ın **limits.memory** ile belirtilen değerin üzerine çıkmasına asla izin verilmez.

Limit Aşıldığında Ne Olur?

Eğer container, **limits.memory** değerini aşarsa:

- Kubernetes container'ı **sonlandırır (OOMKilled)**.
- Container, bu durumdan sonra yeniden başlatılabilir.
- Bu davranış, sistemin diğer pod'ları korumak için verdiği otomatik bir tepkidir.

requests ve limits Ne İşe Yarar?

- **requests.memory:** Pod'un başlatılabilmesi için minimum gereken memory miktarıdır. Scheduler bu değere göre node seçer.
- **limits.memory:** Pod'un kullanabileceği maksimum memory miktarıdır. Aşıldığında container sonlandırılır.

20. Environment variable

Şöyle bir senaryo düşünelim: Bizler bir web uygulaması yazıyoruz ve bu web uygulaması bir veritabanı uygulamasına bağlanarak verilerini burada saklıyor. Bu uygulamanın bağlandığı veritabanı sunucu adresini ve kullanıcı adı, şifre bilgilerini de bu uygulamanın içerisine gömdük diyelim. Daha sonra bu uygulamayı bir container image'i haline getirdik ve artık istediğimiz container platformunda çalıştırmaya hazırız.

Fakat bu senaryo da iki sıkıntımız var:

1) Biz bu veritabanı bağlantı bilgilerini bu container image'i içerisine hard-coded olarak yazdık. Yani bir şekilde bu container image'i istediğimiz birinin eline geçerse bu image'in içerisinde bu bilgiler expose olabilir.

2) Bir diğer sıkıntımız da şu: ben bu image'dan container oluşturmak istediğim zaman bu container her defasında aynı veritabanına aynı kullanıcı adı ve şifre ile bağlanmaya çalışacak. Benim veritabanı adresim, test ortamında ayrı, proud ortamında ayrı olabilir. Ben bu kullanıcı adı, şifre bilgilerini zaman içerisinde güncellemiş olabilirim.

Bu tarz, ortama göre değişebilen ve hassas bilgileri container image'ları içerisine gömmek oldukça fazla sorun çıkarıyor. Hard-coded olarak gömmek yerine çalıştırdıkları sistemden okuyacakları şekilde bir değişken olarak tanımlarız. Yani environment variable'dan okuması için tanım yaparız.

Peki bunları pod tanımlarımıza nasıl ekleriz?

/k8sfundamentals/pod altındaki **podenv.yaml** dosyamızı incelediğimiz zaman **env** anahtarı ile tanımlandığını list şeklinde görüyoruz.

```

sena@k8s-master: ~/k8sfundamentals/pod
GNU nano 7.2 podenv.yaml
apiVersion: v1
kind: Pod
metadata:
  name: envpod
  labels:
    app: frontend
spec:
  containers:
  - name: envpod
    image: ozgurozturknet/env:latest
    ports:
    - containerPort: 80
    env:
    - name: USER
      value: "Ozgur"
    - name: database
      value: "testdb.example.com"

```

kubectl apply -f podenv.yaml ve **kubectl exec envpod -- printenv** komutları ile önce .yaml dosyamızı apply ettik ve bu pod içerisindeki tüm environment variable'ları listeledik.

```

sena@k8s-master: ~/k8sfundamentals/pod
sena@k8s-master:~/k8sfundamentals/pod$ kubectl apply -f podenv.yaml
pod/envpod created
sena@k8s-master:~/k8sfundamentals/pod$ kubectl exec envpod -- printenv
PATH=/usr/local/apache2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=envpod
HTTPD_PREFIX=/usr/local/apache2
HTTPD_VERSION=2.4.46
HTTPD_SHA256=740eddf6e1c641992b22359cab66e6325868c3c5e2e3f98faf349b61ecf41ea
HTTPD_PATCHES=
USER=Ozgur
database=testdb.example.com
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
HOME=/root
sena@k8s-master:~/k8sfundamentals/pod$

```

Bu bir web uygulaması podu ve normalde dış dünyadan buna erişebilmem için ya loadbalancer ya da nodeport şeklinde bir service tanımlamam gerekir. Fakat kubectl aracılığıyla herhangi bir deployment'a, pod'a ya da service'e kendi bilgisayarımdan tünel açarak o objenin portuna trafiğimi yönlendirebilirim. İşte testlerde hızlı şekilde objelerimize bağlanmak için kullanabildiğimiz bu özelliğe port forward denir.

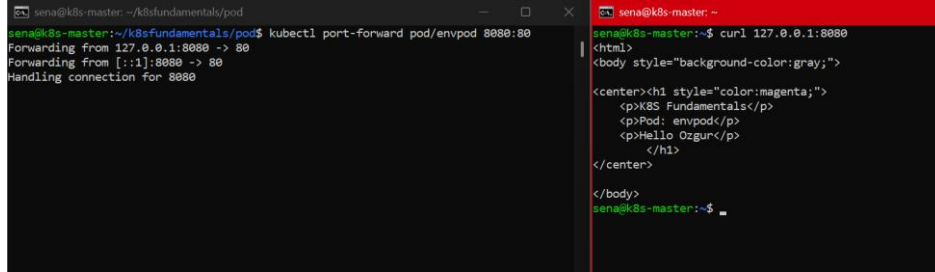
kubectl port-forward pod/envpod 8080:80 komutu ile local makinemdeki localhost:8080 portunu envpod'un 80 portuna tünellemiş olduk. (geçicidir, bu işlem açık kaldığı sürece devam eder CTRL+C ile bağlantıyı sonlandırabilirsiniz)

```

sena@k8s-master: ~/k8sfundamentals/pod
sena@k8s-master:~/k8sfundamentals/pod$ kubectl port-forward pod/envpod 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80

```

Farklı bir shell açıp **curl 127.0.0.1:8080** komutu ile localhost 8080 portuna bağlandık. İstek port-forward üzerinden pod'un 80 portuna yönlendirildi ve pod'daki web uygulamasının döndürdüğü HTML yanıtı **curl** terminalde görmüş olduk. Böylece pod içeriğini dışarıya servis açmadan lokalden test etmiş olduk.



```
sena@k8s-master: ~/k8sfundamentals/pod
sena@k8s-master:~/k8sfundamentals/pod$ kubectl port-forward pod/envpod 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080

sena@k8s-master: ~
sena@k8s-master:~$ curl 127.0.0.1:8080
<html>
<body style="background-color:gray;">

<center><h1 style="color:magenta;">
  <p>K8S Fundamentals</p>
  <p>Pod: envpod</p>
  <p>Hello Ozgur</p>
</h1>
</center>

</body>
sena@k8s-master:~$
```

Sonuç olarak **port-forward** ile pod içindeki uygulamaya lokalden erişip tanımladığımız environment variable'ın uygulamaya geçtiğini ve web sayfasında 'Hello Ozgur' olarak döndüğünü doğrulamış olduk.

KAYNAKLAR

(GitHub repository'si: [GitHub - aytitech/k8sfundamentals](#): Ayti.tech tarafından sunulan "Kubernetes Temelleri" eğitiminde gerekli dosyaların barındırılması için oluşturulan GitHub repository'si.)

Kurulum adımlarını içeren resmi sayfası: [Installing kubeadm | Kubernetes](#)

Kurulum adımlarını içeren Özgür Öztürk Github Reposu: [k8sfundamentals/setup at main · aytitech/k8sfundamentals · GitHub](#)

<https://www.cncf.io/> → Kubernetes'in bağlı olduğu Cloud Native Computing Foundation'ın resmi sitesidir. (Kubernetes'in hangi projelerle entegre olduğunu ve resmi topluluk desteği incelenebilir.)

<https://kubernetes.io/> → Kubernetes'in resmi ana sayfasıdır. (Güncel sürümler, eğitim içerikleri ve resmi yönlendirmeler için ana kaynaktır.)

<https://github.com/kubernetes/kubernetes> → Kubernetes'in kaynak kodlarının yer aldığı resmi GitHub reposudur. (Kod inceleme, katkı sağlama, issue takibi ve güncel geliştirmeleri izlemek için.)

<https://kubernetes.io/docs/home/> → Kubernetes dökümantasyon portalının ana giriş sayfasıdır. (Uygulama rehberleri, bileşen açıklamaları ve yapılandırma örnekleri gibi teknik içeriklere ulaşmak için.)